

Documentation Wazuh

Installation, déploiement des agents &
configuration des alertes

Etudiante : Amélie Garnier

Enseignante : Samira Kherfella-Barchiche

Table des matières :

Partie 1 :	9
Installation de Wazuh	9
Introduction	9
Présentation de Wazuh	9
Notre environnement de travail	9
Objectifs du projet	10
Enjeux et importance	10
Architecture déployée	11
Préparation du système	12
Étape 1 → Installation des outils essentiels	12
Étape 2 → Récupération de l'adresse IP	14
Étape 3 → Ajout du dépôt Wazuh	14
Étape 4 → Mise à jour du système	15
Génération des certificats SSL/TLS	15
Étape 5 → Téléchargement des outils	15
Étape 6 → Configuration des nœuds	16
Étape 7 → Génération des certificats	17
Étape 8 → Compression des certificats	17
Installation des composants Wazuh	18
Étape 9 → Installation des paquets principaux	18
Configuration de Wazuh Indexer	19
Étape 10 → Configuration réseau de l'Indexer	19
Étape 11 → Déploiement des certificats de l'Indexer	20
Étape 12 → Démarrage de Wazuh Indexer	22
Étape 13 → Initialisation du cluster de sécurité	22
Étape 14 → Test de l'installation de l'Indexer	23
Configuration de Wazuh Manager	23
Étape 15 → Démarrage de Wazuh Manager	23
Configuration de Filebeat	24
Étape 16 → Installation de Filebeat	24
Étape 17 → Configuration de Filebeat	24
Étape 18 → Configuration du keystore Filebeat	25
Étape 19 → Installation du template d'alertes	26
Étape 20 → Installation du module Wazuh pour Filebeat	27
Étape 21 → Déploiement des certificats Filebeat	27

Étape 22 → Démarrage de Filebeat	28
Étape 23 → Test de connexion Filebeat	28
Configuration de Wazuh Dashboard	29
Étape 24 → Configuration du Dashboard	29
Étape 25 → Déploiement des certificats du Dashboard	30
Étape 26 → Démarrage et vérification des services	33
Étape 27 → Accès à l'interface web Wazuh	36
Étape 28 → Configuration de l'adresse IP fixe	37
Conclusion partie 1	39
Partie 2 :	40
Intégration de deux machines dans WAZUH → Linux & Windows	40
Introduction	40
1. Intégration de la machine Debian Linux	40
1.1 Connexion à l'interface Web Wazuh	40
1.2 Ajout d'un nouvel agent	41
1.3 Sélection du système d'exploitation	41
1.4 Téléchargement et installation de l'agent	43
1.5 Démarrage de l'agent	44
1.6 Vérification de l'état de l'agent	44
1.7 Confirmation dans l'interface Web	45
2. Intégration de la machine Windows 10	46
2.1 Ajout d'un nouvel agent Windows	46
2.2 Sélection du système d'exploitation	48
2.3 Téléchargement et installation de l'agent	50
2.4 Démarrage du service	51
2.5 Confirmation dans l'interface Web	51
3. Vérification finale	53
3.1 Vue d'ensemble des agents	53
3.2 Surveillance des événements	54
3.3 Premières alertes	54
Conclusion partie 2	55
Partie 3 :	56
Configuration des alertes FIM et détection d'intrusions	56
Introduction	56
Qu'est-ce que le FIM ?	56

4. Configuration FIM de base pour Windows	57
4.1 Modification du fichier de configuration	57
Étape 1 → Accéder au fichier de configuration	58
Étape 2 → Ajouter la directive de surveillance	59
4.2 Personnalisation des règles de surveillance	59
4.3 Redémarrage du service de l'agent Wazuh	60
4.4 Résolution du problème de connexion à l'indexer	61
4.5 Solution	61
Étape 1 → Modifier la configuration du manager	61
Étape 2 → Corriger l'adresse IP	62
Étape 3 → Redémarrer le manager	62
Étape 4 → Vérification	63
4.6 Vérification du fonctionnement	63
5. Configuration FIM de base pour Linux	64
5.1 Modification du fichier de configuration	64
Étape 1 → Accéder au fichier de configuration	64
Étape 2 → Ajouter la directive de surveillance	65
5.2 Redémarrage de l'agent	66
Étape 3 → Redémarrer l'agent Wazuh	66
5.3 Vérification du fonctionnement	66
6. Configuration avancée → Mode Whodata	67
6.1 Qu'est-ce que Whodata ?	67
7. Configuration Whodata pour Windows	68
7.1 Modification de la configuration	68
Étape 1 → Éditer le fichier ossec.conf	68
Étape 2 → Redémarrer le service	68
7.2 Analyse des événements Whodata sous Windows	70
8. Configuration Whodata pour Linux	73
8.1 Prérequis → Installation des outils d'audit	73
Étape 1 → Installation d'auditd	74
Étape 2 → Installation du plugin audispd	74
Étape 3 → Redémarrage du service d'audit	74
8.2 Configuration de Whodata	75
Étape 4 → Modification de la configuration Wazuh	75
Étape 5 → Redémarrage de l'agent Wazuh	76
8.3 Vérification du fonctionnement	76
8.4 Analyse des événements Whodata	76

Partie 3 :	80
Détection d'attaque par force brute et Active Response	80
Introduction et objectifs	80
Architecture et prérequis	80
10. Préparation de la machine victime	83
10.1 Installation et vérification de SSH	83
10.2 Vérification du port 22	84
10.3 Configuration de l'authentification par mot de passe	85
10.4 Collecte des informations de la machine victime	86
10.5 Récapitulatif de la configuration de la machine victime	87
11. Configuration de l'Active Response sur Wazuh Manager	87
11.1 Vérification de la commande firewall-drop	88
11.2 Configuration de la réponse active	89
11.3 Redémarrage du Wazuh Manager	91
12. Préparation de la machine attaquante	92
12.1 Mise à jour du système	92
12.2 Installation de Hydra	93
12.3 Installation de pwgen et génération des mots de passe	94
13. Test de connectivité SSH	95
14. Lancement de l'attaque par force brute	96
14.1 Commande Hydra	96
14.2 Observation de l'exécution	98
14.3 Analyse des résultats dans le Dashboard Wazuh	98
14.4 Vérification du blocage de l'IP attaquante	101
Conclusion de la partie 3	103
Annexe → Troubleshooting et Commandes Utiles	105
Problèmes Courants et Solutions	105
1. Erreur « Connection refused » - Indexer	105
2. Agent déconnecté (Status: Never connected)	106
3. Certificats SSL invalides	107
4. Dashboard inaccessible (ERR_CONNECTION_REFUSED)	108
5. Active Response ne bloque pas l'IP	109
Commandes de Diagnostic Essentielles	110

Table des illustrations :

Screenshot 1 - Installation de curl pour télécharger les fichiers avec apt install curl	12
Screenshot 2 - Installation de GPG pour la gestion des clés de sécurité avec apt install curl gpg	13
Screenshot 3 - Installation de Micro avec apt install micro	13
Screenshot 4 - Afficher les interfaces réseau et leurs adresses IP avec ip a	14
Screenshot 5 - Ajout de la clé GPG officielle Wazuh pour vérifier l'authenticité des paquets avec curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH gpg --no-default-keyring --keyring gnupg- ring:/usr/share/keyrings/wazuh.gpg --import && chmod 644 /usr/share/ke	14
Screenshot 6 - Ajout du dépôt Wazuh à la liste des sources APT avec echo "deb [signed- by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/stable/main " tee -a /etc/apt/sources.list.d/wazuh.list	14
Screenshot 7 - Mise à jour de la base de données des paquets et installation des dépendances requises avec apt update && apt upgrade && sudo apt-get install debconf adduser procps curl gnupg apt-transport-https debhelper libcap2-bin	15
Screenshot 8 - Téléchargement du script de génération de certificats avec curl -sO https://packages.wazuh.com/4.8/wazuh-certs-tool.sh && curl -sO https://packages.wazuh.com/4.8/config.yml	15
Screenshot 9 - Vérification de la présence des fichiers avec dir	16
Screenshot 10 - Édition du fichier de configuration avec micro /config.yml	16
Screenshot 11 - Affichage du contenu pour vérification avec cat config.yml	17
Screenshot 12 - Rendre le script exécutable avec bash ./wazuh-certs-tool.sh -A	17
Screenshot 13 - Création de l'archive TAR avec tar -cvf ./wazuh-certificates.tar -C ./wazuh-certificates/ .	18
Screenshot 14 - Suppression du répertoire source (optionnel) avec rm -rf ./wazuh-certificates	18
Screenshot 15 - Installation de Wazuh Indexer, Manager et Dashboard avec apt install wazuh-indexer wazuh-manager wazuh-dashboard -y	18
Screenshot 16 - Modification de "network.host" avec notre adresse IP	20
Screenshot 17 - Création du répertoire pour les certificats avec mkdir /etc/wazuh-indexer/certs	20
Screenshot 18 - Extraction des certificats depuis l'archive avec tar -xf ./wazuh-certificates.tar -C /etc/wazuh- indexer/certs/ ./node-1.pem ./node-1-key.pem ./admin.pem ./admin-key.pem ./root-ca.pem	21
Screenshot 19 - Renommage des certificats et configuration des permissions de sécurité	21
Screenshot 20 - Attribution de la propriété à l'utilisateur wazuh-indexer avec chown -R wazuh-indexer:wazuh-indexer /etc/wazuh-indexer/certs	21
Screenshot 21 - Activation et démarrage de wazuh-indexer avec systemctl daemon-reload ; systemctl enable wazuh- indexer ; systemctl start wazuh-indexer	22
Screenshot 22 - Affichage du contenu du répertoire /usr/share/wazuh-indexer/bin/ avec ls -la	22
Screenshot 23 - Initialisation des composants de sécurité d'OpenSearch avec /usr/share/wazuh-indexer/bin/indexer- security-init.sh	22
Screenshot 24 - Test de connexion à l'API de l'Indexer avec sudo curl -k -u admin:admin https://192.168.111.62:9200	23
Screenshot 25 - Test de listage des nœuds du cluster avec curl -k -u admin:admin https://192.168.111.62:9200/_cat/nodes?v	23
Screenshot 26 - Activation et démarrage du gestionnaire wazuh avec : systemctl daemon-reload ; systemctl enable wazuh-manager ; systemctl start wazuh-manager	23
Screenshot 27 - Vérification de l'installation et du fonctionnement de wazuh-manager avec systemctl status wazuh- manager	24
Screenshot 28 - Installation du paquet Filebeat avec apt install filebeat	24
Screenshot 29 - Téléchargement du fichier de configuration Wazuh pour Filebeat avec curl -so /etc/filebeat/filebeat.yml https://packages.wazuh.com/4.8/tpl/wazuh/filebeat/filebeat.yml	24
Screenshot 30 - Vérification de la présence des fichiers filebeat avec la commande ls -la	25
Screenshot 31 - Modification de la section output.elasticsearch avec notre adresse IP	25
Screenshot 32 - Création du keystore avec filebeat keystore create	26
Screenshot 33 - Ajout du nom d'utilisateur et du mot de passe echo admin filebeat keystore add username --stdin - force ; echo admin filebeat keystore add password --stdin -force	26
Screenshot 34 - Téléchargement du template JSON avec curl -so /etc/filebeat/wazuh-template.json https://raw.githubusercontent.com/wazuh/wazuh/v4.8.2/extensions/elasticsearch/7.x/wazuh-template.json	26
Screenshot 35 - Attribution des permissions de lecture avec chmod go+r /etc/filebeat/wazuh-template.json	26
Screenshot 36 - Installation du module Wazuh pour Filebeat avec curl -s https://packages.wazuh.com/4.x/filebeat/wazuh-filebeat-0.4.tar.gz tar -xvz -C /usr/share/filebeat/module	27

Screenshot 37 - Création du répertoire des certificats avec <code>mkdir /etc/filebeat/certs</code>	27
Screenshot 38 - Extraction, renommage des certificats et configurations des permissions de Filebeat	28
Screenshot 39 - Recharge, activation et démarrage de Filebeat	28
Screenshot 40 - Test de connexion avec filebeat test output	29
Screenshot 41 - Modification de l'adresse IP « opensearch.hosts » par notre adresse IP	30
Screenshot 42 - Création du répertoire des certificats avec <code>mkdir /etc/wazuh-dashboard/certs</code>	31
Screenshot 43 - Extraction, renommage des certificats, configurations des permissions et changement du propriétaire du dashboard	32
Screenshot 44 - Vérification du contenu avec <code>cat config.yml</code>	33
Screenshot 45 - Rechargement, activation et démarrage de wazuh-dashboard	33
Screenshot 46 - Vérification du Dashboard avec <code>systemctl status wazuh-dashboard</code>	34
Screenshot 47 - Vérification du Manager avec <code>systemctl status wazuh-manager</code>	35
Screenshot 48 - Vérification de l'Indexer avec <code>systemctl status wazuh-indexer</code>	35
Screenshot 49 - Première connexion à l'interface web Wazuh	36
Screenshot 50 - Interface web de Wazuh	37
Screenshot 51 - Configuration d'une adresse IP fixe	38
Screenshot 52 - Interface web de Wazuh sur la debian client	41
Screenshot 53 - Onglet permettant de "deploy new agent"	41
Screenshot 54 - Procédure pour déployer un nouvel agent (1/3)	42
Screenshot 55 - Procédure pour déployer un nouvel agent (2/3)	42
Screenshot 56 - Procédure pour déployer un nouvel agent (3/3)	43
Screenshot 57 - Téléchargement et installation de l'agent avec <code>wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.13.1-1_amd64.deb</code> && <code>sudo WAZUH_MANAGER='192.168.111.62' WAZUH_AGENT_GROUP='default' WAZUH_AGENT_NAME='Wazuh-Secours' dpkg -i w</code>	43
Screenshot 58 - Démarrage et activation de l'agent Wazuh avec <code>systemctl daemon-reload ; systemctl enable wazuh-agent ; systemctl start wazuh-agent</code>	44
Screenshot 59 - Vérification de l'état de l'agent avec <code>systemctl status wazuh-agent</code>	44
Screenshot 60 - Confirmation du bon fonctionnement de l'agent Wazuh	45
Screenshot 61 - Confirmation de l'ajout de notre agent sur l'interface web Wazuh	46
Screenshot 62 - Interface web de Wazuh sur Windows10	46
Screenshot 63 - Navigation vers "Gestion des agents" > "Résumé"	47
Screenshot 64 - Interface permettant de "déployer un nouvel agent"	47
Screenshot 65 - Procédure pour déployer un nouvel agent (1/3)	48
Screenshot 66 - Procédure pour déployer un nouvel agent (2/3)	49
Screenshot 67 - Procédure pour déployer un nouvel agent (3/3)	50
Screenshot 68 - Téléchargement et installation de l'agent avec <code>Invoke-webrequest -uri https://packages.Wazuh.com/4.x.windows/wazuh-agent-4.13.1-1.msi -OutFile \$env:tmp\wazuh-agent ; msixec.exe /i \$env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.111.62' WAZUH_AGENT_GRO</code>	50
Screenshot 69 - Démarrage du service avec <code>NET START Wazuh</code>	51
Screenshot 70 - Vérification que notre WIN10 est bien déployée comme nouvel agent	52
Screenshot 71 - Validation du déploiement de notre nouvel agent Win10	52
Screenshot 72 - Interface web depuis notre VM serveur	53
Screenshot 73 - Confirmation que les deux agents sont bien déployés	54
Screenshot 74 - Déplacement vers le répertoire contenant <code>ossec-agent</code>	58
Screenshot 75 - Nous insérons l'alerte avec <code><directories check_all="yes" realtime="yes" report_changes="yes"></code>	59
Screenshot 76 - Redémarrage du service Wazuh via le Gestionnaire de services Windows (<code>services.msc</code>)	60
Screenshot 77 - Erreur de connexion à l'indexer OpenSearch dans l'interface Wazuh	61
Screenshot 78 - Modification du fichier avec <code>micro /var/ossec/etc/ossec.conf</code>	61
Screenshot 79 - Modification du bloc <code><indexer></code> dans <code>ossec.conf</code> pour corriger l'adresse IP du serveur	62
Screenshot 80 - Redémarrage du service Wazuh Manager avec <code>systemctl restart wazuh-manager</code>	62
Screenshot 81 - Vérification du fonctionnement du Manager Wazuh avec <code>systemctl status wazuh-manager</code>	63
Screenshot 82 - Liste des événements FIM détectés dans le tableau de bord Wazuh	64
Screenshot 83 - Détails d'un événement FIM montrant le type d'opération, les checksums et les métadonnées du fichier	64
Screenshot 84 - Modification du fichier de configuration avec <code>/var/ossec/etc/ossec.conf</code>	64

Screenshot 85 - Modification de ossec.conf pour ajouter des éléments à surveiller	65
Screenshot 86 - Redémarrage de l'agent Wazuh et vérification de son fonctionnement avec systemctl restart wazuh-agent ; systemctl restart wazuh-agent	66
Screenshot 87 - Création d'un fichier de test pour vérifier la surveillance FIM	66
Screenshot 88 - Événements FIM générés suite à la création du fichier de test	67
Screenshot 89 - Détails complets de l'événement FIM incluant les empreintes cryptographiques (MD5, SHA1, SHA256)	67
Screenshot 90 - Modification du fichier ossec.conf avec l'ajout du répertoire à monitorer	68
Screenshot 91 - Redémarrage du service Wazuh	69
Screenshot 92 - Ajout d'un fichier texte de test	69
Screenshot 93 - Liste des événements Whodata dans le tableau de bord Wazuh	70
Screenshot 94 - Informations détaillées d'un événement Whodata incluant le SID et le nom d'utilisateur Windows	71
Screenshot 95 - Détails étendus de l'événement Whodata montrant le processus, le PID, le chemin de l'exécutable et le type d'accès (1/2)	72
Screenshot 96 - Détails étendus de l'événement Whodata montrant le processus, le PID, le chemin de l'exécutable et le type d'accès (2/2)	73
Screenshot 97 - Installation du service auditd pour la surveillance Whodata avec apt get install auditd -y	74
Screenshot 98 - Installation du plugin audispd-plugins avec apt-get install audispd-plugins	74
Screenshot 99 - Redémarrage du service d'audit avec systemctl restart auditd	74
Screenshot 100 - Configuration Whodata dans le fichier ossec.conf sous Linux	75
Screenshot 101 - Redémarrage et vérification du fonctionnement de l'agent Wazuh avec systemctl restart wazuh-agent ; systemctl status wazuh-agent	76
Screenshot 102 - Création d'un fichier "test" avec micro text-who.txt	76
Screenshot 103 - Liste des événements Whodata détectés dans le tableau de bord	77
Screenshot 104 - Détails complets d'un événement Whodata sous Linux incluant UID, PID, PPID et GID	78
Screenshot 105 - Informations étendues montrant la commande exécutée, les arguments et le contexte d'exécution	79
Screenshot 106 - Vérification du fonctionnement de SSH avec systemctl status ssh	83
Screenshot 107 - Vérification de l'écoute du service SSH sur le port 22 avec ss -tulpn grep :22	85
Screenshot 108 - cat /etc/ssh/sshd_config	85
Screenshot 109 - Redémarrage du service SSH avec systemctl restart sshd	86
Screenshot 110 - Récupération de l'adresse IP avec hostname -I	86
Screenshot 111 - Récupération du nom d'utilisateur avec whoami	86
Screenshot 112 - Affichage du fichier de configuration principal de Wazuh avec cat /var/ossec/etc/ossec.conf	88
Screenshot 113 - Vérification de la présence de la commande firewall-drop	88
Screenshot 114 - Édition du fichier ossec.conf pour ajouter l'Active Response avec micro /var/ossec/etc/ossec.conf	89
Screenshot 115 - Configuration complète de l'Active Response dans ossec.conf	90
Screenshot 116 - Redémarrage et vérification du fonctionnement de Wazuh-manager avec systemctl restart wazuh-manager ; systemctl status wazuh-manager	91
Screenshot 117 - Mise à jour du système avec apt update && apt upgrade -y	92
Screenshot 118 - Installation de Hydra avec apt install hydra -y	94
Screenshot 119 - Installation de pwgen avec apt install pwgen -y	94
Screenshot 120 - Génération de 10 mots de passe aléatoires de 8 caractères avec pwgen 8 10 > password-list.txt	94
Screenshot 121 - Vérification du contenu du fichier avec cat password-list.txt	95
Screenshot 122 - Test de connectivité SSH depuis la machine attaquante vers la machine victime avec ssh debian-wazu@192.168.111.63	96
Screenshot 123 - Lancement de l'attaque avec hydra -l debian-wazu -P password-list.txt 192.168.111.63 ssh	97
Screenshot 124 - Dashboard indiquant l'attaque de brute force	99
Screenshot 125 - Application du filtre Authentication failure	100
Screenshot 126 - Interface Events pour obtenir plus d'informations sur l'attaque	100
Screenshot 127 - Onglet MITRE&ATTACK affiche technique T1110	101
Screenshot 128 - SSH bloqué pendant l'Active Response avec ssh -o ConnectTimeout=5 debian-wazu@192.168.111.63	101
Screenshot 129 - Ping bloqué pendant l'Active Response avec ping -c 192.168.111.63	101
Screenshot 130 - Vérification du déblocage automatique de l'adresse IP après 3 minutes	102

Partie 1 :

Installation de Wazuh

Introduction

Présentation de Wazuh

Wazuh est une solution open-source puissante et flexible de gestion des informations et des événements de sécurité (SIEM - Security Information and Event Management). Cette plateforme unifiée permet aux organisations de surveiller leur infrastructure informatique en temps réel, détecter les menaces, répondre aux incidents de sécurité et assurer la conformité réglementaire.

Grâce à ses composants principaux que sont le serveur (Manager), l'indexeur (basé sur OpenSearch), les agents et le tableau de bord (Dashboard), Wazuh offre une surveillance complète et approfondie de l'infrastructure. La solution excelle dans plusieurs domaines critiques :

- **Détection des intrusions (IDS)** → Analyse comportementale et détection d'anomalies
- **Surveillance de l'intégrité des fichiers (FIM)** → Détection de modifications non autorisées
- **Réponse aux incidents** → Automatisation des actions de remédiation
- **Conformité réglementaire** → PCI-DSS, GDPR, HIPAA, CIS
- **Analyse de vulnérabilités** → Identification des failles de sécurité
- **Détection de malwares** → Intégration avec VirusTotal et autres moteurs

Notre environnement de travail

Nous avons choisi de déployer *Wazuh* dans un environnement virtualisé sur une machine virtuelle (VM) Debian 12, une distribution Linux stable et largement utilisée en entreprise. Cette approche nous permet de tester et valider la solution dans des conditions réalistes avant un éventuel déploiement en production.

Caractéristiques de notre VM :

- **Système d'exploitation** → Debian 12 (Bookworm) 64-bit
- **Mémoire RAM** → 8 GB
- **Processeur** → 4 cœurs
- **Espace disque** → 100 GB

Avant de commencer l'installation, nous nous assurons que notre système répond aux exigences suivantes :

- ✓ Un accès *root* ou *sudo* sur la machine Debian 12
- ✓ Une connexion Internet stable pour télécharger les paquets
- ✓ Les ports suivants disponibles → 443 (*Dashboard*), 9200 (*Indexer*), 1514-1515 (*Manager*)
- ✓ Le pare-feu configuré pour autoriser ces ports si nécessaire
- ✓ Les mises à jour système appliquées → *sudo apt update && sudo apt upgrade -y*

Toutes les commandes de ce guide sont exécutées en tant qu'utilisateur « **root** ». Les commandes n'auront de ce fait, pas besoin d'être précédées de « *sudo* ».

Objectifs du projet

Notre objectif principal est de mettre en place une infrastructure de sécurité fonctionnelle capable de surveiller et protéger un parc informatique. Plus spécifiquement, nous cherchons à :

1. **Déployer une architecture all-in-one** → Installation de tous les composants Wazuh (*Indexer*, *Manager*, *Dashboard*) sur un seul serveur pour simplifier la gestion
2. **Maîtriser l'installation manuelle** → Comprendre en profondeur chaque composant et leur interaction
3. **Sécuriser la plateforme** → Mettre en place les certificats SSL/TLS et configurer l'authentification
4. **Préparer l'ajout d'agents** → Configurer le serveur pour accueillir des agents de surveillance sur d'autres machines
5. **Documenter le processus** → Créer une documentation reproductible pour de futurs déploiements

Enjeux et importance

La mise en place d'un SIEM comme *Wazuh* répond à plusieurs enjeux majeurs de la cybersécurité moderne :

➤ **Enjeux techniques :**

- Centralisation de la surveillance de sécurité
- Corrélation d'événements provenant de sources multiples
- Réduction du temps de détection et de réponse aux incidents (MTTD/MTTR)
- Automatisation des processus de sécurité

➤ **Enjeux organisationnels :**

- Conformité aux réglementations (RGPD, NIS2, etc.)
- Réduction des coûts liés aux incidents de sécurité
- Amélioration de la posture de sécurité globale
- Formation aux outils de cybersécurité

➤ **Enjeux stratégiques :**

- Protection des actifs informationnels critiques
- Maintien de la confiance des parties prenantes
- Préparation aux cybermenaces émergentes
- Alignement avec les bonnes pratiques de l'industrie

Architecture déployée

Nous avons opté pour une architecture **all-in-one**, également appelée **single-node**, où un seul serveur héberge l'ensemble des composants *Wazuh*. Cette configuration est idéale pour :

- Les environnements de test et de développement
- Les petites infrastructures (moins de 100 agents)
- L'apprentissage et la formation
- Les démonstrations (POC - Proof of Concept)

Note → Pour les environnements de production à grande échelle, une architecture **multi-node** avec des serveurs dédiés pour chaque composant est recommandée pour garantir performances et haute disponibilité.

Composants installés :

1. **Wazuh Indexer** (basé sur *OpenSearch*)
 - **Rôle** → Stockage, indexation et recherche des données de sécurité
2. **Wazuh Manager**
 - **Rôle** → Cœur du système, analyse des événements, corrélation, alertes
3. **Filebeat**
 - **Rôle** → Agent de collecte et transmission des logs vers l'Indexer
4. **Wazuh Dashboard** (basé sur *OpenSearch Dashboards*)
 - **Rôle** → Interface web de visualisation et gestion

Préparation du système

Étape 1 → Installation des outils essentiels

Nous commençons par installer les outils nécessaires pour le déploiement de *Wazuh*. *Curl* nous permettra de télécharger les fichiers, *GPG* assurera la vérification des signatures cryptographiques, et *Micro* nous servira d'éditeur de texte moderne et intuitif.

Screenshot 1 - Installation de curl pour télécharger les fichiers avec apt install curl

```
root@debian-wazu:/home/debian-wazu# apt install curl
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  curl
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 316 ko dans les archives.
Après cette opération, 501 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 curl amd64 7.88
.1-10+deb12u14 [316 kB]
316 ko réceptionnés en 0s (21,0 Mo/s)
Sélection du paquet curl précédemment désélectionné.
(Lecture de la base de données... 154810 fichiers et répertoires déjà installés.
)
Préparation du dépaquetage de .../curl_7.88.1-10+deb12u14_amd64.deb ...
Dépaquetage de curl (7.88.1-10+deb12u14) ...
Paramétrage de curl (7.88.1-10+deb12u14) ...
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
root@debian-wazu:/home/debian-wazu#
```

Screenshot 2 - Installation de GPG pour la gestion des clés de sécurité avec apt install curl gpg

```

debian-wazu@debian-wazu: ~
root@debian-wazu:/home/debian-wazu# apt install curl gpg
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
curl est déjà la version la plus récente (7.88.1-10+deb12u14).
gpg est déjà la version la plus récente (2.2.40-1.1+deb12u1).
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
root@debian-wazu:/home/debian-wazu#

```

Screenshot 3 - Installation de Micro avec apt install micro

```

debian-wazu@debian-wazu: ~
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 micro amd64 2.0
.11-2+b1 [3 700 kB]
Réception de :2 http://deb.debian.org/debian bookworm/main amd64 xclip amd64 0.1
3-2 [23,3 kB]
3 723 ko réceptionnés en 0s (32,5 Mo/s)
Sélection du paquet micro précédemment désélectionné.
(Lecture de la base de données... 162495 fichiers et répertoires déjà installés.
)
Préparation du dépaquetage de .../micro_2.0.11-2+b1_amd64.deb ...
Dépaquetage de micro (2.0.11-2+b1) ...
Sélection du paquet xclip précédemment désélectionné.
Préparation du dépaquetage de .../xclip_0.13-2_amd64.deb ...
Dépaquetage de xclip (0.13-2) ...
Paramétrage de micro (2.0.11-2+b1) ...
Paramétrage de xclip (0.13-2) ...
Traitement des actions différées (« triggers ») pour desktop-file-utils (0.26-1)
...
Traitement des actions différées (« triggers ») pour gnome-menus (3.36.0-1.1) ..
.
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
Traitement des actions différées (« triggers ») pour mailcap (3.70+nmu1) ...
root@debian-wazu:/home/debian-wazu#

```


Étape 2 → Récupération de l'adresse IP

Nous identifions l'adresse IP de notre serveur, information essentielle pour la configuration des composants *Wazuh*.

Screenshot 4 - Afficher les interfaces réseau et leurs adresses IP avec ip a

```
root@debian-wazu:/etc/filebeat# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens192: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:b2:a7:a9 brd ff:ff:ff:ff:ff:ff
    altname enp11s0
    inet 192.168.111.62/24 brd 192.168.111.255 scope global dynamic noprefixroute ens192
        valid_lft 7047sec preferred_lft 7047sec
    inet6 fe80::20c:29ff:feb2:a7a9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
root@debian-wazu:/etc/filebeat#
```

Dans notre cas, nous avons obtenu l'adresse **192.168.111.62** que nous utiliserons tout au long de l'installation.

Étape 3 → Ajout du dépôt Wazuh

Nous ajoutons maintenant le dépôt officiel *Wazuh* à notre système Debian pour pouvoir installer les paquets.

Screenshot 5 - Ajout de la clé GPG officielle Wazuh pour vérifier l'authenticité des paquets avec curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import && chmod 644 /usr/share/keyrings/wazuh.gpg

```
root@debian-wazu:/home/debian-wazu# curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import && chmod 644 /usr/share/keyrings/wazuh.gpg
gpg: le porte-clefs « /usr/share/keyrings/wazuh.gpg » a été créé
gpg: clef 96B3EE5F29111145 : clef publique « Wazuh.com (Wazuh Signing Key) <support@wazuh.com> » importée
gpg: Quantité totale traitée : 1
gpg: importées : 1
root@debian-wazu:/home/debian-wazu#
```

Cette commande importe la clé publique de Wazuh et la rend lisible par tous les utilisateurs du système.

Screenshot 6 - Ajout du dépôt Wazuh à la liste des sources APT avec echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | tee -a /etc/apt/sources.list.d/wazuh.list

```
root@debian-wazu:/home/debian-wazu# echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | tee -a /etc/apt/sources.list.d/wazuh.list
deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main
root@debian-wazu:/home/debian-wazu#
```

Étape 4 → Mise à jour du système

Nous mettons à jour la liste des paquets et installons les dépendances nécessaires.

Screenshot 7 - Mise à jour de la base de données des paquets et installation des dépendances requises avec apt update && apt upgrade && sudo apt-get install debconf adduser procs curl gnupg apt-transport-https debhelper libcap2-bin

```

Paramétrage de libmail-sendmail-perl (0.80-3) ...
Paramétrage de libltdl-dev:amd64 (2.4.7-7~deb12u1) ...
Paramétrage de libgprofng0:amd64 (2.40-2) ...
Paramétrage de libgcc-12-dev:amd64 (12.2.0-14+deb12u1) ...
Paramétrage de dh-strip-nondeterminism (1.13.1-1) ...
Paramétrage de libc6-dev:amd64 (2.36-9+deb12u13) ...
Paramétrage de binutils-x86-64-linux-gnu (2.40-2) ...
Paramétrage de libstdc++-12-dev:amd64 (12.2.0-14+deb12u1) ...
Paramétrage de binutils (2.40-2) ...
Paramétrage de dpkg-dev (1.21.22) ...
Paramétrage de gcc-12 (12.2.0-14+deb12u1) ...
Paramétrage de g++-12 (12.2.0-14+deb12u1) ...
Paramétrage de libtool (2.4.7-7~deb12u1) ...
Paramétrage de gcc (4:12.2.0-3) ...
Paramétrage de dh-autoreconf (20) ...
Paramétrage de g++ (4:12.2.0-3) ...
update-alternatives: utilisation de « /usr/bin/g++ » pour fournir « /usr/bin/c++ »
(c++) en mode automatique
Paramétrage de build-essential (12.9) ...
Paramétrage de debhelper (13.11.4) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.36-9+deb12u13)
...
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
root@debian-wazu: /home/debian-wazu#

```

Important → Nous n'installons pas *Filebeat* dans cette commande car il sera installé séparément à l'étape appropriée.

Génération des certificats SSL/TLS

La sécurité de notre infrastructure *Wazuh* repose sur l'utilisation de certificats SSL/TLS pour chiffrer toutes les communications entre les composants.

Étape 5 → Téléchargement des outils

Nous téléchargeons le script de génération de certificats et le fichier de configuration.

Screenshot 8 - Téléchargement du script de génération de certificats avec curl -sO https://packages.wazuh.com/4.8/wazuh-certs-tool.sh && curl -sO https://packages.wazuh.com/4.8/config.yml

```

root@debian-wazu: /home/debian-wazu# curl -sO https://packages.wazuh.com/4.8/wazu
h-certs-tool.sh && curl -sO https://packages.wazuh.com/4.8/config.yml
root@debian-wazu: /home/debian-wazu#

```

Nous vérifions que les deux fichiers ont bien été créés.

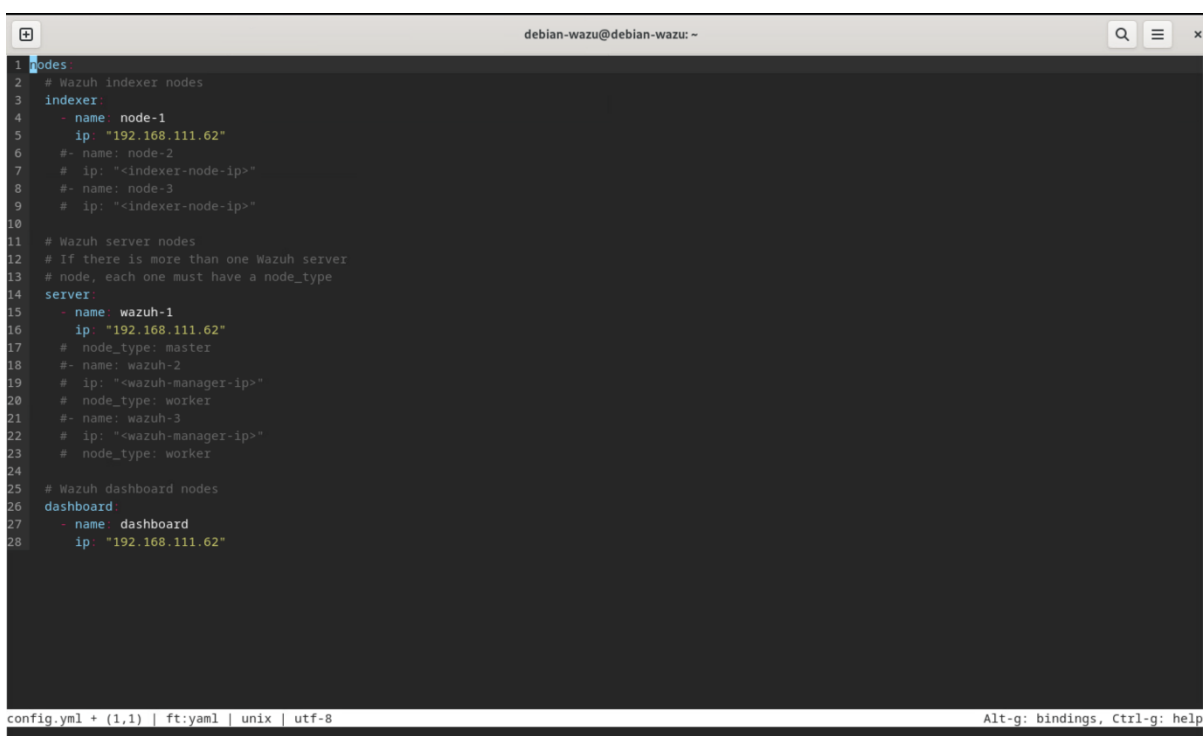
Screenshot 9 - Vérification de la présence des fichiers avec `dir`

```
root@debian-wazu: /home/debian-wazu# dir
Bureau      Documents  Modèles    Public      Vidéos
config.yml  Images     Musique    Téléchargements  wazuh-certs-tool.sh
root@debian-wazu: /home/debian-wazu#
```

Étape 6 → Configuration des nœuds

Nous modifions le fichier `config.yml` pour définir les adresses IP de nos composants (nous modifions les éléments apparaissant en jaune sur le screenshot ci-dessous).

Screenshot 10 - Édition du fichier de configuration avec `micro /config.yml`



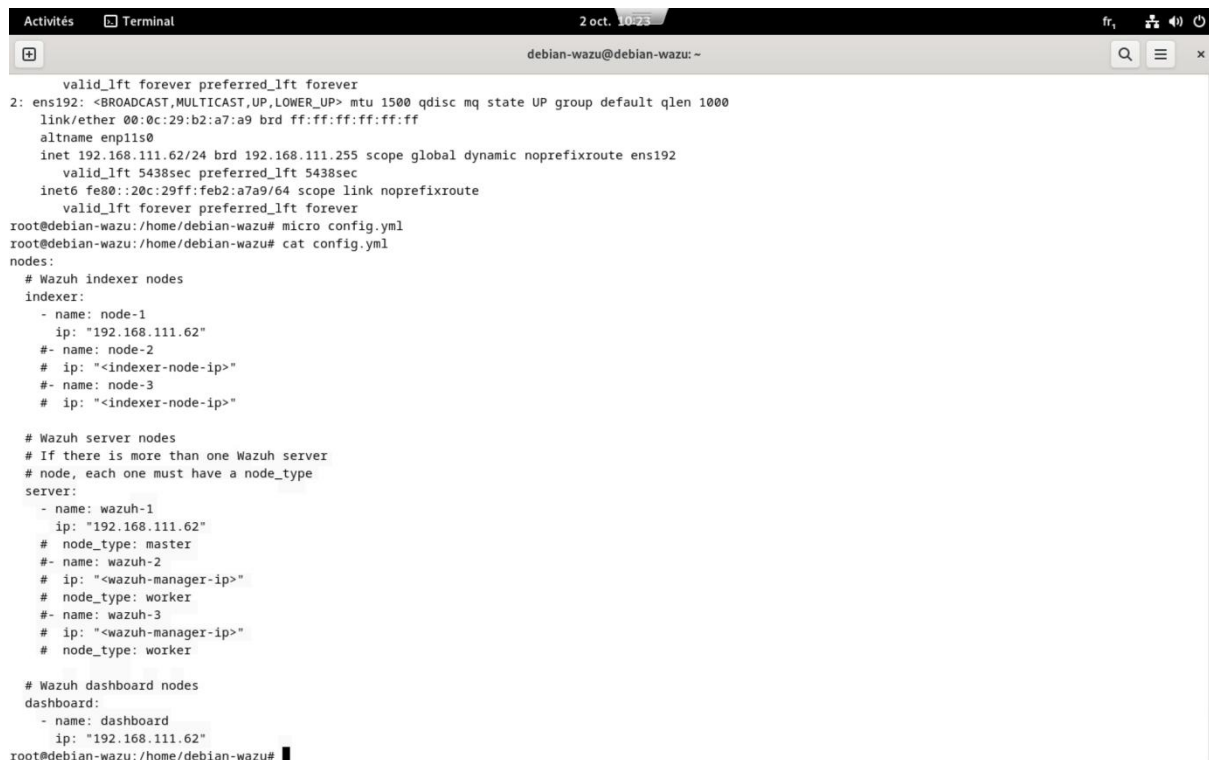
```
1 nodes
2 # Wazuh indexer nodes
3 indexer:
4   - name: node-1
5     ip: "192.168.111.62"
6   #- name: node-2
7   # ip: "<indexer-node-ip>"
8   #- name: node-3
9   # ip: "<indexer-node-ip>"
10
11 # Wazuh server nodes
12 # If there is more than one Wazuh server
13 # node, each one must have a node_type
14 server:
15   - name: wazuh-1
16     ip: "192.168.111.62"
17     # node_type: master
18   #- name: wazuh-2
19   # ip: "<wazuh-manager-ip>"
20   # node_type: worker
21   #- name: wazuh-3
22   # ip: "<wazuh-manager-ip>"
23   # node_type: worker
24
25 # Wazuh dashboard nodes
26 dashboard:
27   - name: dashboard
28     ip: "192.168.111.62"
```

Points importants :

- Nous utilisons la même adresse IP pour tous les composants (configuration single-node)
- Les noms `node-1`, `wazuh-1` et `dashboard` doivent être conservés tels quels pour la suite
- Les guillemets autour de l'adresse IP sont obligatoires

Nous vérifions ensuite notre configuration :

Screenshot 11 - Affichage du contenu pour vérification avec `cat config.yml`



```
valid_lft forever preferred_lft forever
2: ens192: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
   link/ether 00:0c:29:b2:a7:a9 brd ff:ff:ff:ff:ff:ff
   altname enp11s0
   inet 192.168.111.62/24 brd 192.168.111.255 scope global dynamic noprefixroute ens192
       valid_lft 5438sec preferred_lft 5438sec
   inet6 fe80::20c:29ff:feb2:a7a9/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@debian-wazu:/home/debian-wazu# micro config.yml
root@debian-wazu:/home/debian-wazu# cat config.yml
nodes:
  # Wazuh indexer nodes
  indexer:
    - name: node-1
      ip: "192.168.111.62"
    #- name: node-2
    # ip: "<indexer-node-ip>"
    #- name: node-3
    # ip: "<indexer-node-ip>"

  # Wazuh server nodes
  # If there is more than one Wazuh server
  # node, each one must have a node_type
  server:
    - name: wazuh-1
      ip: "192.168.111.62"
      # node_type: master
    #- name: wazuh-2
    # ip: "<wazuh-manager-ip>"
    # node_type: worker
    #- name: wazuh-3
    # ip: "<wazuh-manager-ip>"
    # node_type: worker

  # Wazuh dashboard nodes
  dashboard:
    - name: dashboard
      ip: "192.168.111.62"
root@debian-wazu:/home/debian-wazu#
```

Étape 7 → Génération des certificats

Nous exécutons maintenant le script pour générer tous les certificats nécessaires.

Screenshot 12 - Rendre le script exécutable avec `bash ./wazuh-certs-tool.sh -A`

```
root@debian-wazu:/home/debian-wazu# bash ./wazuh-certs-tool.sh -A
02/10/2025 10:26:02 INFO: Generating the root certificate.
02/10/2025 10:26:03 INFO: Generating Admin certificates.
02/10/2025 10:26:03 INFO: Admin certificates created.
02/10/2025 10:26:03 INFO: Generating Wazuh indexer certificates.
02/10/2025 10:26:03 INFO: Wazuh indexer certificates created.
02/10/2025 10:26:03 INFO: Generating Filebeat certificates.
02/10/2025 10:26:04 INFO: Wazuh Filebeat certificates created.
02/10/2025 10:26:04 INFO: Generating Wazuh dashboard certificates.
02/10/2025 10:26:04 INFO: Wazuh dashboard certificates created.
```

Ce script crée un répertoire `wazuh-certificates/` contenant tous les certificats et clés pour chaque composant.

Étape 8 → Compression des certificats

Nous compressons les certificats dans une archive `TAR` pour faciliter leur déploiement.

Screenshot 13 - Création de l'archive TAR avec `tar -cvf ./wazuh-certificates.tar -C ./wazuh-certificates/`.

```
root@debian-wazu:/home/debian-wazu# tar -cvf ./wazuh-certificates.tar -C ./wazuh-certificates/ .
./
./node-1.pem
./root-ca.pem
./wazuh-1-key.pem
./dashboard.pem
./wazuh-1.pem
./admin-key.pem
./root-ca.key
./node-1-key.pem
./admin.pem
./dashboard-key.pem
root@debian-wazu:/home/debian-wazu#
```

Screenshot 14 - Suppression du répertoire source (optionnel) avec `rm -rf ./wazuh-certificates`

```
root@debian-wazu:/home/debian-wazu# rm -rf ./wazuh-certificates
root@debian-wazu:/home/debian-wazu# █
```

Installation des composants Wazuh

Étape 9 → Installation des paquets principaux

Nous installons maintenant les trois composants principaux de Wazuh en une seule commande.

Screenshot 15 - Installation de Wazuh Indexer, Manager et Dashboard avec `apt install wazuh-indexer wazuh-manager wazuh-dashboard -y`

```
root@debian-wazu:/home/debian-wazu# apt install wazuh-indexer wazuh-manager wazuh-dashboard -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Paquets suggérés :
  expect
Les NOUVEAUX paquets suivants seront installés :
  wazuh-dashboard wazuh-indexer wazuh-manager
0 mis à jour, 3 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 1 470 Mo dans les archives.
Après cette opération, 3 077 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-indexer amd64 4.13.1-1 [856 MB]
Réception de :2 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-dashboard amd64 4.13.1-1 [182 MB]
Réception de :3 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-manager amd64 4.13.1-1 [432 MB]
1 470 Mo réceptionnés en 24s (61,6 Mo/s)
Sélection du paquet wazuh-indexer précédemment désélectionné.
(Lecture de la base de données... 162518 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../wazuh-indexer_4.13.1-1_amd64.deb ...
Running Wazuh Indexer Pre-Installation Script
Dépaquetage de wazuh-indexer (4.13.1-1) ...
Sélection du paquet wazuh-dashboard précédemment désélectionné.
Préparation du dépaquetage de .../wazuh-dashboard_4.13.1-1_amd64.deb ...
Creating wazuh-dashboard group... OK
Creating wazuh-dashboard user... OK
Dépaquetage de wazuh-dashboard (4.13.1-1) ...
Sélection du paquet wazuh-manager précédemment désélectionné.
Préparation du dépaquetage de .../wazuh-manager_4.13.1-1_amd64.deb ...
Dépaquetage de wazuh-manager (4.13.1-1) ...
Paramétrage de wazuh-indexer (4.13.1-1) ...
Running Wazuh Indexer Post-Installation Script
### NOT starting on installation, please execute the following statements to configure wazuh-indexer service to start automatically using systemd
sudo systemctl daemon-reload
sudo systemctl enable wazuh-indexer.service
### You can start wazuh-indexer service by executing
sudo systemctl start wazuh-indexer.service
Paramétrage de wazuh-manager (4.13.1-1) ...
Paramétrage de wazuh-dashboard (4.13.1-1) ...
root@debian-wazu:/home/debian-wazu# █
```

Ce qui est installé :

- ✓ **wazuh-indexer** → Base de données *OpenSearch* pour le stockage des événements
- ✓ **wazuh-manager** → Serveur central d'analyse et de corrélation
- ✓ **wazuh-dashboard** → Interface web de visualisation (basée sur *OpenSearch Dashboards*)

Configuration de Wazuh Indexer**Étape 10 → Configuration réseau de l'Indexer**

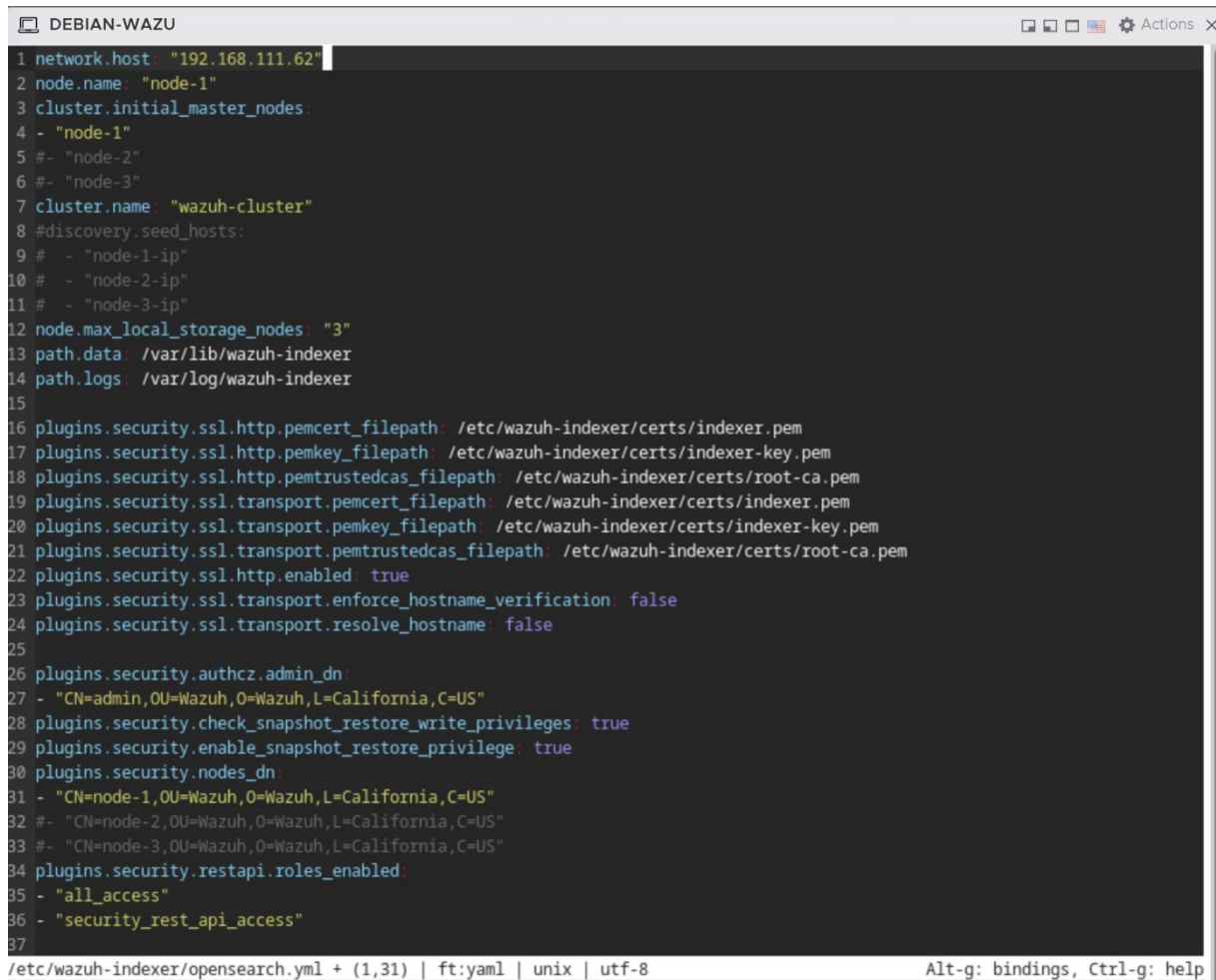
- Nous configurons l'Indexer pour qu'il écoute sur l'adresse IP de notre serveur → *micro /etc/wazuh-indexer/opensearch.yml*

Modification à effectuer :

Édition du fichier de configuration OpenSearch avec la commande → *micro /etc/wazuh-indexer/opensearch.yml*

Nous recherchons la ligne `network.host` et la modifions comme suit :

Screenshot 16 - Modification de "network.host" avec notre adresse IP



```

1 network.host: "192.168.111.62"
2 node.name: "node-1"
3 cluster.initial_master_nodes:
4 - "node-1"
5 #- "node-2"
6 #- "node-3"
7 cluster.name: "wazuh-cluster"
8 #discovery.seed_hosts:
9 # - "node-1-ip"
10 # - "node-2-ip"
11 # - "node-3-ip"
12 node.max_local_storage_nodes: "3"
13 path.data: /var/lib/wazuh-indexer
14 path.logs: /var/log/wazuh-indexer
15
16 plugins.security.ssl.http.pemcert_filepath: /etc/wazuh-indexer/certs/indexer.pem
17 plugins.security.ssl.http.pemkey_filepath: /etc/wazuh-indexer/certs/indexer-key.pem
18 plugins.security.ssl.http.pemtrustedcas_filepath: /etc/wazuh-indexer/certs/root-ca.pem
19 plugins.security.ssl.transport.pemcert_filepath: /etc/wazuh-indexer/certs/indexer.pem
20 plugins.security.ssl.transport.pemkey_filepath: /etc/wazuh-indexer/certs/indexer-key.pem
21 plugins.security.ssl.transport.pemtrustedcas_filepath: /etc/wazuh-indexer/certs/root-ca.pem
22 plugins.security.ssl.http.enabled: true
23 plugins.security.ssl.transport.enforce_hostname_verification: false
24 plugins.security.ssl.transport.resolve_hostname: false
25
26 plugins.security.authcz.admin_dn:
27 - "CN=admin,OU=Wazuh,O=Wazuh,L=California,C=US"
28 plugins.security.check_snapshot_restore_write_privileges: true
29 plugins.security.enable_snapshot_restore_privilege: true
30 plugins.security.nodes_dn:
31 - "CN=node-1,OU=Wazuh,O=Wazuh,L=California,C=US"
32 #- "CN=node-2,OU=Wazuh,O=Wazuh,L=California,C=US"
33 #- "CN=node-3,OU=Wazuh,O=Wazuh,L=California,C=US"
34 plugins.security.restapi.roles_enabled:
35 - "all_access"
36 - "security_rest_api_access"
37
/etc/wazuh-indexer/openssl.yml + (1,31) | ft:yml | unix | utf-8
Alt-g: bindings, Ctrl-g: help

```

Alternative pour écouter sur toutes les interfaces → Nous pouvons utiliser « `network.host: "0.0.0.0"` » pour permettre les connexions depuis n'importe quelle interface réseau.

Étape 11 → Déploiement des certificats de l'Indexer

Nous déployons maintenant les certificats SSL pour sécuriser les communications avec l'Indexer.

Important → Nous devons définir la variable `NODE_NAME` avant d'extraire les certificats. Il doit correspondre au `config.yml`, pour le définir nous utilisons la commande → `NODE_NAME=node-1`

Maintenant nous créons le répertoire des certificats et nous les extrayons :

Screenshot 17 - Création du répertoire pour les certificats avec `mkdir /etc/wazuh-indexer/certs`

```

root@debian-wazu:/home/debian-wazu# micro /etc/wazuh-indexer/openssl.yml
root@debian-wazu:/home/debian-wazu# tar -xvf ./wazuh-certificates.tar -C /etc/wazuh-indexer/certs/ ./NODE_NAME.pem ./NODE_NAME-key.pem ./admin.pem ./admin-key.pem ./root-ca.pem
tar: ./pem : non trouvé dans l'archive
tar: ./key.pem : non trouvé dans l'archive
tar: Arrêt avec code d'échec à cause des erreurs précédentes
root@debian-wazu:/home/debian-wazu#

```

Screenshot 18 - Extraction des certificats depuis l'archive avec `tar -xf ./wazuh-certificates.tar -C /etc/wazuh-indexer/certs/ ./node-1.pem ./node-1-key.pem ./admin.pem ./admin-key.pem ./root-ca.pem`

```
root@debian-wazu:/home/debian-wazu# tar -xf ./wazuh-certificates.tar -C /etc/wazuh-indexer/certs/ ./node-1.pem ./node-1-key.p
em ./admin.pem ./admin-key.pem ./root-ca.pem
root@debian-wazu:/home/debian-wazu#
```

Erreur fréquente → Si nous obtenons « *fichier non trouvé* », nous devons vérifier que :

1. La variable `NODE_NAME` est bien définie → `echo $NODE_NAME`
2. L'archive existe → `ls -lh wazuh-certificates.tar`
3. Vous n'avez pas d'espace parasite dans le nom du fichier

➤ **Nous renommons ensuite les certificats :**

- **Renommage du certificat du nœud** → `mv -n /etc/wazuh-indexer/certs/node-1.pem /etc/wazuh-indexer/certs/indexer.pem`
- **Renommage de la clé privée** → `mv -n /etc/wazuh-indexer/certs/node-1-key.pem /etc/wazuh-indexer/certs/indexer-key.pem`

➤ **Nous configurons également les permissions de sécurité :**

- **Permissions restrictives sur le répertoire** → `chmod 500 /etc/wazuh-indexer/certs`
- **Permissions restrictives sur les fichiers** → `chmod 400 /etc/wazuh-indexer/certs/*`

Screenshot 19 - Renommage des certificats et configuration des permissions de sécurité

```
root@debian-wazu:/home/debian-wazu# mv -n /etc/wazuh-indexer/certs/node-1.pem /etc/wazuh-indexer/certs/indexer.pem
root@debian-wazu:/home/debian-wazu# mv -n /etc/wazuh-indexer/certs/node-1-key.pem /etc/wazuh-indexer/certs/indexer-key.pem
root@debian-wazu:/home/debian-wazu# chmod 500 /etc/wazuh-indexer/certs
root@debian-wazu:/home/debian-wazu# chmod 400 /etc/wazuh-indexer/certs/*
```

Screenshot 20 - Attribution de la propriété à l'utilisateur `wazuh-indexer` avec `chown -R wazuh-indexer:wazuh-indexer /etc/wazuh-indexer/certs`

```
root@debian-wazu:/home/debian-wazu# chown -R wazuh-indexer:wazuh-indexer /etc/wazuh-indexer/certs
root@debian-wazu:/home/debian-wazu#
```


Étape 12 → Démarrage de Wazuh Indexer

Nous activons et démarrons maintenant le service *Wazuh Indexer*.

Screenshot 21 - Activation et démarrage de wazuh-indexer avec systemctl daemon-reload ; systemctl enable wazuh-indexer ; systemctl start wazuh-indexer

```
root@debian-wazu: /home/debian-wazu# sudo systemctl daemon-reload
root@debian-wazu: /home/debian-wazu# sudo systemctl enable wazuh-indexer
Synchronizing state of wazuh-indexer.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable wazuh-indexer
root@debian-wazu: /home/debian-wazu# sudo systemctl start wazuh-indexer
root@debian-wazu: /home/debian-wazu#
```

Étape 13 → Initialisation du cluster de sécurité

Tout d'abord, nous naviguons vers le bon répertoire et nous vérifions la présence des fichiers nécessaires avec la commande *ls -la*.

Screenshot 22 - Affichage du contenu du répertoire usr/share/wazuh-indexer/bin/ avec ls -la

```
root@debian-wazu: /home/debian-wazu# cd /usr/share/wazuh-indexer/bin/
root@debian-wazu: /usr/share/wazuh-indexer/bin# ls -la
total 64
drwxr-x--- 3 wazuh-indexer wazuh-indexer 4096 2 oct. 10:33 .
drwxr-x--- 8 wazuh-indexer wazuh-indexer 4096 2 oct. 10:33 ..
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 6028 23 sept. 13:19 indexer-security-init.sh
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 3117 23 sept. 13:19 opensearch
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 1090 23 sept. 13:19 opensearch-cli
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 5494 23 sept. 13:19 opensearch-env
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 1838 23 sept. 13:19 opensearch-env-from-file
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 222 23 sept. 13:19 opensearch-keystore
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 155 23 sept. 13:19 opensearch-node
drwxr-x--- 2 wazuh-indexer wazuh-indexer 4096 2 oct. 10:33 opensearch-performance-analyzer
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 210 23 sept. 13:19 opensearch-plugin
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 148 23 sept. 13:19 opensearch-shard
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 211 23 sept. 13:19 opensearch-upgrade
-rwxr-x--- 1 wazuh-indexer wazuh-indexer 369 23 sept. 13:19 systemd-entrypoint
root@debian-wazu: /usr/share/wazuh-indexer/bin#
```

Nous initialisons maintenant les composants de sécurité d'*OpenSearch*.

Screenshot 23 - Initialisation des composants de sécurité d'OpenSearch avec /usr/share/wazuh-indexer/bin/indexer-security-init.sh

```
root@debian-wazu: /usr/share/wazuh-indexer/bin# sudo ./indexer-security-init.sh
Security Admin v7
Will connect to 192.168.111.62:9200 ... done
Connected as "CN=admin,O=Wazuh,O=Wazuh,L=California,C=US"
OpenSearch Version: 2.19.2
Contacting opensearch cluster 'opensearch' and wait for YELLOW clusterstate ...
Clustername: wazuh-cluster
Clusterstate: GREEN
Number of nodes: 1
Number of data nodes: 1
.opendistro_security index does not exists, attempt to create it ... done (0-all replicas)
Populate config from /etc/wazuh-indexer/opensearch-security/
Will update '/config' with /etc/wazuh-indexer/opensearch-security/config.yml
SUCC: Configuration for 'config' created or updated
Will update '/roles' with /etc/wazuh-indexer/opensearch-security/roles.yml
SUCC: Configuration for 'roles' created or updated
Will update '/rolesmapping' with /etc/wazuh-indexer/opensearch-security/roles_mapping.yml
SUCC: Configuration for 'rolesmapping' created or updated
Will update '/internalusers' with /etc/wazuh-indexer/opensearch-security/internal_users.yml
SUCC: Configuration for 'internalusers' created or updated
Will update '/actiongroups' with /etc/wazuh-indexer/opensearch-security/action_groups.yml
SUCC: Configuration for 'actiongroups' created or updated
Will update '/tenants' with /etc/wazuh-indexer/opensearch-security/tenants.yml
SUCC: Configuration for 'tenants' created or updated
Will update '/nodesdn' with /etc/wazuh-indexer/opensearch-security/nodes_dn.yml
SUCC: Configuration for 'nodesdn' created or updated
Will update '/whitelist' with /etc/wazuh-indexer/opensearch-security/whitelist.yml
SUCC: Configuration for 'whitelist' created or updated
Will update '/audit' with /etc/wazuh-indexer/opensearch-security/audit.yml
SUCC: Configuration for 'audit' created or updated
Will update '/allowlist' with /etc/wazuh-indexer/opensearch-security/allowlist.yml
SUCC: Configuration for 'allowlist' created or updated
SUCC: Expected 10 config types for node ("updated_config_types":["allowlist","tenants","rolesmapping","nodesdn","audit","roles","whitelist","actiongroups","config","internalusers"],"updated_config_size":10,"message":null) is 10 ([{"allowlist","tenants","rolesmapping","nodesdn","audit","roles","whitelist","actiongroups","config","internalusers"}]) due to: null
Done with success
root@debian-wazu: /usr/share/wazuh-indexer/bin#
```

Ce script configure :

- Les rôles et permissions
- Les utilisateurs internes

- Les politiques de sécurité
- L'authentification SSL/TLS

Étape 14 → Test de l'installation de l'Indexer

Nous vérifions que l'Indexer fonctionne correctement en effectuant des requêtes de test.

Nous utilisons utilisateur/MDP par défaut → *admin/admin*.

Screenshot 24 - Test de connexion à l'API de l'Indexer avec sudo curl -k -u admin:admin https://192.168.111.62:9200

```
root@debian-wazu:/usr/share/wazuh-indexer/bin# sudo curl -k -u admin:admin https://192.168.111.62:9200
{
  "name" : "node-1",
  "cluster_name" : "wazuh-cluster",
  "cluster_uuid" : "eo68-SAtR90PB63tGCG74A",
  "version" : {
    "number" : "7.10.2",
    "build_type" : "deb",
    "build_hash" : "63bef474b662d18fef0c73e0bd7668a8c5024121",
    "build_date" : "2025-09-23T11:13:24.501309488Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.1",
    "minimum_wire_compatibility_version" : "7.10.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
```

Screenshot 25 - Test de listage des nœuds du cluster avec curl -k -u admin:admin https://192.168.111.62:9200/_cat/nodes?v

```
root@debian-wazu:/usr/share/wazuh-indexer/bin# curl -k -u admin:admin https://192.168.111.62:9200/_cat/nodes?v
ip heap.percent ram.percent cpu load_1m load_5m load_15m node.role node.roles cluster_manager name
192.168.111.62 66 95 1 0.02 0.04 0.02 dimr cluster_manager,data,ingest,remote_cluster_client * node-1
root@debian-wazu:/usr/share/wazuh-indexer/bin#
```

Résultat attendu :

- **Première commande** → Informations JSON sur le cluster (nom, version, etc.)
- **Deuxième commande** → Liste tabulaire des nœuds actifs avec statistiques

La sortie indique que l'installation a bien réussi.

Configuration de Wazuh Manager

Étape 15 → Démarrage de Wazuh Manager

Nous activons et démarrons maintenant le gestionnaire Wazuh, cœur du système de surveillance.

Screenshot 26 - Activation et démarrage du gestionnaire wazuh avec : systemctl daemon-reload ; systemctl enable wazuh-manager ; systemctl start wazuh-manager

```
root@debian-wazu:/usr/share/wazuh-indexer/bin# systemctl daemon-reload
root@debian-wazu:/usr/share/wazuh-indexer/bin# systemctl enable wazuh-manager
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-manager.service - /lib/systemd/system/wazuh-manager.service.
root@debian-wazu:/usr/share/wazuh-indexer/bin# systemctl start wazuh-manager
```

Nous vérifions que le service *wazuh-manager* est bien installé et en fonction.

Screenshot 27 - Vérification de l'installation et du fonctionnement de wazuh-manager avec systemctl status wazuh-manager

```
root@debian-wazu:/usr/share/wazuh-indexer/bin# systemctl status wazuh-manager
* wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-10-02 11:45:00 CEST; 14s ago
     Process: 54692 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 203 (limit: 4616)
   Memory: 862.9M
      CPU: 19.059s
   CGroup: /system.slice/wazuh-manager.service
           └─$4754 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─$4795 /var/ossec/bin/wazuh-authd
           └─$4809 /var/ossec/bin/wazuh-db
           └─$4823 /var/ossec/bin/wazuh-execd
           └─$4859 /var/ossec/bin/wazuh-analysisd
           └─$4861 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─$4862 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─$4865 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─$4868 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─$4911 /var/ossec/bin/wazuh-syscheckd
           └─$4925 /var/ossec/bin/wazuh-remoted
           └─$4962 /var/ossec/bin/wazuh-logcollector
           └─$4981 /var/ossec/bin/wazuh-monitord
           └─$5001 /var/ossec/bin/wazuh-modulesd
           └─$5719 apt list iptables

oct. 02 11:44:53 debian-wazu env[54692]: Started wazuh-syscheckd...
oct. 02 11:44:55 debian-wazu env[54692]: Started wazuh-remoted...
oct. 02 11:44:56 debian-wazu env[54692]: Started wazuh-logcollector...
oct. 02 11:44:57 debian-wazu env[54692]: Started wazuh-monitord...
oct. 02 11:44:57 debian-wazu env[54990]: 2025/10/02 11:44:57 wazuh-modulesd:router: INFO: Loaded router module.
oct. 02 11:44:57 debian-wazu env[54990]: 2025/10/02 11:44:57 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
oct. 02 11:44:57 debian-wazu env[54990]: 2025/10/02 11:44:57 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
oct. 02 11:44:58 debian-wazu env[54692]: Started wazuh-modulesd...
oct. 02 11:45:00 debian-wazu env[54692]: Completed.
oct. 02 11:45:00 debian-wazu systemd[1]: Started wazuh-manager.service - Wazuh manager.
root@debian-wazu:/usr/share/wazuh-indexer/bin#
```

Configuration de Filebeat

Filebeat agit comme un collecteur de logs qui transmet les événements du *Wazuh Manager* vers *l'Indexer*.

Étape 16 → Installation de Filebeat

Screenshot 28 - Installation du paquet Filebeat avec apt install filebeat

```
root@debian-wazu:/usr/share/wazuh-indexer/bin# apt install filebeat
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
filebeat est déjà la version la plus récente (7.10.2-1).
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
root@debian-wazu:/usr/share/wazuh-indexer/bin#
```

Étape 17 → Configuration de Filebeat

Nous téléchargeons et configurons le fichier de configuration optimisé pour *Wazuh*.

Screenshot 29 - Téléchargement du fichier de configuration Wazuh pour Filebeat avec curl -so /etc/filebeat/filebeat.yml https://packages.wazuh.com/4.8/tpl/wazuh/filebeat/filebeat.yml

```
root@debian-wazu:/usr/share/wazuh-indexer/bin# curl -so /etc/filebeat/filebeat.yml https://packages.wazuh.com/4.8/tpl/wazuh/filebeat/filebeat.yml
root@debian-wazu:/usr/share/wazuh-indexer/bin#
```

Nous nous assurons qu'il est bien présent dans le répertoire.

Screenshot 30 - Vérification de la présence des fichiers filebeat avec la commande `ls -la`

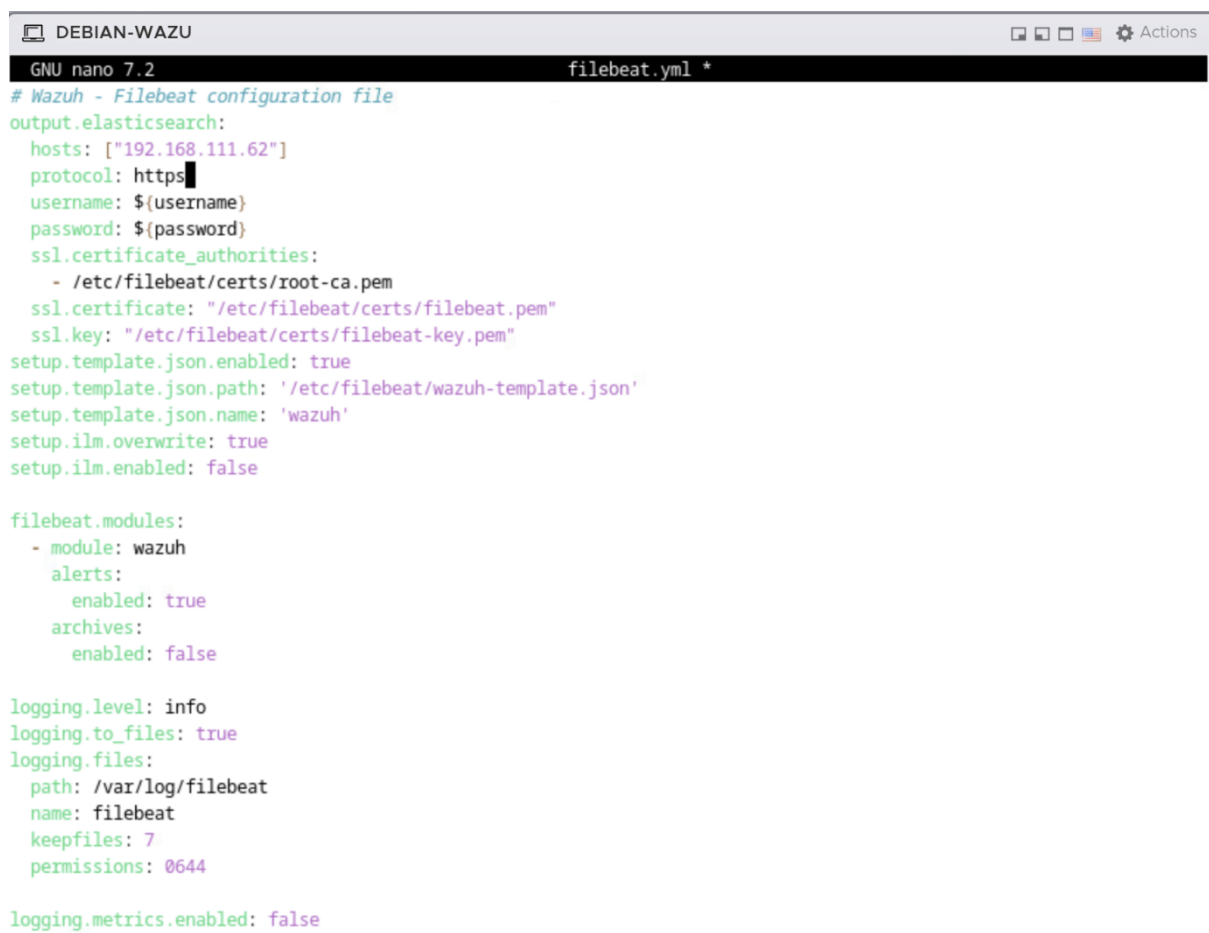
```
root@debian-wazu:/etc/filebeat# ls -la
total 416
drwxr-xr-x  3 root root  4096 2 oct.  10:11 .
drwxr-xr-x 124 root root 12288 2 oct.  10:34 ..
-rw-r--r--  1 root root 297349 4 avril 13:32 fields.yml
-rw-r--r--  1 root root  91838 4 avril 13:32 filebeat.reference.yml
-rw-----  1 root root   9982 4 avril 13:32 filebeat.yml
drwxr-xr-x  2 root root  4096 2 oct.  10:11 modules.d
```

Nous modifions maintenant l'adresse de l'Indexer dans la configuration :

➤ **Édition de la configuration Filebeat** → `nano /etc/filebeat/filebeat.yml`

Nous recherchons la section `output.elasticsearch` et modifions l'adresse

Screenshot 31 - Modification de la section `output.elasticsearch` avec notre adresse IP



```
DEBIAN-WAZU
GNU nano 7.2 filebeat.yml *
# Wazuh - Filebeat configuration file
output.elasticsearch:
  hosts: ["192.168.111.62"]
  protocol: https
  username: ${username}
  password: ${password}
  ssl.certificate_authorities:
    - /etc/filebeat/certs/root-ca.pem
  ssl.certificate: "/etc/filebeat/certs/filebeat.pem"
  ssl.key: "/etc/filebeat/certs/filebeat-key.pem"
setup.template.json.enabled: true
setup.template.json.path: '/etc/filebeat/wazuh-template.json'
setup.template.json.name: 'wazuh'
setup.ilm.overwrite: true
setup.ilm.enabled: false

filebeat.modules:
  - module: wazuh
    alerts:
      enabled: true
    archives:
      enabled: false

logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
  permissions: 0644

logging.metrics.enabled: false
```

Étape 18 → Configuration du keystore Filebeat

Nous créons le fichier `filebeat keystore` pour stocker en toute sécurité les informations d'authentification.

Screenshot 32 - Création du keystore avec filebeat `keystore create`

```
root@debian-wazu:/etc/filebeat# filebeat keystore create
Created filebeat keystore
root@debian-wazu:/etc/filebeat#
```

Nous ajoutons le nom d'utilisateur et le mot de passe par défaut au keystore :

- **Ajout du nom d'utilisateur** → `echo admin | filebeat keystore add username --stdin --force`
- **Ajout du mot de passe** → `echo admin | filebeat keystore add password --stdin --force`

Le nom d'utilisateur et le mot de passe sont tous deux spécifiés à « *admin* ».

Screenshot 33 - Ajout du nom d'utilisateur et du mot de passe `echo admin | filebeat keystore add username --stdin --force ; echo admin | filebeat keystore add password --stdin --force`

```
root@debian-wazu:/etc/filebeat# echo admin | filebeat keystore add username --stdin --force
Successfully updated the keystore
root@debian-wazu:/etc/filebeat# echo admin | filebeat keystore add password --stdin --force
Successfully updated the keystore
```

Note de sécurité → En production, nous devrions utiliser des identifiants personnalisés au lieu de « *admin/admin* ».

Étape 19 → Installation du template d'alertes

Nous téléchargeons le *template Elasticsearch* pour formater correctement les alertes Wazuh.

Screenshot 34 - Téléchargement du template JSON avec `curl -so /etc/filebeat/wazuh-template.json https://raw.githubusercontent.com/wazuh/wazuh/v4.8.2/extensions/elasticsearch/7.x/wazuh-template.json`

```
root@debian-wazu:/etc/filebeat# curl -so /etc/filebeat/wazuh-template.json https://raw.githubusercontent.com/wazuh/wazuh/v4.8.2/extensions/elasticsearch/7.x/wazuh-template.json
```

Screenshot 35 - Attribution des permissions de lecture avec `chmod go+r /etc/filebeat/wazuh-template.json`

```
root@debian-wazu:/etc/filebeat# chmod go+r /etc/filebeat/wazuh-template.json
root@debian-wazu:/etc/filebeat#
```

Étape 20 → Installation du module Wazuh pour Filebeat

Screenshot 36 - Installation du module Wazuh pour Filebeat avec curl -s <https://packages.wazuh.com/4.x/filebeat/wazuh-filebeat-0.4.tar.gz> | tar -xvz -C /usr/share/filebeat/module

```
root@debian-wazu:/etc/filebeat# curl -s https://packages.wazuh.com/4.x/filebeat/wazuh-filebeat-0.4.tar.gz | tar -xvz
-C /usr/share/filebeat/module
wazuh/
wazuh/alerts/
wazuh/alerts/ingest/
wazuh/alerts/ingest/pipeline.json
wazuh/alerts/manifest.yml
wazuh/alerts/config/
wazuh/alerts/config/alerts.yml
wazuh/_meta/
wazuh/_meta/docs.asciidoc
wazuh/_meta/config.yml
wazuh/_meta/fields.yml
wazuh/module.yml
wazuh/archives/
wazuh/archives/ingest/
wazuh/archives/ingest/pipeline.json
wazuh/archives/manifest.yml
wazuh/archives/config/
wazuh/archives/config/archives.yml
root@debian-wazu:/etc/filebeat#
```

Étape 21 → Déploiement des certificats Filebeat

Nous déployons les certificats SSL pour sécuriser la communication entre *Filebeat* et *l'Indexer*.

Ici encore une fois, remplacer le nom du certificat <SERVER_NODE_NAME> par le nom utilisé lors de la création des certificats. Pour nous, il restera " « wazuh-1 » »

Définition du nom du nœud serveur → `NODE_NAME=wazuh-1`

Screenshot 37 - Création du répertoire des certificats avec `mkdir /etc/filebeat/certs`

```
root@debian-wazu:/etc/filebeat# mkdir /etc/filebeat/certs
```

➤ Extraction des certificats :

```
tar -xf ./wazuh-certificates.tar -C /etc/filebeat/certs/ ./${NODE_NAME}.pem
./${NODE_NAME}-key.pem ./root-ca.pem
```

➤ Renommage des certificats :

```
mv -n /etc/filebeat/certs/${NODE_NAME}.pem /etc/filebeat/certs/filebeat.pem
mv -n /etc/filebeat/certs/${NODE_NAME}-key.pem /etc/filebeat/certs/filebeat-key.pem
```

➤ **Configuration des permissions :**

```
chmod 500 /etc/filebeat/certs
chmod 400 /etc/filebeat/certs/*
chown -R root:root /etc/filebeat/certs
```

Screenshot 38 - Extraction, renommage des certificats et configurations des permissions de Filebeat

```
root@debian-wazu:/home/debian-wazu# tar -xf ./wazuh-certificates.tar -C /etc/filebeat/certs/ ./wazuh-1.pem ./wazuh-1-key.pem ./root-ca.pem
root@debian-wazu:/home/debian-wazu# mv -n /etc/filebeat/certs/wazuh-1.pem /etc/filebeat/certs/filebeat.pem
root@debian-wazu:/home/debian-wazu# mv -n /etc/filebeat/certs/wazuh-1-key.pem /etc/filebeat/certs/filebeat-key.pem
root@debian-wazu:/home/debian-wazu# chmod 500 /etc/filebeat/certs
root@debian-wazu:/home/debian-wazu# chmod 400 /etc/filebeat/certs/*
root@debian-wazu:/home/debian-wazu# chown -R root:root /etc/filebeat/certs
root@debian-wazu:/home/debian-wazu# █
```

Étape 22 → Démarrage de Filebeat

- **Rechargement de système** → *systemctl daemon-reload*
- **Activation au démarrage** → *systemctl enable filebeat*
- **Démarrage du service** → *systemctl start filebeat*

Screenshot 39 - Recharge, activation et démarrage de Filebeat

```
root@debian-wazu:/home/debian-wazu# systemctl daemon-reload
root@debian-wazu:/home/debian-wazu# systemctl enable filebeat
Synchronizing state of filebeat.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable filebeat
Created symlink /etc/systemd/system/multi-user.target.wants/filebeat.service → /lib/systemd/system/filebeat.service.
root@debian-wazu:/home/debian-wazu# systemctl start filebeat
root@debian-wazu:/home/debian-wazu#
```

Étape 23 → Test de connexion Filebeat

Nous vérifions que *Filebeat* peut se connecter à l'Indexer.

Screenshot 40 - Test de connexion avec filebeat test output

```

root@debian-wazu:/home/debian-wazu# filebeat test output
elasticsearch: https://192.168.111.62:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 192.168.111.62
    dial up... OK
  TLS...
    security: server's certificate chain verification is enabled
    handshake... OK
    TLS version: TLSv1.3
    dial up... OK
  talk to server... OK
  version: 7.10.2
root@debian-wazu:/home/debian-wazu# █

```

Nous obtenons « *talk to server... OK* », la connexion est fonctionnelle.

Configuration de Wazuh Dashboard

Le *Dashboard* est l'interface web qui nous permet de visualiser et gérer notre infrastructure Wazuh.

Étape 24 → Configuration du Dashboard

Nous configurons maintenant le *Dashboard Wazuh* pour qu'il puisse communiquer avec *l'Indexer*. Cette étape est cruciale car elle établit le lien entre l'interface de visualisation et la base de données qui stocke nos événements de sécurité.

- **Édition du fichier de configuration** → `micro /etc/wazuh-dashboard/opensearch_dashboards.yml`

Nous modifions le paramètre « *opensearch.hosts* » pour pointer vers l'adresse IP de notre serveur. Dans notre configuration *All-in-One*, l'Indexer se trouve sur la même machine, nous utilisons donc notre adresse locale.

Screenshot 41 - Modification de l'adresse IP « opensearch.hosts » par notre adresse IP

```

1 server.host: 0.0.0.0
2 server.port: 443
3 opensearch.hosts: https://192.168.111.62:9200
4 opensearch.ssl.verificationMode: certificate
5 #opensearch.username:
6 #opensearch.password:
7 opensearch.requestHeadersAllowlist: ["securitytenant","Authorization"]
8 opensearch_security.multitenancy.enabled: false
9 opensearch_security.readonly_mode.roles: ["kibana_read_only"]
10 server.ssl.enabled: true
11 server.ssl.key: "/etc/wazuh-dashboard/certs/dashboard-key.pem"
12 server.ssl.certificate: "/etc/wazuh-dashboard/certs/dashboard.pem"
13 opensearch.ssl.certificateAuthorities: ["/etc/wazuh-dashboard/certs/root-ca.pem"]
14 uiSettings.overrides.defaultRoute: /app/wz-home
15 # Session expiration settings
16 opensearch_security.cookie.ttl: 900000
17 opensearch_security.session.ttl: 900000
18 opensearch_security.session.keepalive: true
19
/etc/wazuh-dashboard/opensearch_dashboards.yml + (4,45) | ft:yaml | unix | utf-8  Alt-g: bindings, Ctrl-g: help

```

Point important → Nous remplaçons l'adresse par défaut par `https://192.168.111.62:9200` pour que le *Dashboard* puisse interroger l'*Indexer* sur le port standard 9200 en HTTPS sécurisé.

Étape 25 → Déploiement des certificats du Dashboard

Nous déployons maintenant les certificats SSL/TLS pour le Dashboard. Ces certificats sont essentiels pour sécuriser les communications entre les différents composants de *Wazuh* et garantir la confidentialité des données de sécurité que nous manipulons.

Nous devons remplacer `<DASHBOARD_NODE_NAME>` par le nom défini lors de la configuration du fichier *config.yml* à l'étape 6 au niveau du tableau de bord. Nous n'avons rien modifié à ce moment-là, nous utilisons donc le nom par défaut « *dashboard* ».

➤ **Définition du nom du nœud dashboard** → `NODE_NAME=dashboard`

Cette variable d'environnement nous permet de référencer facilement le nom de notre nœud dans les commandes suivantes.

Nous créons le répertoire qui hébergera tous les certificats nécessaires au fonctionnement sécurisé du Dashboard.

Screenshot 42 - Création du répertoire des certificats avec `mkdir /etc/wazuh-dashboard/certs`

```
root@debian-wazu: /home/debian-wazu# NODE_NAME=dashboard
root@debian-wazu: /home/debian-wazu# mkdir /etc/wazuh-dashboard/certs
root@debian-wazu: /home/debian-wazu# █
```

➤ **Extraction des certificats**

```
tar -xf ./wazuh-certificates.tar -C /etc/wazuh-dashboard/certs/
./dashboard.pem ./dashboard-key.pem ./root-ca.pem
```

- **Nous extrayons les trois fichiers essentiels de notre archive de certificats :**
 - Le certificat public du **dashboard** → `dashboard.pem`
 - La clé privée du **dashboard** → `dashboard-key.pem`
 - Le certificat de l'autorité de certification racine → `root-ca.pem`

➤ **Renommage des certificats :**

Renommage du certificat du nœud → `mv -n /etc/wazuh-dashboard/certs/dashboard.pem /etc/wazuh-dashboard/certs/dashboard.pem`

Renommage de la clé privée → `mv -n /etc/wazuh-dashboard/certs/dashboard-key.pem /etc/wazuh-dashboard/certs/dashboard-key.pem`

Nous appliquons maintenant des permissions restrictives pour protéger nos certificats. C'est une étape critique car la compromission de ces certificats pourrait permettre à un attaquant de se faire passer pour notre Dashboard.

- Nous configurons également les permissions de sécurité :

Permissions restrictives sur le répertoire (lecture + exécution pour le propriétaire uniquement) → `chmod 500 /etc/wazuh-dashboard/certs`

Permissions restrictives sur les fichiers (lecture uniquement pour le propriétaire) → `chmod 400 /etc/wazuh-dashboard/certs/*`

- **Attribution de la propriété à Wazuh-dasboard** → `chown -R wazuh-dashboard:wazuh-dashboard /etc/wazuh-dashboard/certs`

Screenshot 43 - Extraction, renommage des certificats, configurations des permissions et changement du propriétaire du dashboard

```
root@debian-wazu:/home/debian-wazu# tar -xf ./wazuh-certificates.tar -C /etc/wazuh-dashboard/certs/ ./dashboard.pem
./dashboard-key.pem ./root-ca.pem
root@debian-wazu:/home/debian-wazu# mv -n /etc/wazuh-dashboard/certs/dashboard.pem /etc/wazuh-dashboard//certs/dashb
oard.pem
root@debian-wazu:/home/debian-wazu# mv -n /etc/wazuh-dashboard/certs/dashboard-key.pem /etc/wazuh-dashboard//certs/d
ashboard-key.pem
root@debian-wazu:/home/debian-wazu# chmod 500 /etc/wazuh-dashboard/certs
root@debian-wazu:/home/debian-wazu# chmod 400 /etc/wazuh-dashboard/certs/*
root@debian-wazu:/home/debian-wazu# chown -R wazuh-dashboard:wazuh-dashboard /etc/wazuh-dashboard/certs
root@debian-wazu:/home/debian-wazu#
```

Ces permissions garantissent que :

- ✓ Seul l'utilisateur `wazuh-dashboard` peut lire les certificats
- ✓ Personne ne peut modifier ou supprimer les fichiers sans élévation de privilèges
- ✓ Les autres utilisateurs du système n'ont aucun accès

Vérification des certificats :

Nous vérifions maintenant que notre configuration est correcte en consultant le fichier de configuration initial afin de s'assurer que le nom du nœud de notre *dashboard Wazuh* est correct.

Screenshot 44 - Vérification du contenu avec cat config.yml

```
root@debian-wazu:/home/debian-wazu# cat config.yml
nodes:
  # Wazuh indexer nodes
  indexer:
    - name: node-1
      ip: "192.168.111.62"
    #- name: node-2
    # ip: "<indexer-node-ip>"
    #- name: node-3
    # ip: "<indexer-node-ip>"

  # Wazuh server nodes
  # If there is more than one Wazuh server
  # node, each one must have a node_type
  server:
    - name: wazuh-1
      ip: "192.168.111.62"
      # node_type: master
    #- name: wazuh-2
    # ip: "<wazuh-manager-ip>"
    # node_type: worker
    #- name: wazuh-3
    # ip: "<wazuh-manager-ip>"
    # node_type: worker

  # Wazuh dashboard nodes
  dashboard:
    - name: dashboard
      ip: "192.168.111.62"
```

Étape 26 → Démarrage et vérification des services

Nous sommes maintenant prêts à démarrer le service *Dashboard* et à vérifier que l'ensemble de notre installation *Wazuh* fonctionne correctement.

Notez que nous n'avons pas besoin d'utiliser *sudo* étant déjà en « *root* ».

Démarrer le service wazuh-dashboard :

- **Rechargement de la configuration système pour prendre en compte les changements** → *systemctl daemon-reload*
- **Activation du service au démarrage du système** → *systemctl enable wazuh-dashboard*
- **Démarrage du service** → *systemctl start wazuh-dashboard*

Screenshot 45 - Rechargement, activation et démarrage de wazuh-dashboard

```
root@debian-wazu:/home/debian-wazu# systemctl daemon-reload
root@debian-wazu:/home/debian-wazu# systemctl enable wazuh-dashboard
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-dashboard.service → /etc/systemd/system/wazuh-dashboard.service.
root@debian-wazu:/home/debian-wazu# systemctl start wazuh-dashboard
```

Ces commandes nous permettent de :

1. Rafraîchir la configuration *systemd*
2. Garantir que le *Dashboard* démarrera automatiquement après un redémarrage
3. Lancer immédiatement le service

Vérification de l'état des services → Nous vérifions maintenant que tous les composants de *Wazuh* sont opérationnels :

Nous devons voir, pour les trois tests, un *statut* « *active (running)* » en vert, ce qui indique que le service fonctionne correctement.

Le *Dashboard* doit être actif → c'est l'interface web qui nous permettra de visualiser les données de sécurité.

Screenshot 46 - Vérification du Dashboard avec `systemctl status wazuh-dashboard`

```
root@debian-wazu: /home/debian-wazu# systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-10-02 14:32:16 CEST; 1min 41s ago
     Main PID: 57902 (node)
        Tasks: 11 (limit: 4616)
       Memory: 299.1M
          CPU: 7.038s
      CGroup: /system.slice/wazuh-dashboard.service
              └─57902 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandle>

oct. 02 14:33:58 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:58 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: [agentkeepalive:deprecated] options.freeSocketKeepAliveT>
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: {"type": "log", "@timestamp": "2025-10-02T12:33:59Z", "tags">
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: {"type": "log", "@timestamp": "2025-10-02T12:33:59Z", "tags">
oct. 02 14:33:59 debian-wazu opensearch-dashboards[57902]: {"type": "log", "@timestamp": "2025-10-02T12:33:59Z", "tags">
lines 1-20/20 (END)
```

Le *Wazuh Manager* doit également être actif → c'est lui qui analyse les événements de sécurité et déclenche les alertes.

Screenshot 47 - Vérification du Manager avec `systemctl status wazuh-manager`

```

root@debian-wazu:/home/debian-wazu# systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-10-02 11:45:00 CEST; 2h 49min ago
     Tasks: 210 (limit: 4616)
    Memory: 1.3G
       CPU: 12min 50.439s
   CGroup: /system.slice/wazuh-manager.service
           └─54754 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─54795 /var/ossec/bin/wazuh-authd
               └─54809 /var/ossec/bin/wazuh-db
                 └─54823 /var/ossec/bin/wazuh-execd
                   └─54859 /var/ossec/bin/wazuh-analysisd
                     └─54861 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                       └─54862 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                         └─54865 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                           └─54868 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                             └─54911 /var/ossec/bin/wazuh-syscheckd
                               └─54925 /var/ossec/bin/wazuh-remoted
                                 └─54962 /var/ossec/bin/wazuh-logcollector
                                   └─54981 /var/ossec/bin/wazuh-monitord
                                     └─55001 /var/ossec/bin/wazuh-modulesd

oct. 02 11:44:53 debian-wazu env[54692]: Started wazuh-syscheckd...
oct. 02 11:44:55 debian-wazu env[54692]: Started wazuh-remoted...
oct. 02 11:44:56 debian-wazu env[54692]: Started wazuh-logcollector...
oct. 02 11:44:57 debian-wazu env[54692]: Started wazuh-monitord...
oct. 02 11:44:57 debian-wazu env[54990]: 2025/10/02 11:44:57 wazuh-modulesd:router: INFO: Loaded router module.
oct. 02 11:44:57 debian-wazu env[54990]: 2025/10/02 11:44:57 wazuh-modulesd:content_manager: INFO: Loaded content_m>
oct. 02 11:44:57 debian-wazu env[54990]: 2025/10/02 11:44:57 wazuh-modulesd:inventory-harvester: INFO: Loaded Inven>
oct. 02 11:44:58 debian-wazu env[54692]: Started wazuh-modulesd...
oct. 02 11:45:00 debian-wazu env[54692]: Completed.
oct. 02 11:45:00 debian-wazu systemd[1]: Started wazuh-manager.service - Wazuh manager.
lines 1-32/32 (END)

```

L'Indexer (basé sur OpenSearch) doit être opérationnel pour stocker et indexer nos logs de sécurité.

Screenshot 48 - Vérification de l'Indexer avec `systemctl status wazuh-indexer`

```

root@debian-wazu:/home/debian-wazu# systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-10-02 11:09:51 CEST; 3h 25min ago
     Docs: https://documentation.wazuh.com
    Main PID: 54169 (java)
       Tasks: 62 (limit: 4616)
      Memory: 1.1G
         CPU: 1min 35.559s
   CGroup: /system.slice/wazuh-indexer.service
           └─54169 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -D>

oct. 02 11:09:16 debian-wazu systemd-entrypoint[54169]: WARNING: System::setSecurityManager has been called by org.>
oct. 02 11:09:16 debian-wazu systemd-entrypoint[54169]: WARNING: Please consider reporting this to the maintainers >
oct. 02 11:09:16 debian-wazu systemd-entrypoint[54169]: WARNING: System::setSecurityManager will be removed in a fu>
oct. 02 11:09:17 debian-wazu systemd-entrypoint[54169]: oct. 02, 2025 11:09:17 AM sun.util.locale.provider.LocalePr>
oct. 02 11:09:17 debian-wazu systemd-entrypoint[54169]: WARNING: COMPAT locale provider will be removed in a future>
oct. 02 11:09:18 debian-wazu systemd-entrypoint[54169]: WARNING: A terminally deprecated method in java.lang.System>
oct. 02 11:09:18 debian-wazu systemd-entrypoint[54169]: WARNING: System::setSecurityManager has been called by org.>
oct. 02 11:09:18 debian-wazu systemd-entrypoint[54169]: WARNING: Please consider reporting this to the maintainers >
oct. 02 11:09:18 debian-wazu systemd-entrypoint[54169]: WARNING: System::setSecurityManager will be removed in a fu>
oct. 02 11:09:51 debian-wazu systemd[1]: Started wazuh-indexer.service - wazuh-indexer.
lines 1-21/21 (END)

```

Nous pouvons confirmer que les trois services essentiels sont actifs et fonctionnent correctement. Si l'un des services affiche un statut « *failed* » ou « *inactive* », nous devons consulter les logs avec *journalctl -xeu nom-du-service* pour diagnostiquer le problème.

[Etape 27 → Accès à l'interface web Wazuh](#)

Maintenant que tous nos services sont opérationnels, nous pouvons accéder à l'interface web de *Wazuh* pour commencer à utiliser notre plateforme de sécurité.

Une fois l'installation terminée, il faut se connecter avec <https://192.168.111.62>

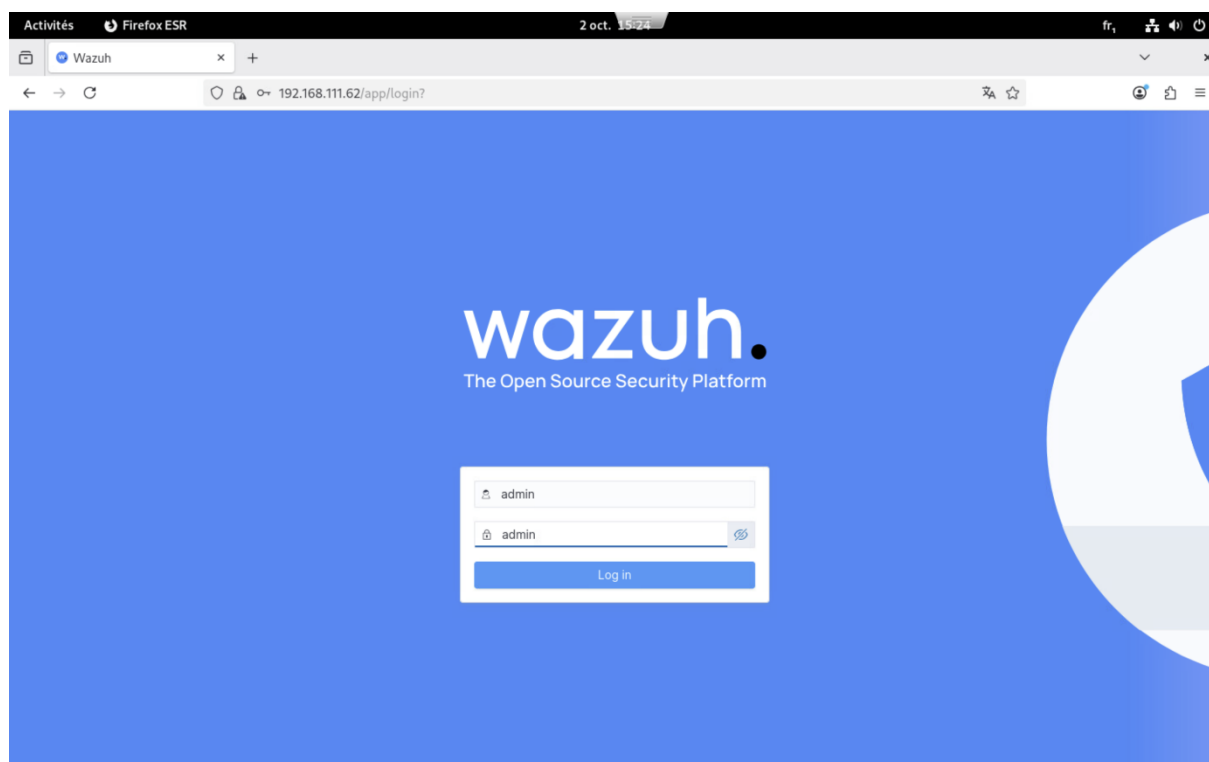
Nous utilisons les identifiants par défaut :

- **Nom d'utilisateur** → admin
- **Mot de passe** → admin

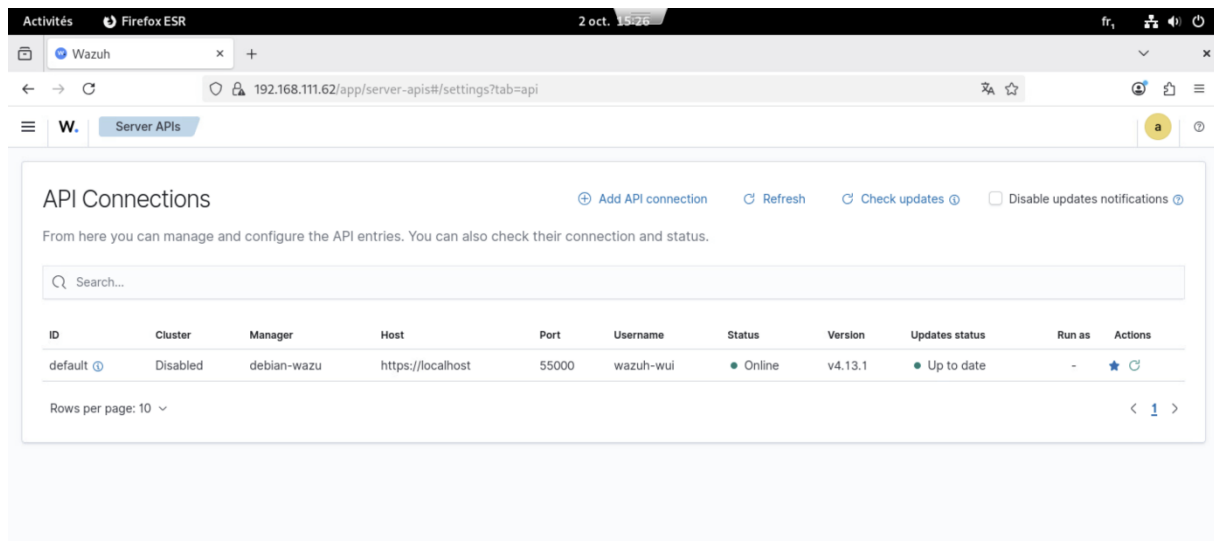
Ces identifiants par défaut ne sont absolument pas sécurisés et ne doivent **JAMAIS** être conservés en production. Cette configuration est acceptable uniquement pour notre environnement de test initial.

De plus, lors de la première connexion, le navigateur nous a affiché un avertissement concernant le certificat SSL auto-signé. Dans notre environnement de test, nous avons accepté ce certificat. En production, nous devrions utiliser des certificats signés par une autorité de certification reconnue.

Screenshot 49 - Première connexion à l'interface web Wazuh



Screenshot 50 - Interface web de Wazuh



Modification des identifiants, obligatoire avant toute utilisation en production.

Via l'interface web :

1. Menu hamburger (≡) → Security → Internal users
2. Sélectionner "admin" → Edit
3. Changer le mot de passe → Save

Étape 28 → Configuration de l'adresse IP fixe

Pour garantir la stabilité de notre installation et éviter que l'adresse IP ne change de manière dynamique avec le DHCP, nous configurons maintenant une adresse IP statique pour notre VM.

Pourquoi utiliser une IP fixe ?

Une adresse IP dynamique poserait plusieurs problèmes :

- Les agents *Wazuh* ne pourraient plus communiquer avec le Manager si l'IP change
- Nous perdrons l'accès au Dashboard
- Les certificats SSL seraient invalidés (car liés à l'IP)

Screenshot 51 - Configuration d'une adresse IP fixe

Annuler

Filaire

Appliquer

DétailsIdentitéIPv4IPv6Sécurité

Méthode IPv4

☐ Automatique (DHCP)☐ Réseau local seulement
☒ Manuel☐ Désactiver
☐ Partagée avec d'autres ordinateurs

Adresses

Adresse	Masque de réseau	Passerelle	
192.168.111.62	255.255.255.0	192.168.111.250	⊗
			⊗

Conclusion partie 1

Il nous était demandé de déployer une solution SIEM complète en installant *Wazuh* sur une machine virtuelle Debian 12, avec la configuration de l'ensemble des composants (*Manager*, *Indexer* et *Dashboard*) pour obtenir une plateforme de supervision de la sécurité opérationnelle.

Nous avons réussi à mettre en place une installation complète de *Wazuh* en architecture **All-in-One**. Nous avons configuré le Dashboard avec communication HTTPS sécurisée, déployé les certificats SSL/TLS avec les permissions appropriées, démarré et vérifié tous les services essentiels, sécurisé l'accès à l'interface web, et configuré une adresse IP statique pour garantir la stabilité de la plateforme.

Les principales difficultés ont concerné la gestion des certificats SSL/TLS et leurs permissions restrictives, la connectivité entre les différents composants nécessitant une configuration réseau précise, les problèmes de démarrage de services nécessitant l'analyse des logs *systemd*, et la vigilance requise face aux identifiants par défaut non sécurisés.

Ce projet nous a permis de développer des compétences essentielles en architecture SIEM (compréhension des interactions entre collecte, indexation, analyse et visualisation), en gestion des certificats SSL/TLS, en administration Linux (*systemd*, configuration réseau, permissions), et en bonnes pratiques de sécurité. Nous avons également amélioré nos capacités de documentation technique.

Pour évoluer vers une solution de production, plusieurs améliorations sont envisageables :

- Remplacer les certificats auto-signés par des certificats signés par une CA reconnue
- Implémenter l'authentification multi-facteurs
- Déployer une architecture haute disponibilité avec plusieurs nœuds en cluster
- Mettre en place des sauvegardes automatisées
- Déployer des agents *Wazuh* sur les systèmes à superviser
- Développer des règles de détection personnalisées
- Intégrer *Wazuh* à d'autres outils de sécurité (SOAR, ticketing).
- La configuration de rapports de conformité automatisés (PCI-DSS, GDPR, ISO 27001) permettrait également de répondre aux exigences réglementaires.

Notre installation *Wazuh* est maintenant opérationnelle et constitue une base solide pour la supervision de la sécurité de notre infrastructure. Nous sommes prêts à étendre sa supervision et à continuer notre apprentissage dans ce domaine en constante évolution.

Partie 2 :

Intégration de deux machines dans WAZUH → Linux & Windows

Introduction

Dans cette partie, nous allons intégrer deux machines virtuelles dans notre infrastructure *Wazuh* :

- Une machine Debian 12
- Une machine Windows 10

L'objectif est de déployer les agents *Wazuh* sur ces deux systèmes pour permettre leur surveillance centralisée.

Informations de configuration :

- Adresse IP du serveur Wazuh Manager → 192.168.111.62
- ✓ Agent Linux → Wazuh-Secours
- ✓ Agent Windows → Win10

Prérequis :

Avant de commencer l'intégration, nous nous assurons que :

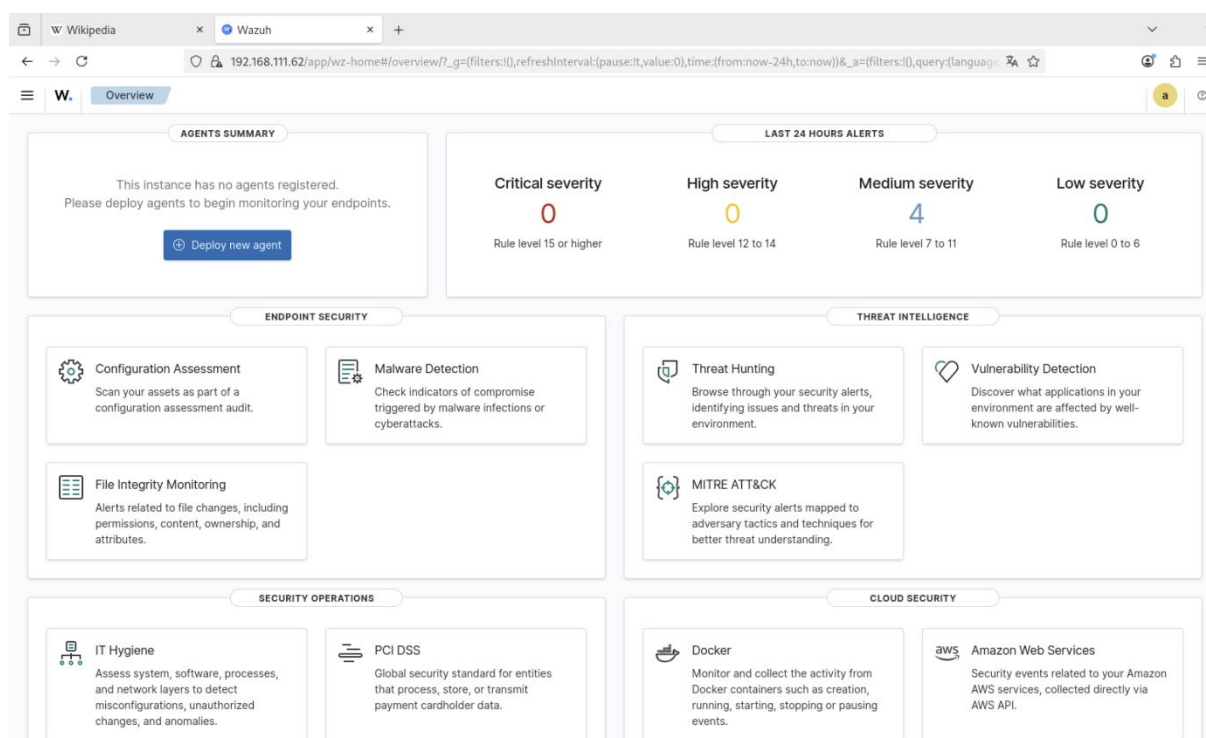
- Les machines virtuelles peuvent communiquer avec le serveur *Wazuh* (192.168.111.62)
- Le pare-feu autorise les connexions sur les ports 1514 (*agent communication*) et 1515 (*agent enrollment*)
- Nous disposons des droits administrateurs sur les deux machines

1. Intégration de la machine Debian Linux

1.1 Connexion à l'interface Web Wazuh

Nous commençons par nous connecter à l'interface Web de *Wazuh* depuis notre VM Debian cliente. Nous accédons à l'URL du serveur *Wazuh* via un navigateur Web à l'adresse <https://192.168.111.62>.

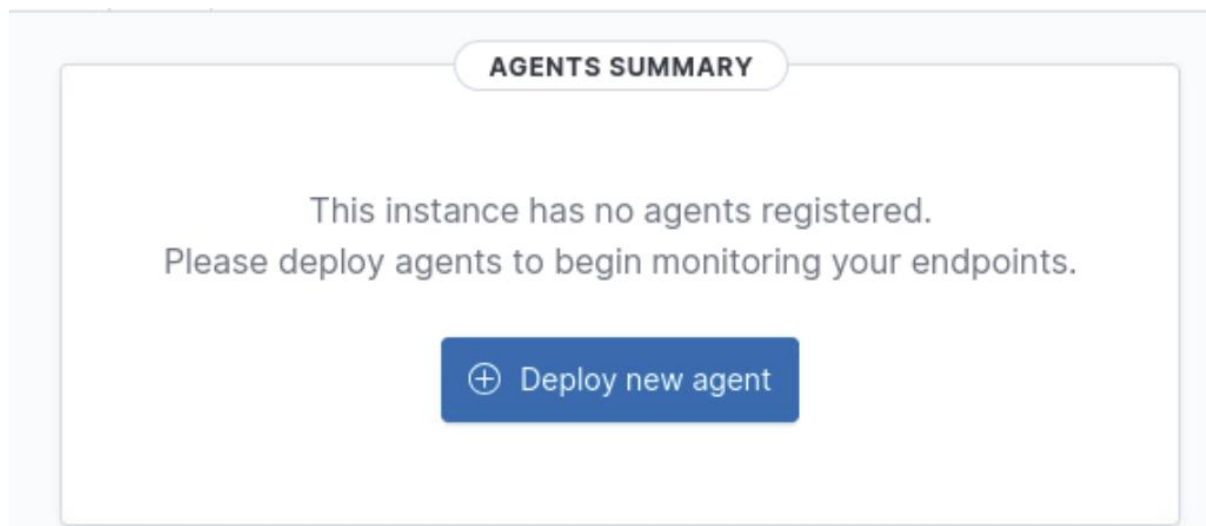
Screenshot 52 - Interface web de Wazuh sur la debian client



1.2 Ajout d'un nouvel agent

Une fois connectés à l'interface, nous cliquons sur le bouton « **Deploy new agent** » pour démarrer le processus d'intégration.

Screenshot 53 - Onglet permettant de "deploy new agent"



1.3 Sélection du système d'exploitation

Nous indiquons qu'il s'agit d'une machine **Linux** en sélectionnant l'option correspondante dans l'interface. L'interface nous guide ensuite à travers les étapes de configuration :

- **Système d'exploitation** → DEB amd64
- **Adresse du serveur** → 192.168.111.62
- **Nom de l'agent** → Wazuh-Secours
- **Groupe** → default

Screenshot 54 - Procédure pour déployer un nouvel agent (1/3)

Wikipedia x Wazuh x +

192.168.111.62/app/endpoints-summary#/agents-preview/deploy

Endpoints Deploy new agent

✓ Select the package to download and install on your system:

LINUX

☐ RPM amd64 ☐ RPM aarch64

☒ DEB amd64 ☐ DEB aarch64

WINDOWS

☐ MSI 32/64 bits

macOS

☐ Intel ☐ Apple silicon

For additional systems and architectures, please check our [documentation](#).

✓ Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address

192.168.111.62

☐ Remember server address

3 Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name:

Agent name

Screenshot 55 - Procédure pour déployer un nouvel agent (2/3)

✓ Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name:

Wazuh-Secours

The agent name must be unique. It can't be changed once the agent has been enrolled.

Select one or more existing groups:

default

4 Run the following commands to download and install the agent:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.13.1-1_amd64.deb && sudo WAZUH_MANAGER=192.168.111.62 WAZUH_AGENT_GROUP=default WAZUH_AGENT_NAME=Wazuh-Secours dpkg -i ./wazuh-agent_4.13.1-1_amd64.deb
```

Requirements

- You will need administrator privileges to perform this installation.
- Shell Bash is required.

Screenshot 56 - Procédure pour déployer un nouvel agent (3/3)

4 Run the following commands to download and install the agent:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.13.1-1_amd64.deb && sudo WAZUH_MANAGER='192.168.111.62' WAZUH_AGENT_GROUP='default' WAZUH_AGENT_NAME='Wazuh-Secours' dpkg -i ./wazuh-agent_4.13.1-1_amd64.deb
```

Requirements

- You will need administrator privileges to perform this installation.
- Shell Bash is required.

Keep in mind you need to run this command in a Shell Bash terminal.

5 Start the agent:

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

1.4 Téléchargement et installation de l'agent

L'interface nous fournit les commandes nécessaires pour télécharger et installer l'agent *Wazuh* sur notre machine Debian. Nous exécutons la commande suivante dans le terminal de la VM Debian :

Screenshot 57 - Téléchargement et installation de l'agent avec wget

```
https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.13.1-1_amd64.deb && sudo WAZUH_MANAGER='192.168.111.62' WAZUH_AGENT_GROUP='default' WAZUH_AGENT_NAME='Wazuh-Secours' dpkg -i w
```

```

+      debian-wazu@debian-wazu2: ~
68.111.62' WAZUH_AGENT_GROUP='default' WAZUH_AGENT_NAME='Wazuh-Secours' dpkg -i
./wazuh-agent_4.13.1-1_amd64.deb
--2025-10-07 09:21:04-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-ag
ent/wazuh-agent_4.13.1-1_amd64.deb
Résolution de packages.wazuh.com (packages.wazuh.com)... 108.158.32.121, 108.158.3
2.124, 108.158.32.25, ...
Connexion à packages.wazuh.com (packages.wazuh.com)|108.158.32.121|:443... connect
é.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 12075052 (12M) [application/vnd.debian.binary-package]
Sauvegarde en : « wazuh-agent_4.13.1-1_amd64.deb.2 »

wazuh-agent_4.13.1- 100%[=====] 11,52M --.-KB/s ds 0,1s

2025-10-07 09:21:04 (109 MB/s) - « wazuh-agent_4.13.1-1_amd64.deb.2 » sauvegardé
[12075052/12075052]

Sélection du paquet wazuh-agent précédemment désélectionné.
(Lecture de la base de données... 154988 fichiers et répertoires déjà installés.
)
Préparation du dépaquetage de .../wazuh-agent_4.13.1-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.13.1-1) ...
Paramétrage de wazuh-agent (4.13.1-1) ...
root@debian-wazu2: /home/debian-wazu#

```

Détails de la commande :

- `wget` → télécharge le package d'installation de l'agent Wazuh version 4.13.1
- `WAZUH_MANAGER='192.168.111.62'` → définit l'adresse IP du serveur Wazuh Manager
- `WAZUH_AGENT_GROUP='default'` → assigne l'agent au groupe par défaut
- `WAZUH_AGENT_NAME='Wazuh-Secours'` → définit le nom de l'agent
- `dpkg -i` → installe le package téléchargé

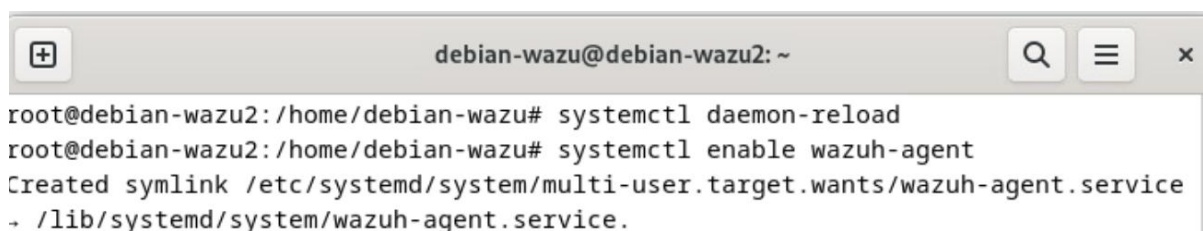
1.5 Démarrage de l'agent**Après l'installation, nous démarrons et activons le service de l'agent Wazuh :**

- Pour recharger la configuration systemd → `systemctl daemon-reload`
- Pour activer le démarrage automatique de l'agent au boot → `systemctl enable wazuh-agent`
- Pour démarrer immédiatement le service → `systemctl start wazuh-agent`

Ces commandes permettent de :

- ✓ Recharger la configuration `systemd`
- ✓ Activer le démarrage automatique de l'agent au boot
- ✓ Démarrer immédiatement le service

Screenshot 58 - Démarrage et activation de l'agent Wazuh avec `systemctl daemon-reload` ; `systemctl enable wazuh-agent` ; `systemctl start wazuh-agent`



```

root@debian-wazu2: /home/debian-wazu# systemctl daemon-reload
root@debian-wazu2: /home/debian-wazu# systemctl enable wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service
→ /lib/systemd/system/wazuh-agent.service.
```

1.6 Vérification de l'état de l'agent

Nous vérifions que l'agent est bien actif et connecté au serveur Wazuh :

Screenshot 59 - Vérification de l'état de l'agent avec `systemctl status wazuh-agent`

```
root@debian-wazu2: /home/debian-wazu# systemctl start wazuh-agent
```

Nous devons observer un statut « **active (running)** » en vert, confirmant que le service fonctionne correctement, ce qui est actuellement effectivement le cas.

Screenshot 60 - Confirmation du bon fonctionnement de l'agent Wazuh

[illegible]

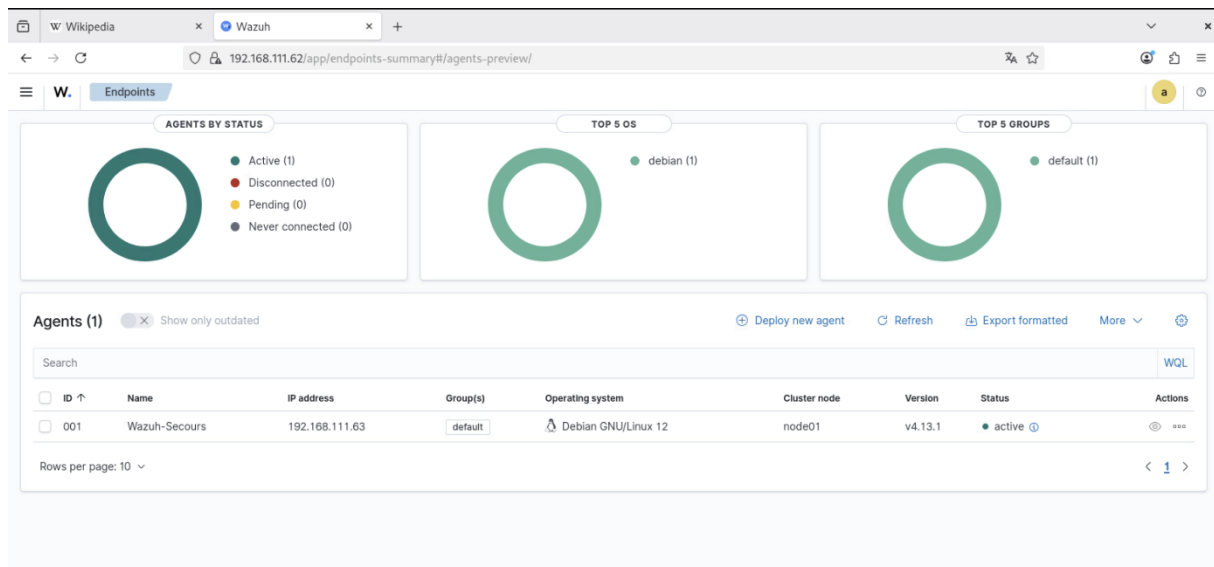
1.7 Confirmation dans l'interface Web

Nous retournons sur l'interface Web de *Wazuh* et accédons à la section « **Agent** » pour confirmer que l'agent **Wazuh-Secours** apparaît bien dans la liste des agents connectés avec le statut « **Active** ».

Dans le tableau de bord, nous pouvons voir :

- **Le nom de l'agent** → Wazuh-Secours
- **Le système d'exploitation** → Debian
- **L'adresse IP de la machine**
- **Le statut** → Active (avec un indicateur vert)
- **La version de l'agent**

Screenshot 61 - Confirmation de l'ajout de notre agent sur l'interface web Wazuh

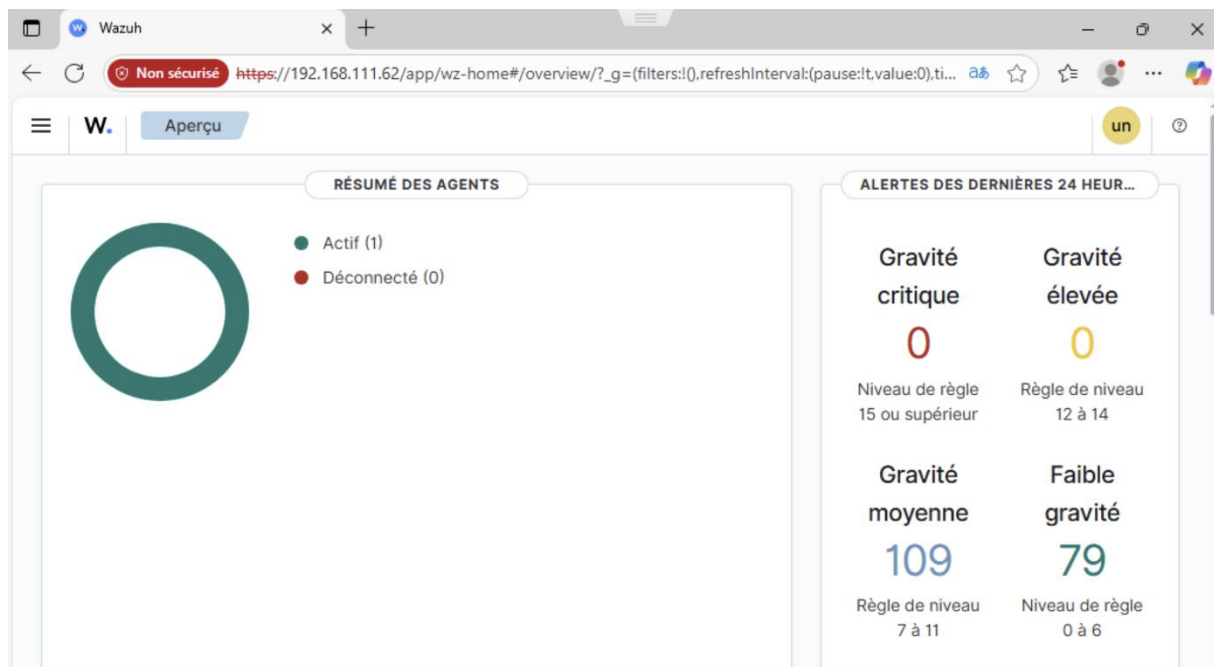


2. Intégration de la machine Windows 10

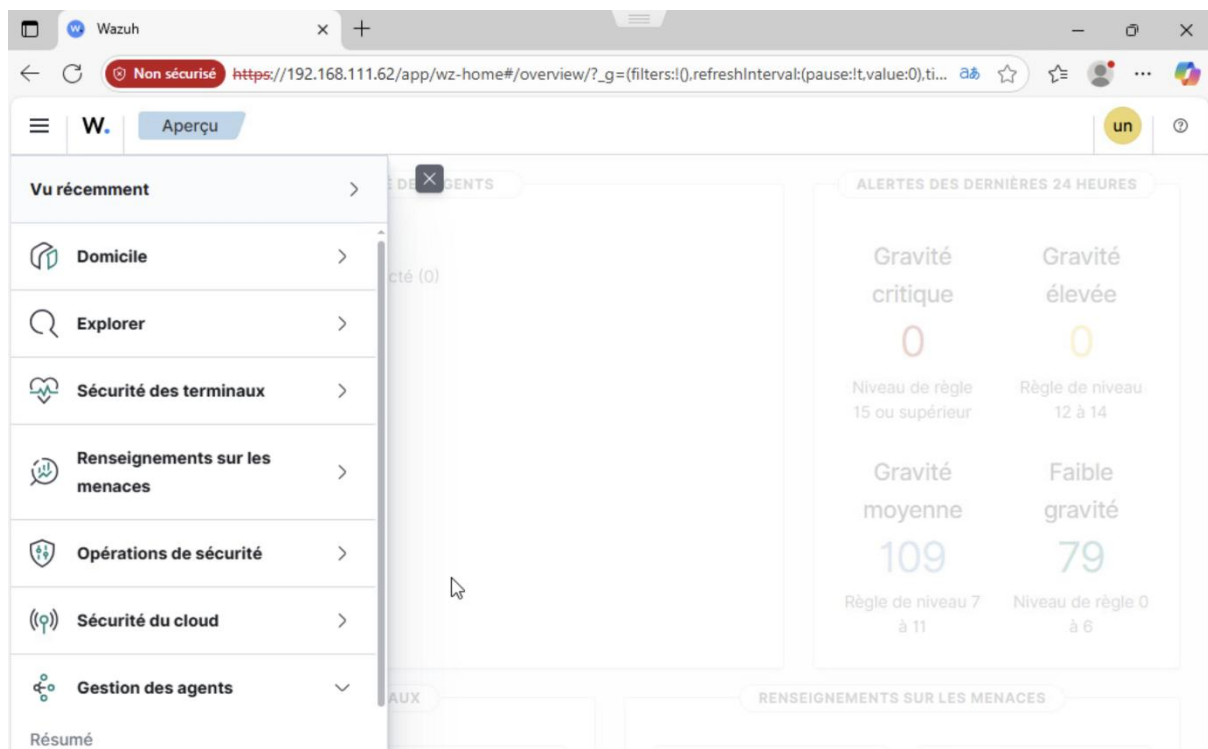
2.1 Ajout d'un nouvel agent Windows

De retour sur l'interface Web de Wazuh, nous cliquons à nouveau sur « **Ajouter un nouvel agent** » pour intégrer notre machine Windows 10.

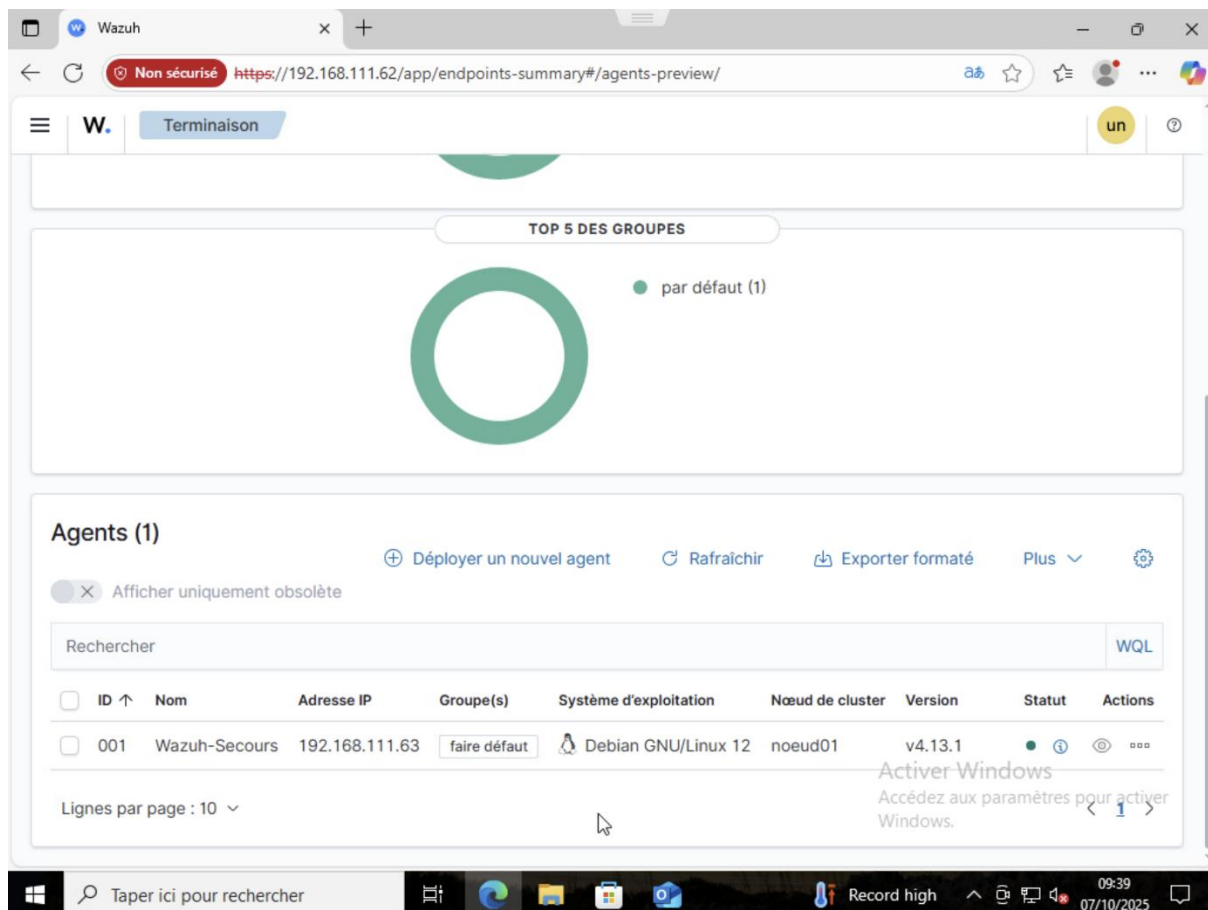
Screenshot 62 - Interface web de Wazuh sur Windows10



Screenshot 63 - Navigation vers "Gestion des agents" > "Résumé"



Screenshot 64 - Interface permettant de "déployer un nouvel agent"

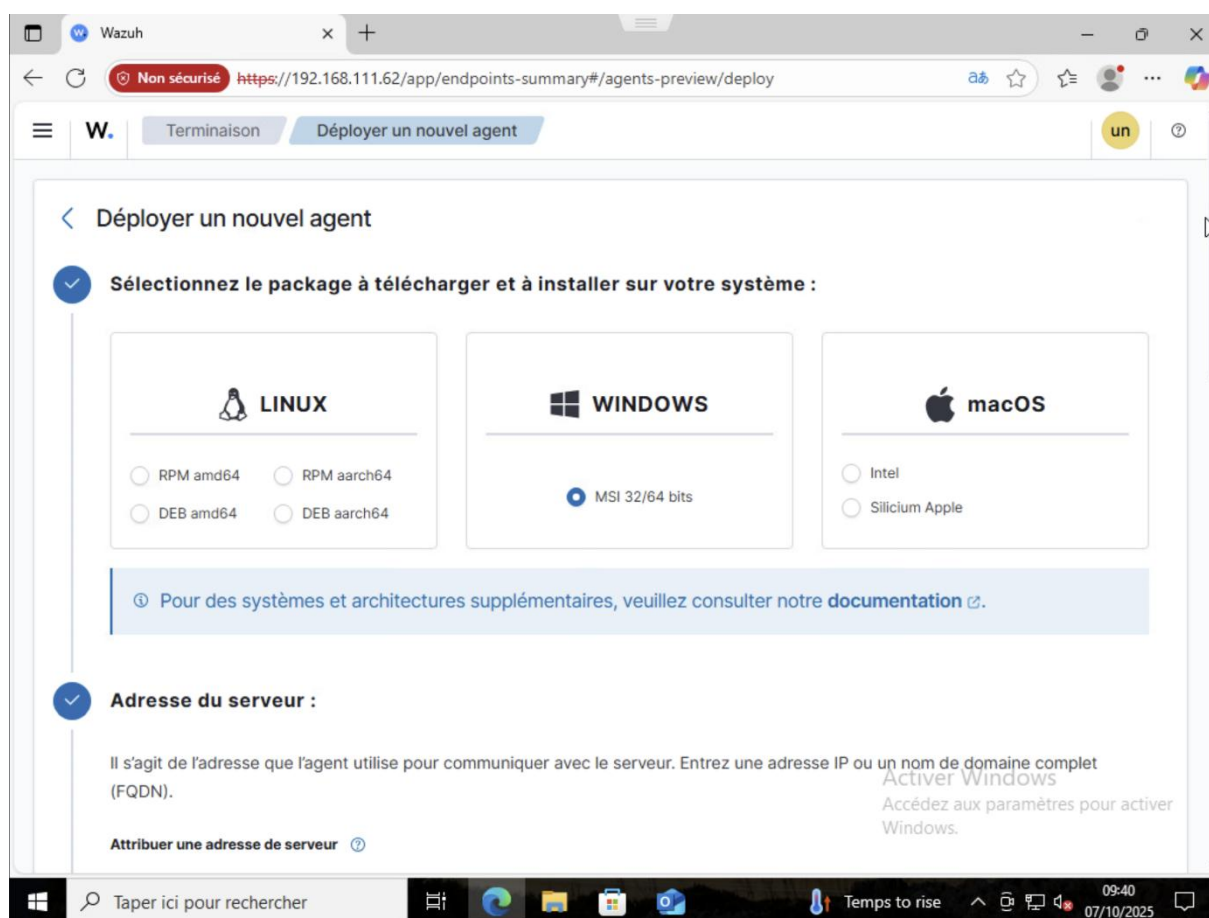


2.2 Sélection du système d'exploitation

Nous sélectionnons **Windows** comme système d'exploitation cible. L'interface nous guide à travers la configuration :

- ✓ **Système d'exploitation** → Windows
- ✓ **Adresse du serveur** → 192.168.111.62
- ✓ **Nom de l'agent** → Win10
- ✓ **Groupe** → default

Screenshot 65 - Procédure pour déployer un nouvel agent (1/3)



Screenshot 66 - Procédure pour déployer un nouvel agent (2/3)

Wazuh

Non sécurisé <https://192.168.111.62/app/endpoints-summary#/agents-preview/deploy>

Terminaison Déployer un nouvel agent

✓ Paramètres facultatifs :

Par défaut, le déploiement utilise le nom d'hôte comme nom d'agent. Si vous le souhaitez, vous pouvez utiliser un autre nom d'agent dans le champ ci-dessous.

Attribuez un nom d'agent : ?

Win10

ⓘ Le nom de l'agent doit être unique. Il ne peut pas être modifié une fois que l'agent a été inscrit. ?

Sélectionnez un ou plusieurs groupes existants : ?

faire défaut ×

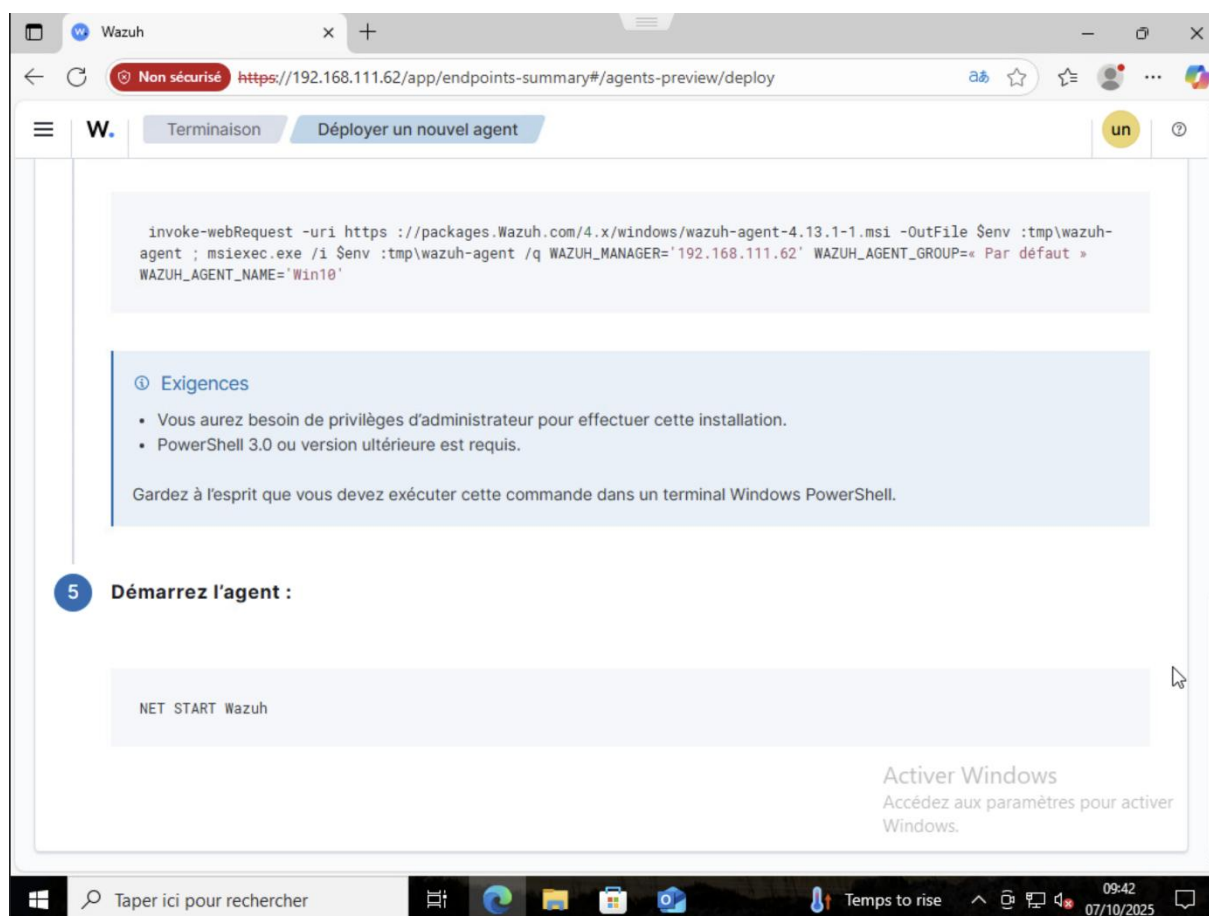
4 Exécutez les commandes suivantes pour télécharger et installer l'agent :

```
invoke-webRequest -uri https://packages.Wazuh.com/4.x/windows/wazuh-agent-4.13.1-1.msi -OutFile $env :tmp\wazuh-agent ; msixec.exe /i $env :tmp\wazuh-agent /q WAZUH_MANAGER='192.168.111.62' WAZUH_AGENT_GROUP=' Par défaut '
```

Activer Windows
Accédez aux paramètres pour activer Windows

Temps to rise 09:41 07/10/2025

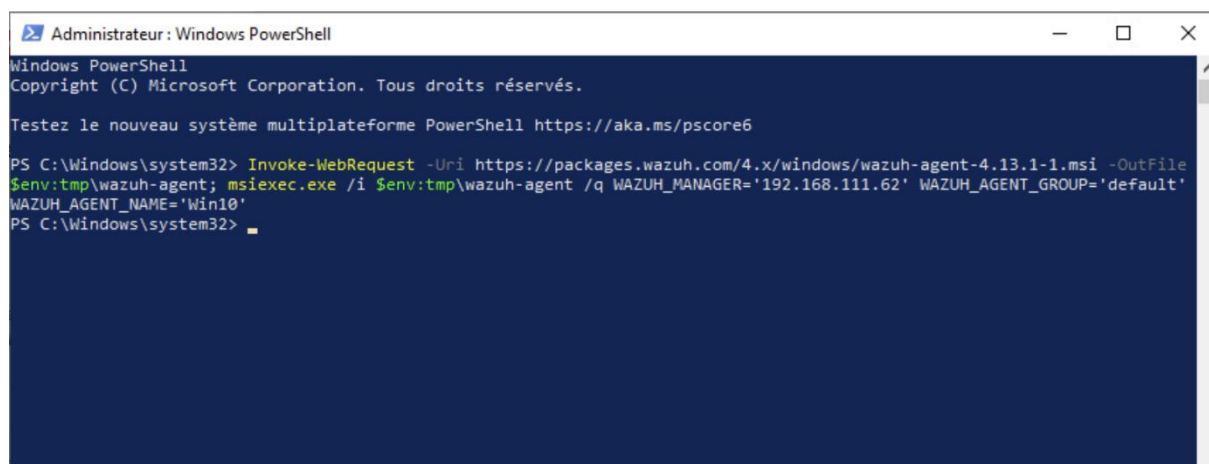
Screenshot 67 - Procédure pour déployer un nouvel agent (3/3)



2.3 Téléchargement et installation de l'agent

Nous ouvrons **PowerShell en tant qu'administrateur** sur la machine Windows 10 et exécutons la commande suivante :

Screenshot 68 - Téléchargement et installation de l'agent avec `Invoke-webrequest -uri https://packages.Wazuh.com/4.x.windows/wazuh-agent-4.13.1-1.msi -OutFile $env:tmp\wazuh-agent ; msixec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.111.62' WAZUH_AGENT_GRO`



Détails de la commande :

- *Invoke-WebRequest* → télécharge le fichier MSI de l'agent *Wazuh* pour Windows
- *OutFile \$env:tmp\wazuh-agent.msi* → enregistre le fichier dans le dossier temporaire
- *msiexec.exe /i* → lance l'installation du MSI
- */q* → mode silencieux (sans interface graphique)
- *WAZUH_MANAGER='192.168.111.62'* → adresse du serveur *Wazuh Manager*
- *WAZUH_AGENT_GROUP='default'* → groupe d'appartenance de l'agent
- *WAZUH_AGENT_NAME='Win10'* → nom de l'agent Windows

2.4 Démarrage du service

Après l'installation, nous démarrons le service *Wazuh* avec la commande suivante :

Screenshot 69 - Démarrage du service avec NET START Wazuh

```
PS C:\Windows\system32> NET START Wazuh
Le service Wazuh démarre.
Le service Wazuh a démarré.

PS C:\Windows\system32> █
```

Nous pouvons voir le message → « **Le service WazuhSvc a démarré.** »

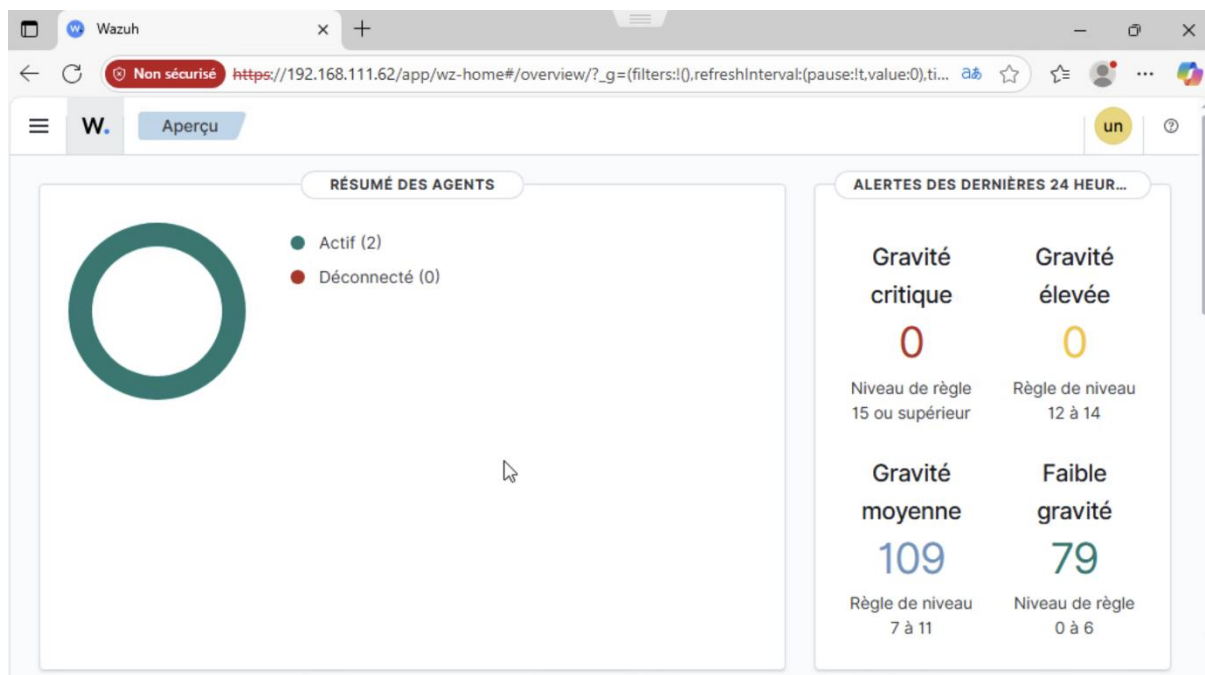
2.5 Confirmation dans l'interface Web

Nous retournons sur l'interface Web de *Wazuh* pour vérifier que l'agent **Win10** apparaît dans la liste des agents avec le statut « *Active* ».

Dans le tableau de bord, nous pouvons maintenant voir :

- ✓ **Le nom de l'agent** → Win10
- ✓ **Le système d'exploitation** → Windows 10
- ✓ **L'adresse IP de la machine**
- ✓ **Le statut** → Active (avec un indicateur vert)
- ✓ **La version de l'agent**

Screenshot 70 - Vérification que notre WIN10 est bien déployée comme nouvel agent



Screenshot 71 - Validation du déploiement de notre nouvel agent Win10

The screenshot shows the Wazuh dashboard endpoints summary. The top section displays a donut chart indicating 2 agents by default (green). Below, the 'Agents (2)' section shows a table of active agents. The table includes columns for ID, Nom, Adresse IP, Groupe(s), Système d'exploitation, Nœud de cluster, Version, Statut, and Actions. Two agents are listed: 'Wazuh-Secours' (Debian GNU/Linux 12) and 'Win10' (Microsoft Windows 10 Professional 10.0.19045.6332). Both agents are active and located on 'noeud01'.

ID	Nom	Adresse IP	Groupe(s)	Système d'exploitation	Nœud de cluster	Version	Statut	Actions
001	Wazuh-Secours	192.168.11.1.63	faire défaut	Debian GNU/Linux 12	noeud01	v4.13.1	●	ⓘ Ⓞ ⋮
002	Win10	192.168.11.1.11	faire défaut	Microsoft Windows 10 Professionnel 10.0.19045.6332	noeud01	v4.13.1	●	ⓘ Ⓞ ⋮

Agents (2) actions: Déployer un nouvel agent, Rafraîchir, Exporter formaté, Plus, WQL.

status=active

Lignes par page : 10

Activer Windows
Accédez aux paramètres pour activer Windows.

3. Vérification finale

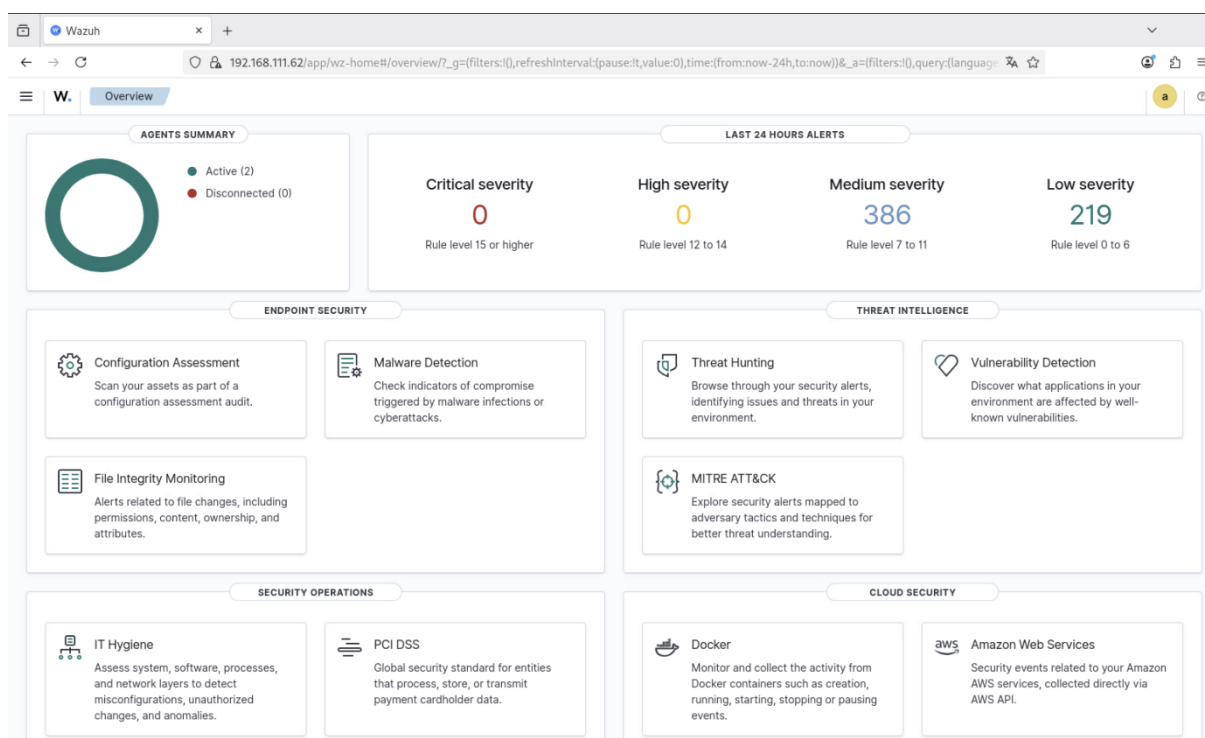
3.1 Vue d'ensemble des agents

Dans l'interface Web de *Wazuh*, nous accédons à la section « **Agents** » pour visualiser l'ensemble de nos agents déployés. Nous devrions voir :

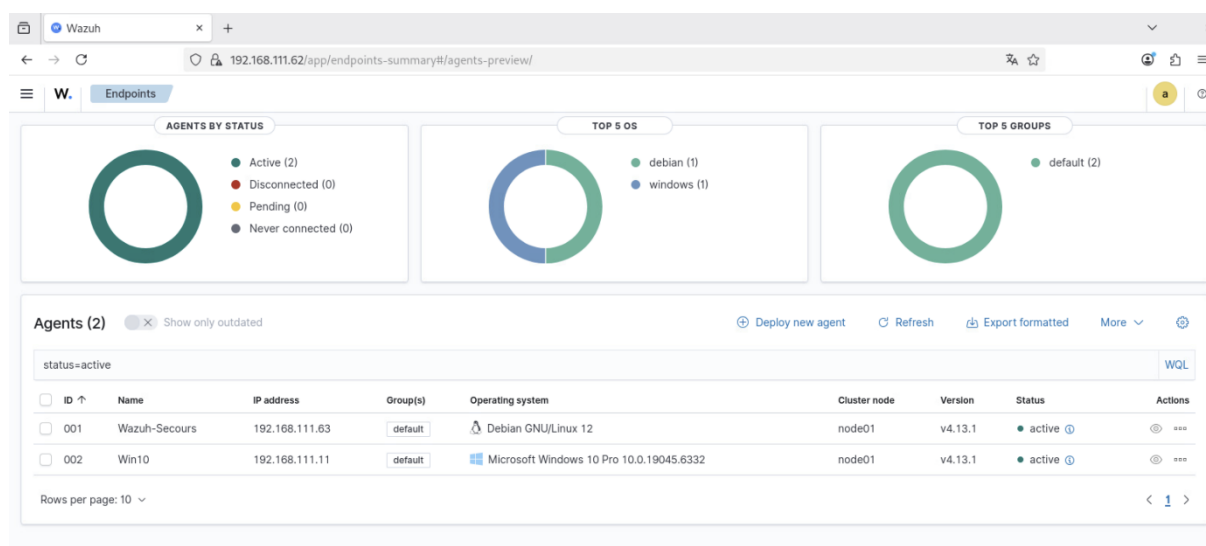
- **Wazuh-Secours** (Debian Linux) - Statut → Active
- **Win10** (Windows 10) - Statut → Active

Les deux agents affichent un indicateur vert et le statut « **Active** », confirmant qu'ils communiquent correctement avec le serveur *Wazuh Manager*.

Screenshot 72 - Interface web depuis notre VM serveur



Screenshot 73 - Confirmation que les deux agents sont bien déployés



Nous pouvons confirmer que nos deux nouveaux agents sont bien déployés et en fonctionnement.

3.2 Surveillance des événements

Nous pouvons maintenant commencer à surveiller les événements de sécurité remontés par nos deux agents.

L'interface Wazuh nous permet de :

- Consulter les alertes en temps réel → Section « *Security events* »
- Analyser les logs système → Visualisation par agent
- Détecter les anomalies de sécurité → Tableau de bord des menaces
- Générer des rapports de conformité → PCI-DSS, GDPR, CIS
- Surveiller l'intégrité des fichiers (FIM) → Détection de modifications
- Analyser les vulnérabilités → Scan automatique des systèmes

3.3 Premières alertes

Après quelques minutes, nous devrions commencer à voir des événements remontés par les agents :

- Événements de connexion utilisateur
- Démarrage/arrêt de services
- Modifications de configuration

- Tentatives d'authentification
- Activité réseau suspecte

Conclusion partie 2

Nous avons réussi à intégrer deux machines (Linux et Windows) dans notre infrastructure *Wazuh*. Les agents sont maintenant opérationnels et communiquent avec le serveur *Wazuh Manager*. Cette configuration nous permet de bénéficier d'une surveillance centralisée de la sécurité de nos systèmes.

Points clés à retenir :

- Les agents *Wazuh* nécessitent l'adresse IP du serveur *Manager* pour établir la communication
- L'installation peut être automatisée via des commandes en ligne pour faciliter le déploiement à grande échelle
- La vérification du statut des agents est essentielle pour s'assurer du bon fonctionnement
- L'interface Web offre une vue centralisée de tous les agents déployés
- Les ports 1514 et 1515 doivent être accessibles depuis les agents vers le *Manager*
- Les logs des agents sont précieux pour le diagnostic en cas de problème

Prochaines étapes → Maintenant que nos agents sont déployés, nous pouvons :

1. **Personnaliser les règles de détection** selon nos besoins spécifiques
2. **Configurer la surveillance de l'intégrité des fichiers (FIM)** sur des répertoires critiques
3. **Mettre en place des alertes email** pour les événements critiques
4. **Créer des tableaux de bord personnalisés** pour nos équipes
5. **Intégrer d'autres agents** sur le reste de notre infrastructure
6. **Tester des scénarios d'attaque** pour valider la détection (*Partie 3*)

Notre infrastructure de surveillance est désormais opérationnelle et prête à détecter les menaces de sécurité sur nos systèmes Linux et Windows.

Partie 3 :

Configuration des alertes FIM et détection d'intrusions

Introduction

Dans cette troisième partie, nous allons configurer les fonctionnalités de surveillance avancées de *Wazuh*. Nous verrons comment mettre en place la surveillance d'intégrité des fichiers (*File Integrity Monitoring ci-après FIM*), puis comment détecter et répondre automatiquement aux attaques par force brute.

La surveillance FIM est essentielle pour détecter toute modification non autorisée sur nos systèmes. Elle nous permet d'être alertés en temps réel des changements apportés aux fichiers critiques de notre infrastructure.

Qu'est-ce que le FIM ?

Les alertes FIM sont des alertes générées lorsque des modifications sont apportées aux fichiers ou répertoires surveillés. Il peut s'agir de la création, la modification ou la suppression d'un fichier. Le FIM détecte aussi les changements de permissions de lecture et d'écriture.

Tableau comparatif des modes de surveillance

Mode de surveillance	Temps réel	Identité de l'auteur	Impact performance	Usage recommandé
Basic (scheduled)	Non	Non	Faible	Surveillance générale
Realtime	Oui	Non	Moyen	Fichiers critiques
Whodata	Oui	Oui (utilisateur, processus)	Élevé	Audit de sécurité avancé

4. Configuration FIM de base pour Windows

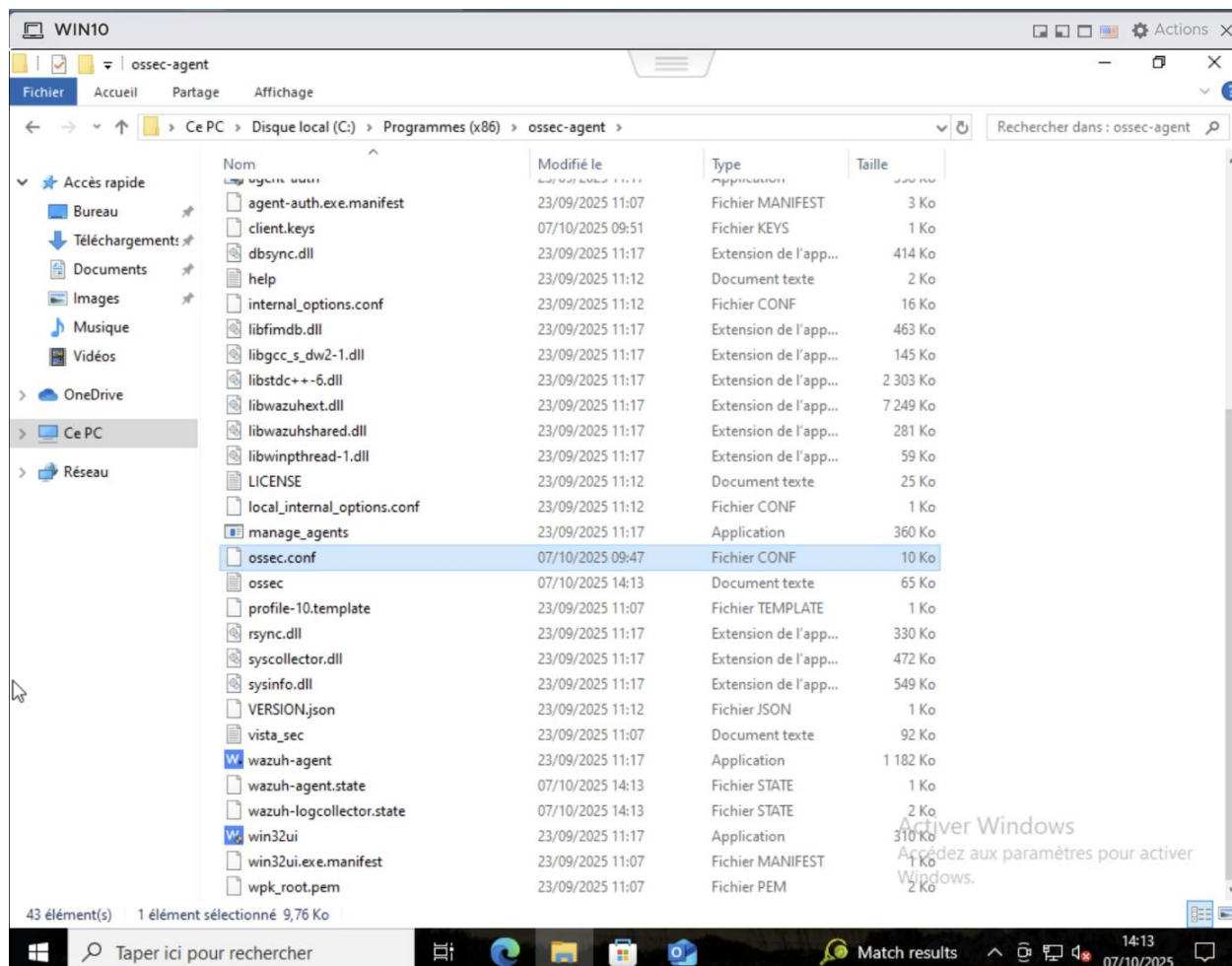
4.1 Modification du fichier de configuration

Nous allons maintenant configurer la surveillance de base sur un agent Windows.

Étape 1 → Accéder au fichier de configuration

Nous nous déplaçons dans le répertoire `C:\Program Files (x86)\ossec-agent\` et nous modifions en tant qu'administrateur le fichier `ossec.conf` avec le Bloc-notes.

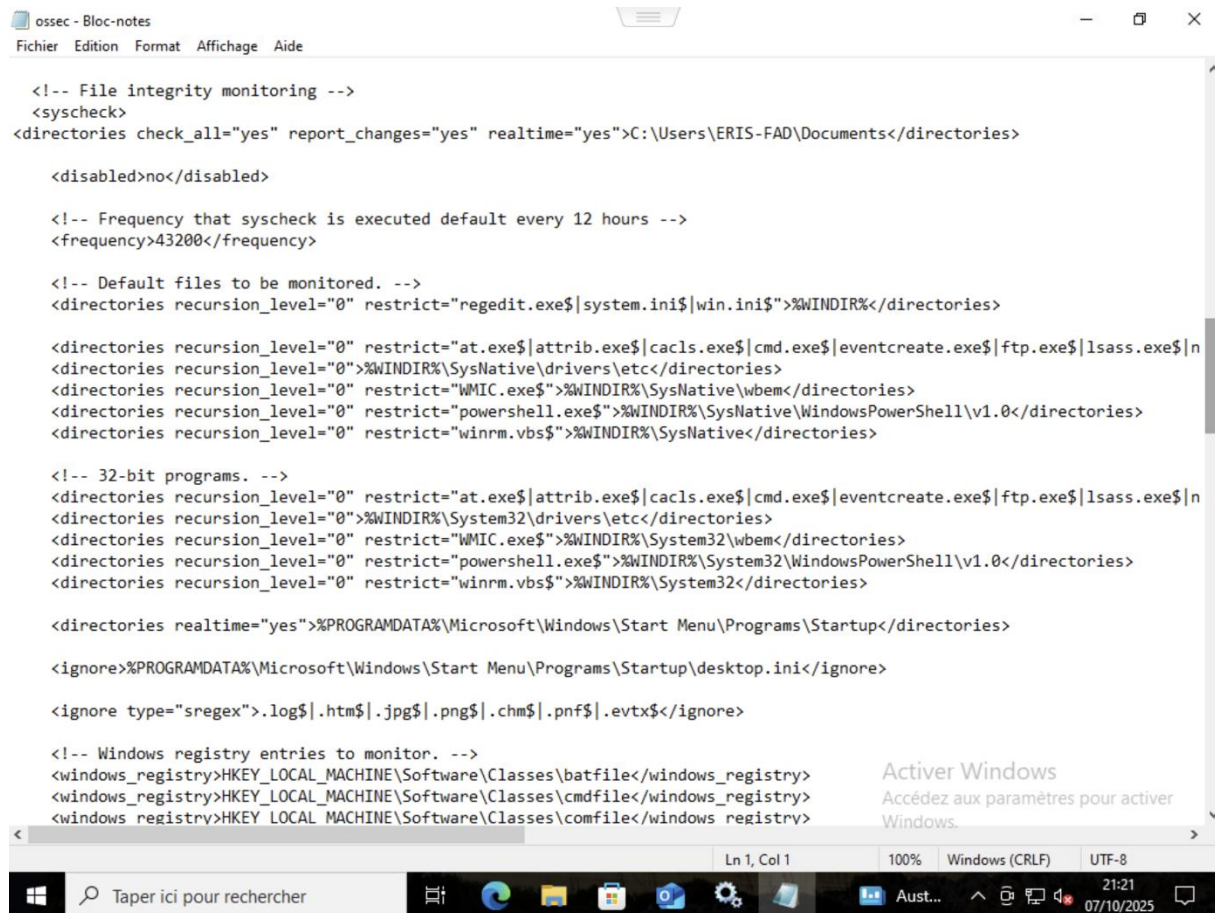
Screenshot 74 - Déplacement vers le répertoire contenant ossec-agent



Étape 2 → Ajouter la directive de surveillance

Nous ajoutons la commande suivante dans le bloc `<syscheck>`. Nous remplaçons `C:\Dossier\A\Surveiller` par le chemin du répertoire que nous souhaitons monitorer :

Screenshot 75 - Nous insérons l'alerte avec `<directories check_all="yes" realtime="yes" report_changes="yes">`



```

<!-- File integrity monitoring -->
<syscheck>
<directories check_all="yes" report_changes="yes" realtime="yes">C:\Users\ERIS-FAD\Documents</directories>

<disabled>no</disabled>

<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>43200</frequency>

<!-- Default files to be monitored. -->
<directories recursion_level="0" restrict="regedit.exe|system.ini|win.ini">%WINDIR%</directories>

<directories recursion_level="0" restrict="at.exe|attrib.exe|cacls.exe|cmd.exe|eventcreate.exe|ftp.exe|lsass.exe|n
<directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
<directories recursion_level="0" restrict="WMIC.exe">%WINDIR%\SysNative\wbem</directories>
<directories recursion_level="0" restrict="powershell.exe">%WINDIR%\SysNative\WindowsPowerShell\v1.0</directories>
<directories recursion_level="0" restrict="winrm.vbs">%WINDIR%\SysNative</directories>

<!-- 32-bit programs. -->
<directories recursion_level="0" restrict="at.exe|attrib.exe|cacls.exe|cmd.exe|eventcreate.exe|ftp.exe|lsass.exe|n
<directories recursion_level="0">%WINDIR%\System32\drivers\etc</directories>
<directories recursion_level="0" restrict="WMIC.exe">%WINDIR%\System32\wbem</directories>
<directories recursion_level="0" restrict="powershell.exe">%WINDIR%\System32\WindowsPowerShell\v1.0</directories>
<directories recursion_level="0" restrict="winrm.vbs">%WINDIR%\System32</directories>

<directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup</directories>

<ignore>%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup\desktop.ini</ignore>

<ignore type="sregex">.log$|.htm$|.jpg$|.png$|.chm$|.pnf$|.evtx$</ignore>

<!-- Windows registry entries to monitor. -->
<windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\batfile</windows_registry>
<windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\cmdfile</windows_registry>
<windows_registry>HKEY_LOCAL_MACHINE\Software\Classes\comfile</windows_registry>

```

Explication des paramètres :

realtime= « yes » → Active la surveillance en temps réel (détection immédiate des modifications)

report_changes= « yes » → Signale toutes les modifications apportées aux fichiers

check_all= « yes » → Surveille toutes les métadonnées des fichiers (permissions, propriétaire, taille, etc.)

4.2 Personnalisation des règles de surveillance

Il est possible d'être plus précis sur les paramètres à monitorer et de créer des règles personnalisées. Nous pouvons également spécifier individuellement les attributs à surveiller :

check_md5= « yes » → Vérification de l'empreinte MD5

check_sha1= « yes » → Vérification de l'empreinte SHA1

check_sha256= « yes » → Vérification de l'empreinte SHA256

check_size= « yes » → Surveillance de la taille des fichiers

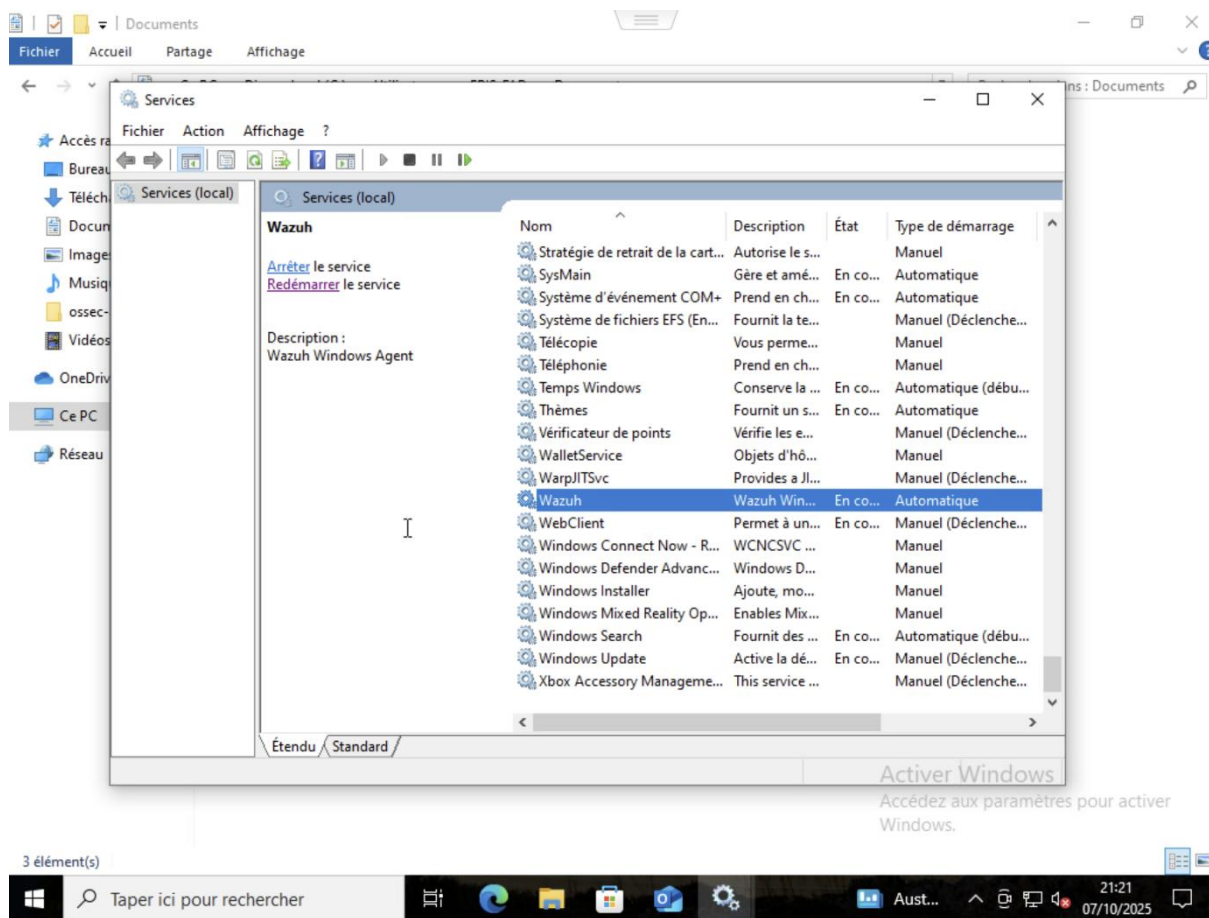
check_owner= « yes » → Surveillance du propriétaire

check_perm= « yes » → Surveillance des permissions

4.3 Redémarrage du service de l'agent Wazuh

Nous ouvrons le Gestionnaire de services via *Win+R*, nous tapons *services.msc* et nous redémarrons le service *Wazuh* dans la liste.

Screenshot 76 - Redémarrage du service Wazuh via le Gestionnaire de services Windows (*services.msc*)

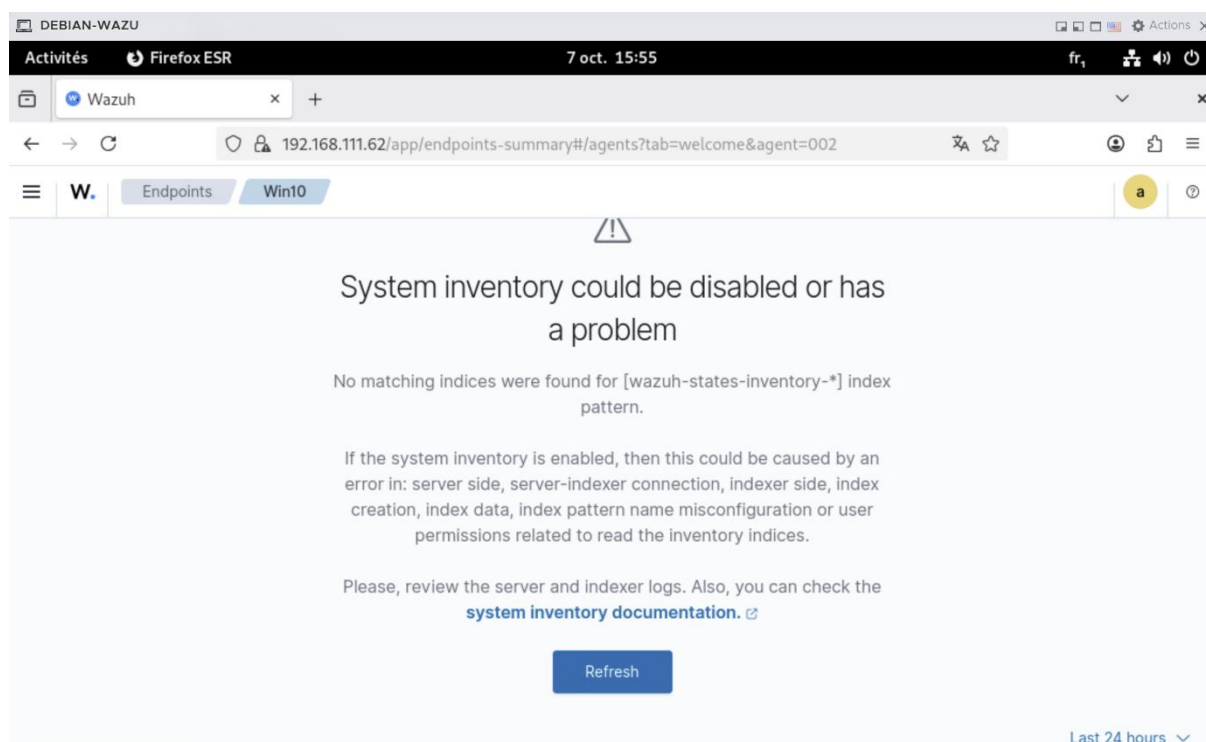


4.4 Résolution du problème de connexion à l'indexer

Si nous rencontrons une erreur de connexion entre le manager *Wazuh* et l'*indexer* (*OpenSearch*), cela se manifeste généralement par une impossibilité d'accéder au tableau de bord ou par des erreurs dans les logs du manager.

Cause → Cette erreur survient lorsque le *Wazuh Manager* tente de communiquer avec l'*Indexer* via une adresse IP incorrecte (0.0.0.0 au lieu de l'IP réelle du serveur).

Screenshot 77 - Erreur de connexion à l'indexer OpenSearch dans l'interface Wazuh



4.5 Solution

Étape 1 → Modifier la configuration du manager

Nous modifions le fichier de configuration du manager.

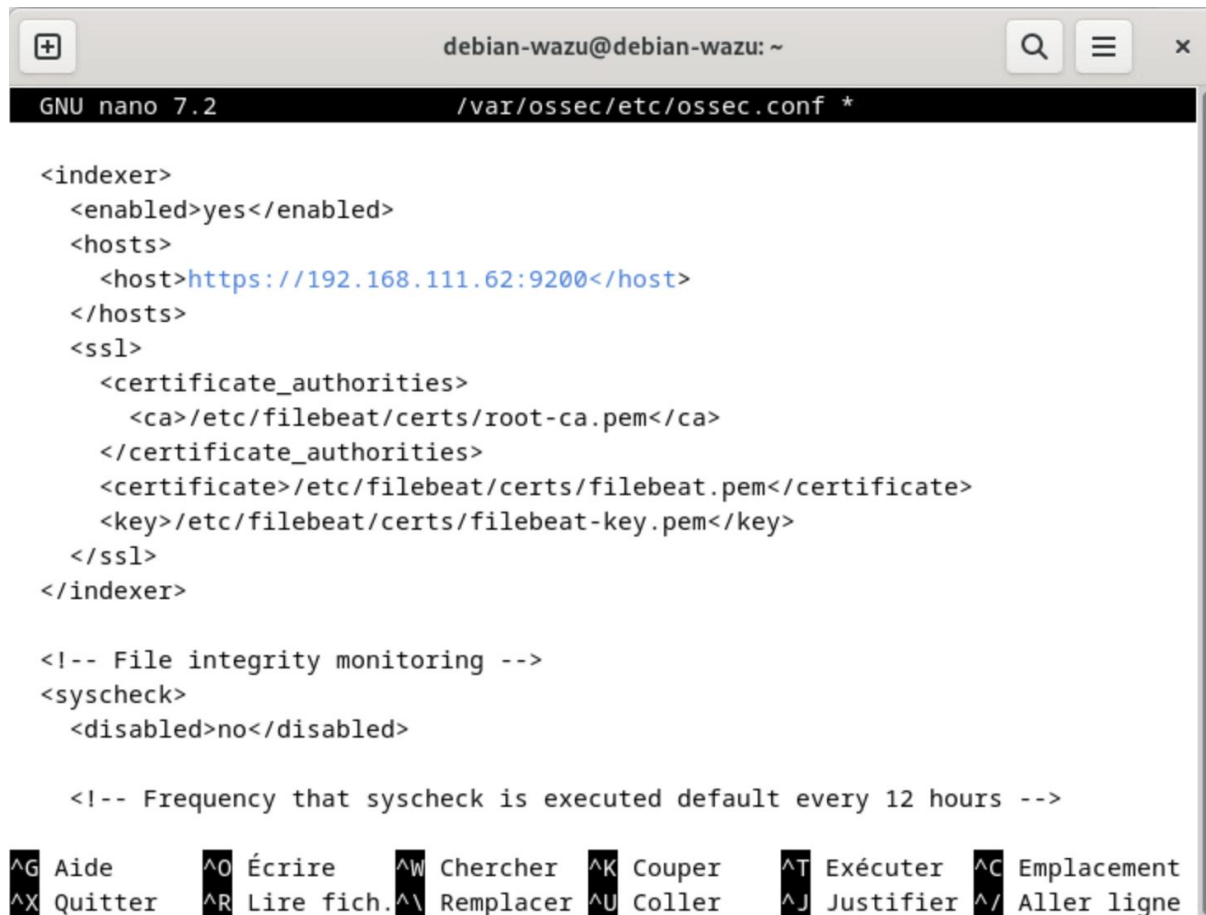
Screenshot 78 - Modification du fichier avec micro /var/ossec/etc/ossec.conf

```
root@debian-wazu: /home/debian-wazu# micro /var/ossec/etc/ossec.conf
```


Étape 2 → Corriger l'adresse IP

Dans le bloc `<indexer>`, nous remplaçons 0.0.0.0 par l'adresse IP de notre serveur Wazuh.

Screenshot 79 - Modification du bloc `<indexer>` dans `ossec.conf` pour corriger l'adresse IP du serveur



```

debian-wazu@debian-wazu: ~
GNU nano 7.2 /var/ossec/etc/ossec.conf *

<indexer>
  <enabled>yes</enabled>
  <hosts>
    <host>https://192.168.111.62:9200</host>
  </hosts>
  <ssl>
    <certificate_authorities>
      <ca>/etc/filebeat/certs/root-ca.pem</ca>
    </certificate_authorities>
    <certificate>/etc/filebeat/certs/filebeat.pem</certificate>
    <key>/etc/filebeat/certs/filebeat-key.pem</key>
  </ssl>
</indexer>

<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->

^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier ^/ Aller ligne
  
```

Étape 3 → Redémarrer le manager

Nous redémarrons le service Wazuh Manager :

Screenshot 80 - Redémarrage du service Wazuh Manager avec `systemctl restart wazuh-manager`

```
root@debian-wazu2: /var/ossec/etc# systemctl restart wazuh-agent
```

Étape 4 → Vérification

Nous vérifions que le service fonctionne correctement :

Screenshot 81 - Vérification du fonctionnement du Manager Wazuh avec `systemctl status wazuh-manager`

```
root@debian-wazu: /home/debian-wazu# systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-10-10 21:10:53 CEST; 14h ago
     Process: 65410 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 201 (limit: 9447)
   Memory: 1.1G
      CPU: 6min 53.128s
   CGroup: /system.slice/wazuh-manager.service
           └─65474 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─65475 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─65476 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─65479 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─65482 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─65524 /var/ossec/bin/wazuh-authd
                       └─65539 /var/ossec/bin/wazuh-db
                         └─65553 /var/ossec/bin/wazuh-execd
                           └─65587 /var/ossec/bin/wazuh-analysisd
                             └─65607 /var/ossec/bin/wazuh-syscheckd
                               └─65624 /var/ossec/bin/wazuh-remoted
                                 └─65660 /var/ossec/bin/wazuh-logcollector
                                   └─65679 /var/ossec/bin/wazuh-monitord
                                     └─65720 /var/ossec/bin/wazuh-modulesd

oct. 10 21:10:47 debian-wazu env[65410]: Started wazuh-syscheckd...
oct. 10 21:10:48 debian-wazu env[65410]: Started wazuh-remoted...
oct. 10 21:10:49 debian-wazu env[65410]: Started wazuh-logcollector...
oct. 10 21:10:50 debian-wazu env[65410]: Started wazuh-monitord...
oct. 10 21:10:50 debian-wazu env[65718]: 2025/10/10 21:10:50 wazuh-modulesd:router: INFO: Loaded router module.
oct. 10 21:10:50 debian-wazu env[65718]: 2025/10/10 21:10:50 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
oct. 10 21:10:50 debian-wazu env[65718]: 2025/10/10 21:10:50 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
oct. 10 21:10:51 debian-wazu env[65410]: Started wazuh-modulesd...
oct. 10 21:10:53 debian-wazu env[65410]: Completed.
oct. 10 21:10:53 debian-wazu systemd[1]: Started wazuh-manager.service - Wazuh manager.
root@debian-wazu: /home/debian-wazu#
```

4.6 Vérification du fonctionnement

Une fois la configuration appliquée, nous pouvons vérifier que la surveillance fonctionne en créant, modifiant ou supprimant un fichier dans le répertoire surveillé. Toutes les actions seront répertoriées dans le tableau de bord *Wazuh*.

Nous pouvons voir les détails de chaque événement en cliquant dessus :

- Nom du fichier modifié
- Type d'opération (création, modification, suppression)
- Date et heure de la modification
- Checksums (MD5, SHA1, SHA256)

Screenshot 82 - Liste des événements FIM détectés dans le tableau de bord Wazuh

Time ↓	Path	Action	Rule description	Rule Lev...	Rule Id
Oct 7, 2025 @ 16:13:50.963	c:\users\eris-fad\documents\test6wazuh.txt	added	File added to the system.	5	554
Oct 7, 2025 @ 16:13:50.878	c:\users\eris-fad\documents\nouveau doc...	deleted	File deleted.	7	553
Oct 7, 2025 @ 16:13:45.387	c:\users\eris-fad\documents\nouveau doc...	added	File added to the system.	5	554

Screenshot 83 - Détails d'un événement FIM montrant le type d'opération, les checksums et les métadonnées du fichier

192.168.111.62/app/endpoints-summary#/agents?tab=welcome&agent=002&a={query:(language:kuery,query:"")}

W. Endpoints Win10

Vulnerability Detection

0 Critical

0 High

0 Medium

0 Low

Top 5 Packages

c:\users\eris-fad\documents\test6wazuh.txt

Details

Last analysis: Oct 7, 2025 @ 16:13:50.000

Last modified: Oct 7, 2025 @ 16:13:45.000

User: ERIS-FAD

User ID: S-1-5-21-625155561-1667484677-447342189-1001

Size: 0

MD5: d41d8cd98f00b204e9800998ecf8427e

SHA1: da39a3ee5e6b4b0d3255bfef95601890afd80709

SHA256: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934c495991b7852b855

Permissions

Recent events

Search DQL Last 24 hours Show dates Refresh

1 hit

Oct 6, 2025 @ 21:34:17.630 - Oct 7, 2025 @ 21:34:17.630

Time	Action	Description	Level	Rule ID
Oct 7, 2025 @ 16:13:50.963	added	File added to the system.	5	554

5. Configuration FIM de base pour Linux

5.1 Modification du fichier de configuration

Étape 1 → Accéder au fichier de configuration

Nous accédons au fichier de configuration avec les droits *root* en utilisant la commande suivante :

Screenshot 84 - Modification du fichier de configuration avec /var/ossec/etc/ossec.conf

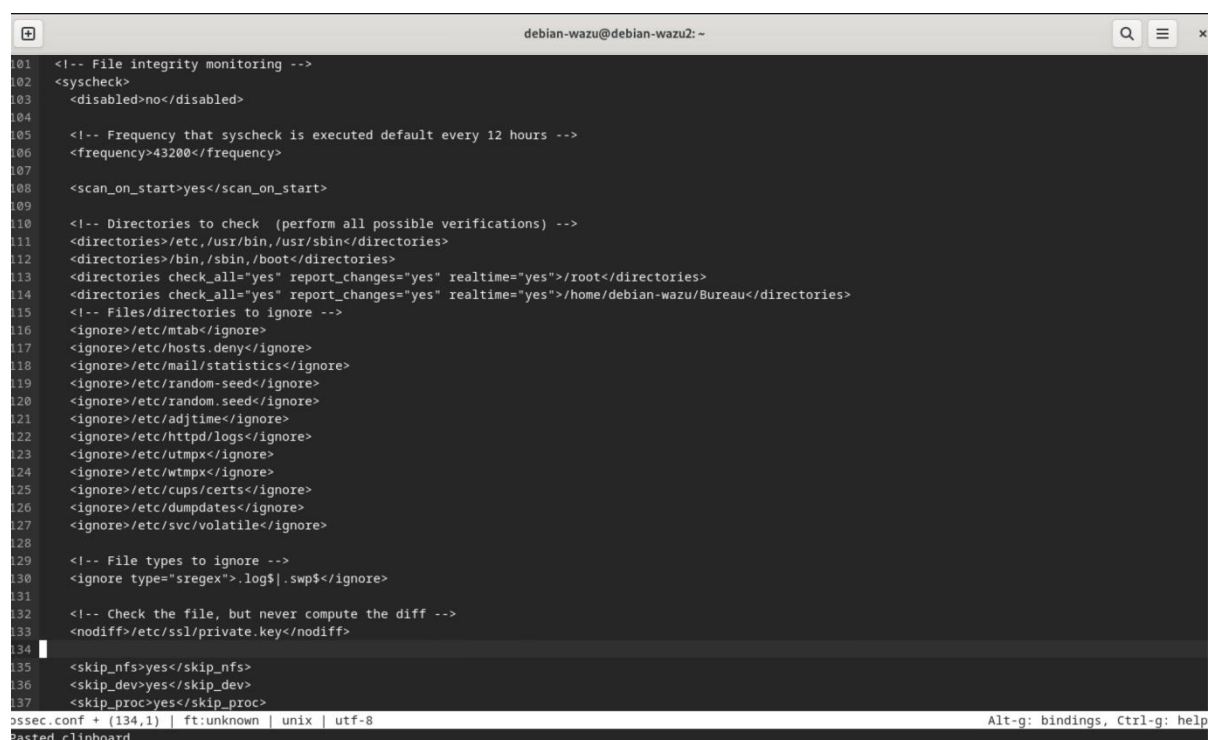
```
root@debian-wazu2:~# micro /root/test-wazuh-debian12.txt
```

Étape 2 → Ajouter la directive de surveillance

Nous ajoutons la ligne suivante à l'intérieur du bloc <syscheck> et nous remplaçons /chemin/du/repertoire par le chemin du répertoire que nous souhaitons monitorer :

```
<directories realtime="yes" report_changes="yes"
check_all="yes">/chemin/du/repertoire</directories>
```

Screenshot 85 - Modification de ossec.conf pour ajouter des éléments à surveiller



```

001 <!-- File integrity monitoring -->
002 <syscheck>
003   <disabled>no</disabled>
004
005   <!-- Frequency that syscheck is executed default every 12 hours -->
006   <frequency>43200</frequency>
007
008   <scan_on_start>yes</scan_on_start>
009
010   <!-- Directories to check (perform all possible verifications) -->
011   <directories>/etc,/usr/bin,/usr/sbin</directories>
012   <directories>/bin,/sbin,/boot</directories>
013   <directories check_all="yes" report_changes="yes" realtime="yes">/root</directories>
014   <directories check_all="yes" report_changes="yes" realtime="yes">/home/debian-wazu/Bureau</directories>
015   <!-- Files/directories to ignore -->
016   <ignore>/etc/mtab</ignore>
017   <ignore>/etc/hosts.deny</ignore>
018   <ignore>/etc/mail/statistics</ignore>
019   <ignore>/etc/random-seed</ignore>
020   <ignore>/etc/random-seed</ignore>
021   <ignore>/etc/adjtime</ignore>
022   <ignore>/etc/httpd/logs</ignore>
023   <ignore>/etc/utmpx</ignore>
024   <ignore>/etc/wtmpx</ignore>
025   <ignore>/etc/cups/certs</ignore>
026   <ignore>/etc/dumpdates</ignore>
027   <ignore>/etc/svc/volatile</ignore>
028
029   <!-- File types to ignore -->
030   <ignore type="sregex">.log$|.swp$</ignore>
031
032   <!-- Check the file, but never compute the diff -->
033   <nodiff>/etc/ssl/private.key</nodiff>
034
035   <skip_nfs>yes</skip_nfs>
036   <skip_dev>yes</skip_dev>
037   <skip_proc>yes</skip_proc>

```

Exemples de répertoires critiques à surveiller sous Linux :

- /etc → Fichiers de configuration système
- /bin et /sbin → Binaires système
- /usr/bin → Applications utilisateur
- /home → Répertoires utilisateurs
- /var/www → Sites web (si serveur web)

5.2 Redémarrage de l'agent

Étape 3 → Redémarrer l'agent Wazuh

Nous redémarrons l'agent Wazuh pour appliquer les modifications :

Screenshot 86 - Redémarrage de l'agent Wazuh et vérification de son fonctionnement avec `systemctl restart wazuh-agent` ; `systemctl restart wazuh-agent`

```
root@debian-wazu2:/var/ossec/etc# systemctl restart wazuh-agent
root@debian-wazu2:/var/ossec/etc# systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-08 19:58:28 CEST; 19s ago
     Process: 51758 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 33 (limit: 4616)
   Memory: 74.6M
      CPU: 12.877s
   CGroup: /system.slice/wazuh-agent.service
           └─51781 /var/ossec/bin/wazuh-execd
             └─51792 /var/ossec/bin/wazuh-agentd
               └─51806 /var/ossec/bin/wazuh-syscheckd
                 └─51819 /var/ossec/bin/wazuh-logcollector
                   └─51838 /var/ossec/bin/wazuh-modulesd

oct. 08 19:58:21 debian-wazu2 systemd[1]: Starting wazuh-agent.service - Wazuh agent...
oct. 08 19:58:21 debian-wazu2 env[51758]: Starting Wazuh v4.13.1...
oct. 08 19:58:22 debian-wazu2 env[51758]: Started wazuh-execd...
oct. 08 19:58:23 debian-wazu2 env[51758]: Started wazuh-agentd...
oct. 08 19:58:24 debian-wazu2 env[51758]: Started wazuh-syscheckd...
oct. 08 19:58:25 debian-wazu2 env[51758]: Started wazuh-logcollector...
oct. 08 19:58:26 debian-wazu2 env[51758]: Started wazuh-modulesd...
oct. 08 19:58:28 debian-wazu2 env[51758]: Completed.
oct. 08 19:58:28 debian-wazu2 systemd[1]: Started wazuh-agent.service - Wazuh agent.
root@debian-wazu2:/var/ossec/etc#
```

5.3 Vérification du fonctionnement

Pour tester notre configuration, nous créons un fichier de test dans le répertoire surveillé :

Screenshot 87 - Création d'un fichier de test pour vérifier la surveillance FIM

```
root@debian-wazu2:~# micro /home/debian-wazu/Bureau/test-sur-utilisateur
```

Nous pouvons ensuite consulter le tableau de bord *Wazuh* pour voir que toutes les actions sont répertoriées. En cliquant sur un événement, nous visualisons les détails suivants :

- Chemin complet du fichier
- Type de modification
- Attributs modifiés
- Empreintes cryptographiques

Screenshot 88 - Événements FIM générés suite à la création du fichier de test

Time ↓	Path	Action	Rule description	Rule Lev...	Rule Id
Oct 8, 2025 @ 20:02:54.154	/root/.config/micro/buffers/history	modified	Integrity checksum changed.	7	550
Oct 8, 2025 @ 20:02:54.148	/root/.config/micro/backups/%home%debian-wazu%Bureau%test-...	deleted	File deleted.	7	553
Oct 8, 2025 @ 20:02:54.142	/home/debian-wazu/Bureau/test-sur-utilisateur	added	File added to the system.	5	554
Oct 8, 2025 @ 20:02:50.482	/root/.config/micro/backups/%home%debian-wazu%Bureau%test-...	added	File added to the system.	5	554
Oct 8, 2025 @ 20:02:42.506	/root/.config/micro/settings.json	modified	Integrity checksum changed.	7	550

Screenshot 89 - Détails complets de l'événement FIM incluant les empreintes cryptographiques (MD5, SHA1, SHA256)

/home/debian-wazu/Bureau/test-sur-utilisateur

User ID

0

Group

root

Group ID

0

Size

51 Bytes

Inode

3545371

MD5

028902062a0d68d29746570c0b179f68

SHA1

a475be837922f6d5363b54b72df7a4d4e5710a08

SHA256

bea0201cb3679b63dd0b900a5c2d135e89ba181378bb1daa8751ada447f044f7

Permissions

rw-r--r--

Recent events

Search

DQL

Last 24 hours

Show dates

Refresh

Add filter

1 hit

Oct 7, 2025 @ 20:04:46.242 - Oct 8, 2025 @ 20:04:46.242

Time	Action	Description	Level	Rule ID
Oct 8, 2025 @ 20:02:54.142	added	File added to the system.	5	554

Rows per page: 20

< 1 >

6. Configuration avancée → Mode Whodata

6.1 Qu'est-ce que Whodata ?

La surveillance en mode « whodata » est une amélioration de la surveillance « realtime ». Elle inclut non seulement la surveillance en temps réel, mais aussi des informations supplémentaires sur :

- L'identité de l'utilisateur ayant effectué la modification
- Le programme utilisé pour la modification

- Les processus impliqués dans l'opération
- Les arguments de la ligne de commande

Ce mode est particulièrement utile pour les audits de sécurité et les investigations d'incidents, car il permet de tracer précisément qui a fait quoi et comment.

Attention → Le mode *whodata* consomme plus de ressources système que les modes *basic* ou *realtime*. Nous le réservons aux répertoires les plus critiques.

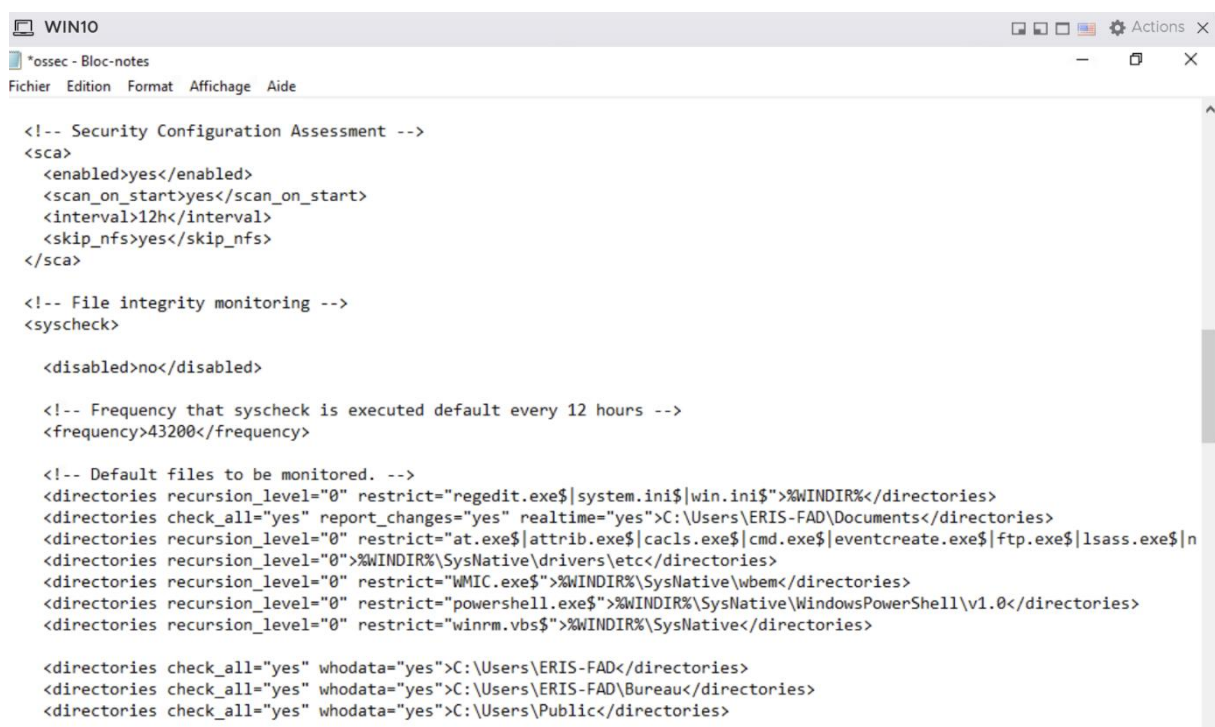
7. Configuration Whodata pour Windows

7.1 Modification de la configuration

Étape 1 → Éditer le fichier *ossec.conf*

Nous modifions le fichier *ossec.conf* dans le répertoire *C:\Program Files (x86)\ossec-agent* en remplaçant *C:\Dossier\A\Surveiller* par le chemin du répertoire que nous souhaitons monitorer :

Screenshot 90 - Modification du fichier *ossec.conf* avec l'ajout du répertoire à monitorer



```

<!-- Security Configuration Assessment -->
<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <!-- Default files to be monitored. -->
  <directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$" %WINDIR%</directories>
  <directories check_all="yes" report_changes="yes" realtime="yes">C:\Users\ERIS-FAD\Documents</directories>
  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.exe$|n
  <directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe$" %WINDIR%\SysNative\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe$" %WINDIR%\SysNative\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs$" %WINDIR%\SysNative</directories>

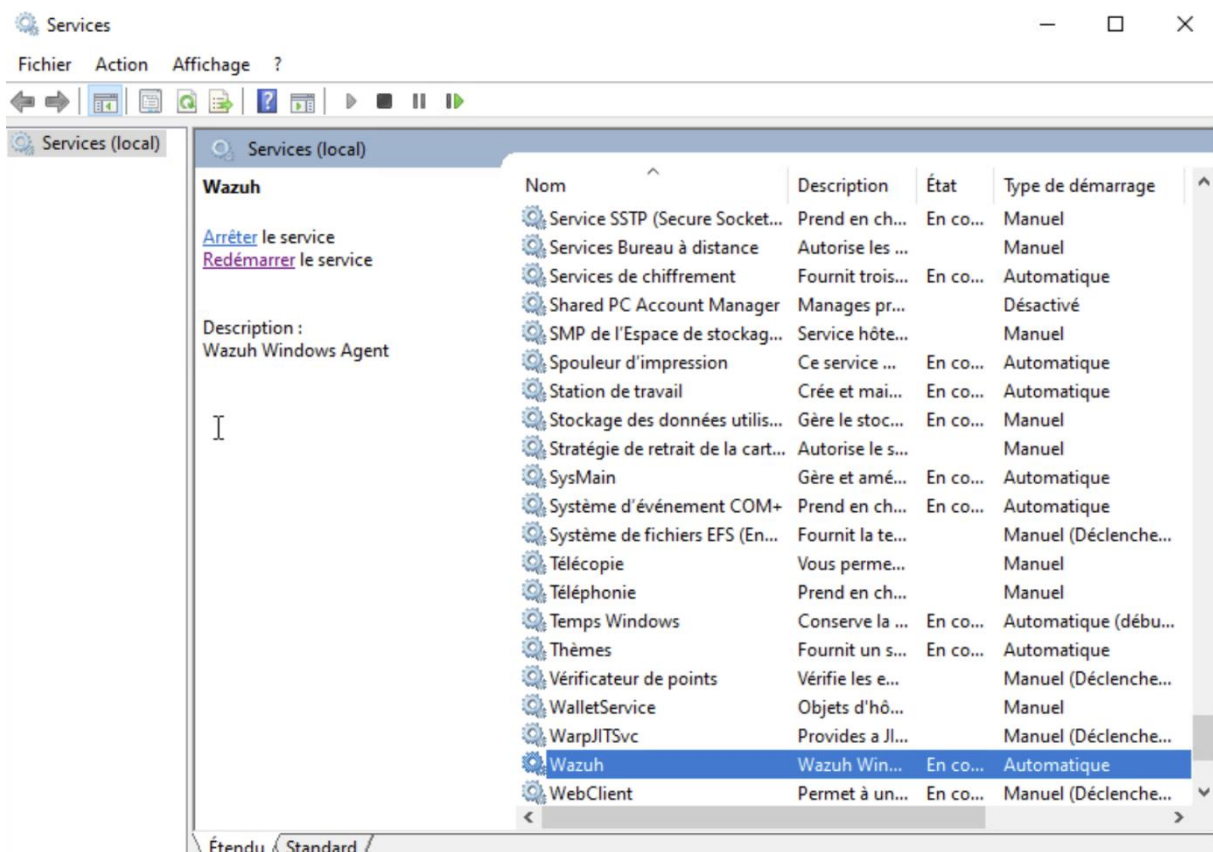
  <directories check_all="yes" whodata="yes">C:\Users\ERIS-FAD</directories>
  <directories check_all="yes" whodata="yes">C:\Users\ERIS-FAD\Bureau</directories>
  <directories check_all="yes" whodata="yes">C:\Users\Public</directories>

```

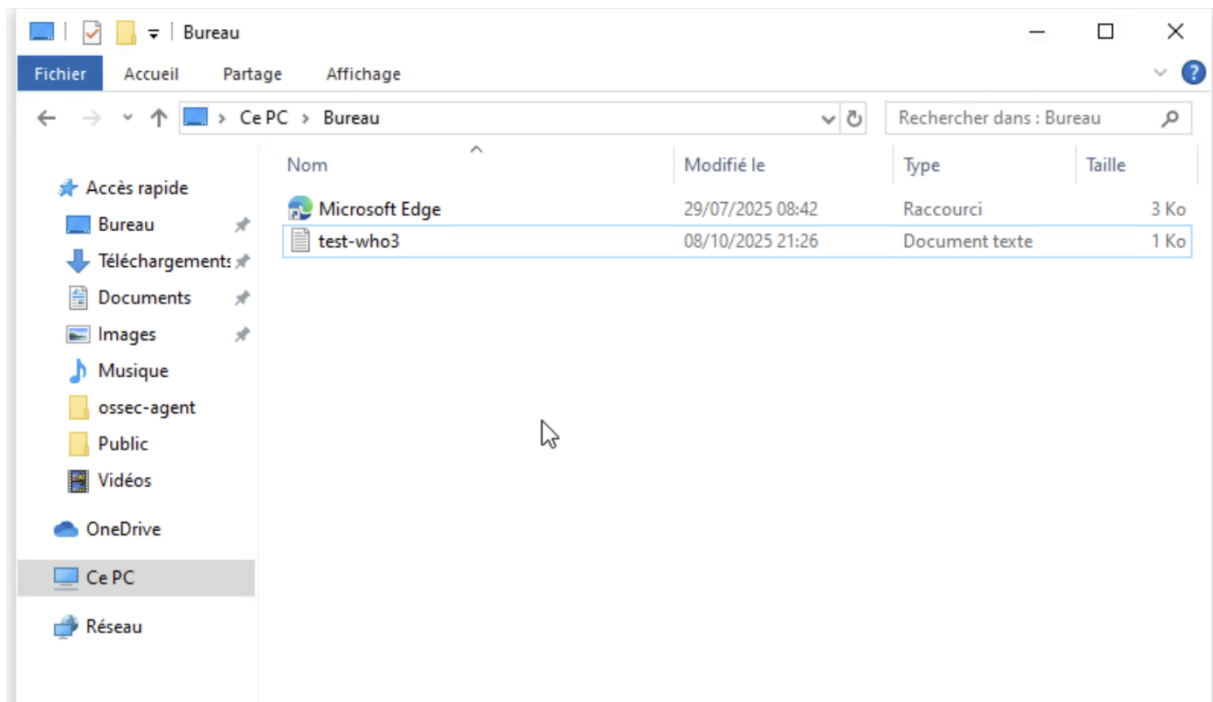
Étape 2 → Redémarrer le service

Nous ouvrons le Gestionnaire de services (*Win+R* → *services.msc*) et nous redémarrons le service Wazuh.

Screenshot 91 - Redémarrage du service Wazuh



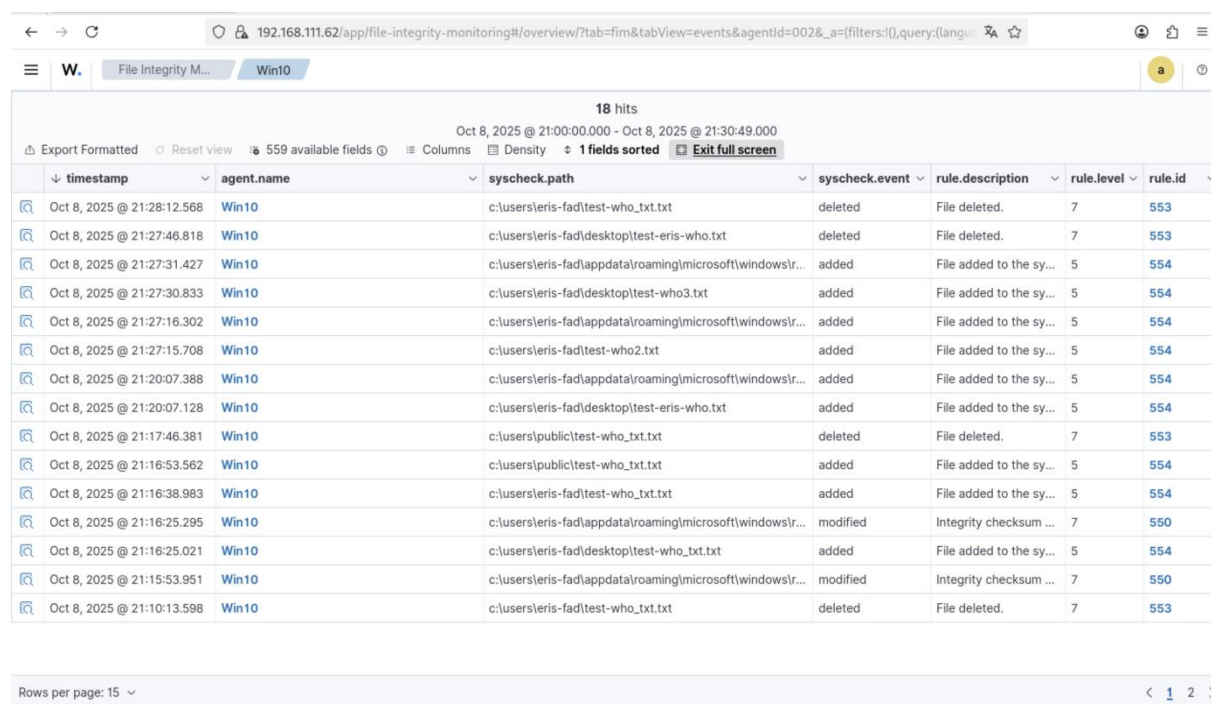
Screenshot 92 - Ajout d'un fichier texte de test



7.2 Analyse des événements Whodata sous Windows

Une fois la configuration appliquée et un fichier modifié dans le répertoire surveillé, nous pouvons visualiser dans le tableau de bord Wazuh des informations détaillées :

Screenshot 93 - Liste des événements Whodata dans le tableau de bord Wazuh



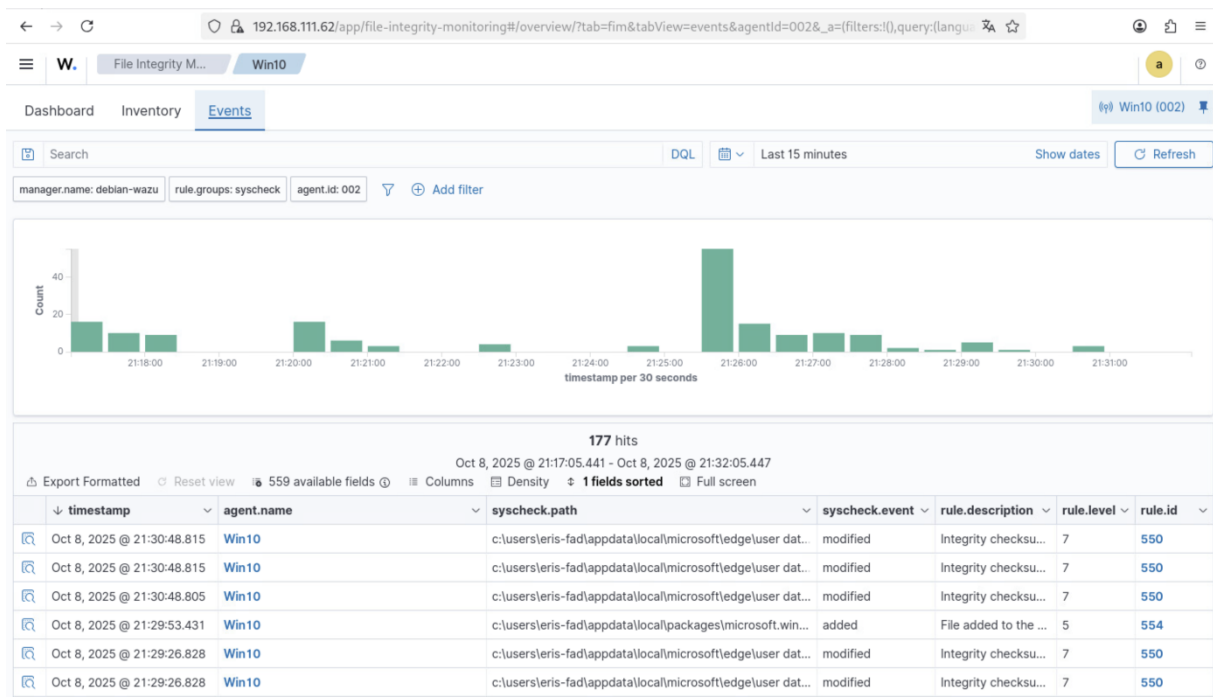
timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Oct 8, 2025 @ 21:28:12.568	Win10	c:\users\eris-fad\test-who.txt.txt	deleted	File deleted.	7	553
Oct 8, 2025 @ 21:27:46.818	Win10	c:\users\eris-fad\desktop\test-eris-who.txt	deleted	File deleted.	7	553
Oct 8, 2025 @ 21:27:31.427	Win10	c:\users\eris-fad\appdata\roaming\microsoft\windowsr...	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:27:30.833	Win10	c:\users\eris-fad\desktop\test-who3.txt	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:27:16.302	Win10	c:\users\eris-fad\appdata\roaming\microsoft\windowsr...	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:27:15.708	Win10	c:\users\eris-fad\test-who2.txt	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:20:07.388	Win10	c:\users\eris-fad\appdata\roaming\microsoft\windowsr...	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:20:07.128	Win10	c:\users\eris-fad\desktop\test-eris-who.txt	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:17:46.381	Win10	c:\users\public\test-who.txt.txt	deleted	File deleted.	7	553
Oct 8, 2025 @ 21:16:53.562	Win10	c:\users\public\test-who.txt.txt	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:16:38.983	Win10	c:\users\eris-fad\test-who.txt.txt	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:16:25.295	Win10	c:\users\eris-fad\appdata\roaming\microsoft\windowsr...	modified	Integrity checksum ...	7	550
Oct 8, 2025 @ 21:16:25.021	Win10	c:\users\eris-fad\desktop\test-who.txt.txt	added	File added to the sy...	5	554
Oct 8, 2025 @ 21:15:53.951	Win10	c:\users\eris-fad\appdata\roaming\microsoft\windowsr...	modified	Integrity checksum ...	7	550
Oct 8, 2025 @ 21:10:13.598	Win10	c:\users\eris-fad\test-who.txt.txt	deleted	File deleted.	7	553

Informations supplémentaires disponibles :

- Nom d'utilisateur Windows (SID et nom)
- Nom du processus ayant effectué la modification
- ID du processus (PID)
- Chemin complet de l'exécutable
- Type d'accès (lecture, écriture, suppression)
- Résultat de l'opération (succès/échec)

Ces informations permettent de reconstituer précisément le contexte d'une modification de fichier, ce qui est crucial lors d'une investigation de sécurité.

Screenshot 94 - Informations détaillées d'un événement Whodata incluant le SID et le nom d'utilisateur Windows



Screenshot 95 - Détails étendus de l'événement Whodata montrant le processus, le PID, le chemin de l'exécutable et le type d'accès (1/2)

Document Details
View surrounding documents
View single document

Table
JSON

t _index	wazuh-alerts-4.x-2025.10.08
t agent.id	002
t agent.ip	192.168.111.11
t agent.name	Win10
t decoder.name	syscheck_new_entry
t full_log	File 'c:\users\eris-fad\desktop\test-who3.txt' added Mode: whodata
t id	1759951650.5196800
t input.type	log
t location	syscheck
t manager.name	debian-wazu
t rule.description	File added to the system.
# rule.firedtimes	118
t rule.gdpr	II_5.1.f
t rule.gpg13	4.11
t rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
t rule.hipaa	164.312.c.1, 164.312.c.2
t rule.id	554
# rule.level	5
rule.mail	false

Screenshot 96 - Détails étendus de l'événement Whodata montrant le processus, le PID, le chemin de l'exécutable et le type d'accès (2/2)

Document Details		View surrounding documents	View single document	✕
rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file			
rule.hipaa	164.312.c.1, 164.312.c.2			
rule.id	554			
rule.level	5			
rule.mail	false			
rule.nist_800_53	SI.7			
rule.pci_dss	11.5			
rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3			
syscheck.attrs_after	ARCHIVE			
syscheck.audit.process.id	1876			
syscheck.audit.process.name	C:\Windows\System32\notepad.exe			
syscheck.audit.user.id	S-1-5-21-625155561-1667484677-447342189-1001			
syscheck.audit.user.name	ERIS-FAD			
syscheck.event	added			
syscheck.md5_after	5e5f65b8742a19315365f29ebb9a5222			
syscheck.mode	whodata			
syscheck.mtime_after	Oct 8, 2025 @ 23:26:20.000			
syscheck.path	c:\users\eris-fad\desktop\test-who3.txt			
syscheck.sha1_after	03998241d4951126e3fff7d006dc7d61e3a1717c			
syscheck.sha256_after	c814a6b0ce320071aa188a48d9be566c2b1adbf72ef7b05f248b8543de23401			
syscheck.size_after	8			

Ces informations permettent de reconstituer précisément le contexte d'une modification de fichier, ce qui est crucial lors d'une investigation de sécurité.

8. Configuration Whodata pour Linux

Le mode *whodata* sous Linux s'appuie sur le système d'audit du noyau Linux (*auditd*). Nous devons donc installer les composants nécessaires.

8.1 Prérequis → Installation des outils d'audit

Le mode *whodata* sous Linux s'appuie sur le système d'audit du noyau Linux (*auditd*). Nous devons donc installer les composants nécessaires.

Étape 1 → Installation d'auditd

Nous installons le service d'audit avec la commande suivante :

Screenshot 97 - Installation du service auditd pour la surveillance Whodata avec apt-get install auditd -y

```
root@debian-wazu2:~# apt-get install auditd -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  libauparse0
Paquets suggérés :
  audispd-plugins
Les NOUVEAUX paquets suivants seront installés :
  auditd libauparse0
0 mis à jour, 2 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 279 ko dans les archives.
Après cette opération, 934 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 libauparse0 amd64 1:3.0.9-1 [61,9 kB]
Réception de :2 http://deb.debian.org/debian bookworm/main amd64 auditd amd64 1:3.0.9-1 [218 kB]
279 ko réceptionnés en 0s (11,1 Mo/s)
Sélection du paquet libauparse0:amd64 précédemment désélectionné.
(Lecture de la base de données... 155511 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../libauparse0_1%3a3.0.9-1_amd64.deb ...
Dépaquetage de libauparse0:amd64 (1:3.0.9-1) ...
Sélection du paquet auditd précédemment désélectionné.
Préparation du dépaquetage de .../auditd_1%3a3.0.9-1_amd64.deb ...
Dépaquetage de auditd (1:3.0.9-1) ...
Paramétrage de libauparse0:amd64 (1:3.0.9-1) ...
Paramétrage de auditd (1:3.0.9-1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/auditd.service → /lib/systemd/system/auditd.service.
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.36-9+deb12u13) ...
root@debian-wazu2:~#
```

Étape 2 → Installation du plugin audispd

Nous installons le plugin *audispd-plugins* qui permet à *Wazuh* de communiquer avec *auditd* :

Screenshot 98 - Installation du plugin audispd-plugins avec apt-get install audispd-plugins

```
root@debian-wazu2:~# apt-get install audispd-plugins
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  audispd-plugins
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 45,6 ko dans les archives.
Après cette opération, 132 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 audispd-plugins amd64 1:3.0.9-1 [45,6 kB]
45,6 ko réceptionnés en 0s (2 783 ko/s)
Sélection du paquet audispd-plugins précédemment désélectionné.
(Lecture de la base de données... 155602 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../audispd-plugins_1%3a3.0.9-1_amd64.deb ...
Dépaquetage de audispd-plugins (1:3.0.9-1) ...
Paramétrage de audispd-plugins (1:3.0.9-1) ...
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
root@debian-wazu2:~#
```

Étape 3 → Redémarrage du service d'audit

Nous redémarrons le service *auditd* pour appliquer les changements :

Screenshot 99 - Redémarrage du service d'audit avec systemctl restart auditd

```
root@debian-wazu2:~# systemctl restart auditd
```

8.2 Configuration de Whodata

Étape 4 → Modification de la configuration Wazuh

Nous modifions le fichier `/var/ossec/etc/ossec.conf` :

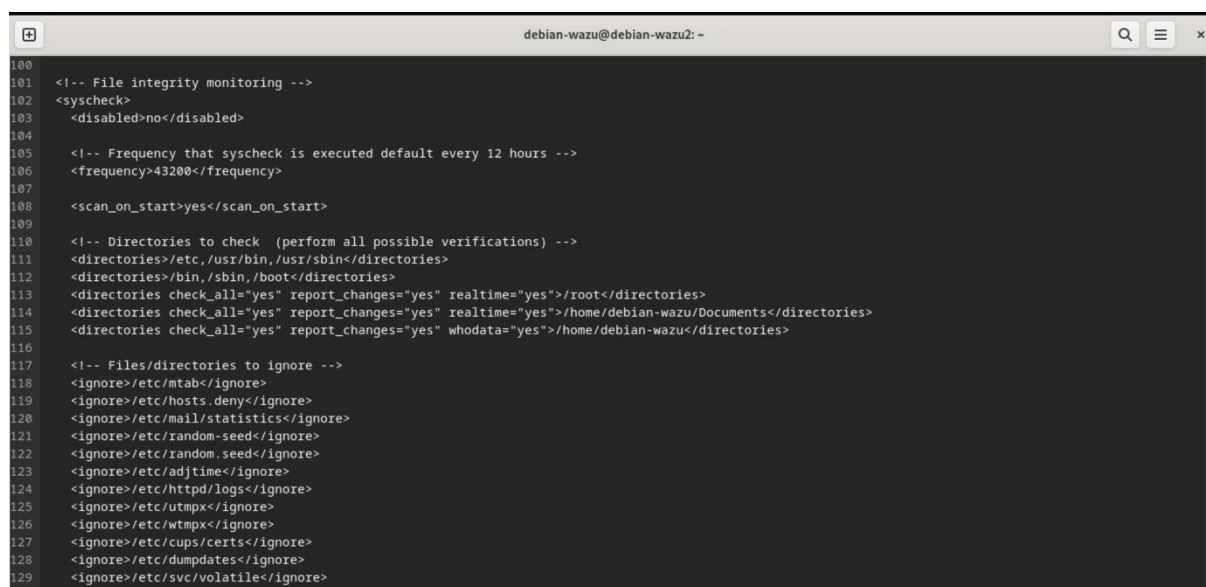
Nous ajoutons la ligne suivante à l'intérieur du bloc `<syscheck>` en remplaçant `/chemin/du/repertoire` par le chemin du répertoire que nous souhaitons monitorer :

```
<directories whodata="yes">/root</directories>
```

```
<directories whodata="yes">/home/debian-wazu/Documents</directories>
```

```
<directories whodata="yes">/home/debian-wazu </directories>
```

Screenshot 100 - Configuration Whodata dans le fichier `ossec.conf` sous Linux



```

100
101 <!-- File integrity monitoring -->
102 <syscheck>
103   <disabled>no</disabled>
104
105   <!-- Frequency that syscheck is executed default every 12 hours -->
106   <frequency>43200</frequency>
107
108   <scan_on_start>yes</scan_on_start>
109
110   <!-- Directories to check (perform all possible verifications) -->
111   <directories>/etc,/usr/bin,/usr/sbin</directories>
112   <directories>/bin,/sbin,/boot</directories>
113   <directories check_all="yes" report_changes="yes" realtime="yes">/root</directories>
114   <directories check_all="yes" report_changes="yes" realtime="yes">/home/debian-wazu/Documents</directories>
115   <directories check_all="yes" report_changes="yes" whodata="yes">/home/debian-wazu</directories>
116
117   <!-- Files/directories to ignore -->
118   <ignore>/etc/mtab</ignore>
119   <ignore>/etc/hosts.deny</ignore>
120   <ignore>/etc/mail/statistics</ignore>
121   <ignore>/etc/random-seed</ignore>
122   <ignore>/etc/random.seed</ignore>
123   <ignore>/etc/adjtime</ignore>
124   <ignore>/etc/httpd/logs</ignore>
125   <ignore>/etc/utmpx</ignore>
126   <ignore>/etc/wtmpx</ignore>
127   <ignore>/etc/cups/certs</ignore>
128   <ignore>/etc/dumpdates</ignore>
129   <ignore>/etc/svc/volatile</ignore>
130

```

Étape 5 → Redémarrage de l'agent Wazuh

Nous redémarrons l'agent Wazuh :

Screenshot 101 - Redémarrage et vérification du fonctionnement de l'agent Wazuh avec `systemctl restart wazuh-agent` ; `systemctl status wazuh-agent`

```
root@debian-wazu2:/home/debian-wazu# systemctl restart wazuh-agent
root@debian-wazu2:/home/debian-wazu# systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-08 21:58:13 CEST; 12s ago
     Process: 57455 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 32 (limit: 4616)
   Memory: 137.0M
      CPU: 19.540s
   CGroup: /system.slice/wazuh-agent.service
           └─57478 /var/ossec/bin/wazuh-execd
             └─57489 /var/ossec/bin/wazuh-agentd
               └─57503 /var/ossec/bin/wazuh-syscheckd
                 └─57545 /var/ossec/bin/wazuh-logcollector
                   └─57562 /var/ossec/bin/wazuh-modulesd

oct. 08 21:58:06 debian-wazu2 systemd[1]: Starting wazuh-agent.service - Wazuh agent...
oct. 08 21:58:06 debian-wazu2 env[57455]: Starting Wazuh v4.13.1...
oct. 08 21:58:07 debian-wazu2 env[57455]: Started wazuh-execd...
oct. 08 21:58:08 debian-wazu2 env[57455]: Started wazuh-agentd...
oct. 08 21:58:09 debian-wazu2 env[57455]: Started wazuh-syscheckd...
oct. 08 21:58:10 debian-wazu2 env[57455]: Started wazuh-logcollector...
oct. 08 21:58:11 debian-wazu2 env[57455]: Started wazuh-modulesd...
oct. 08 21:58:13 debian-wazu2 env[57455]: Completed.
oct. 08 21:58:13 debian-wazu2 systemd[1]: Started wazuh-agent.service - Wazuh agent.
root@debian-wazu2:/home/debian-wazu#
```

8.3 Vérification du fonctionnement

Pour tester la configuration, nous créons un fichier dans le répertoire surveillé :

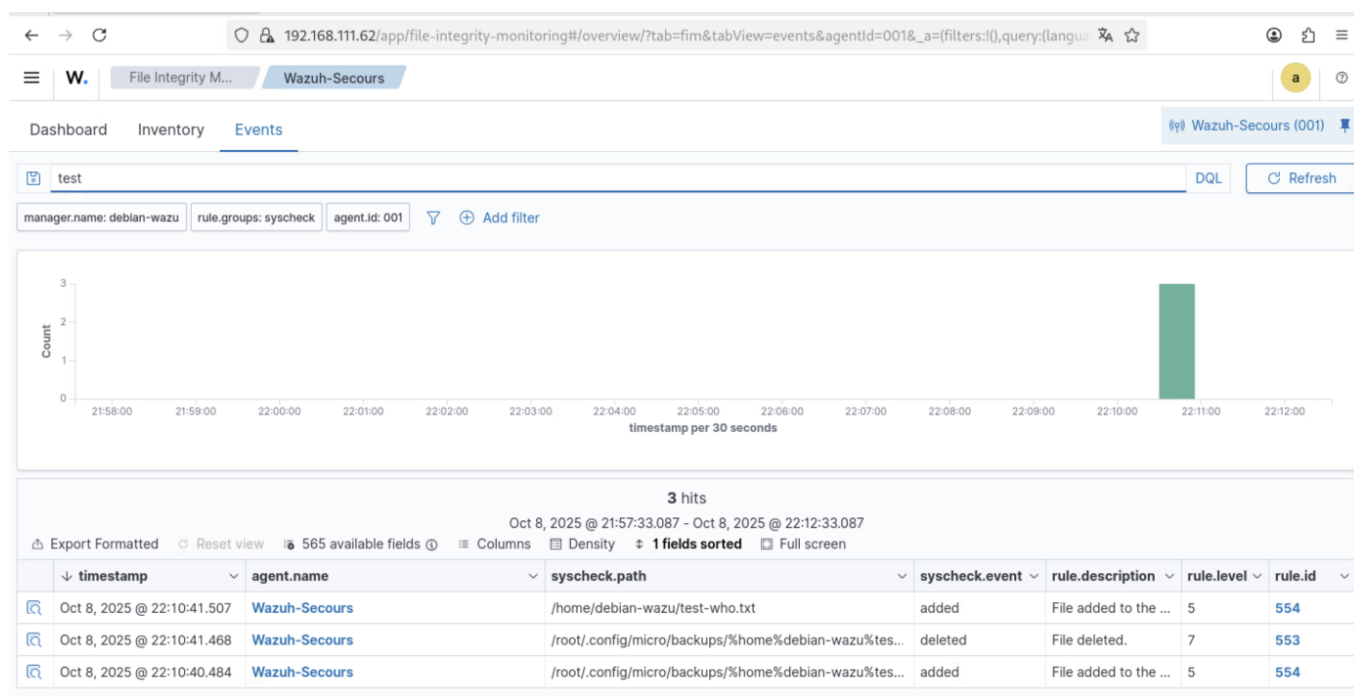
Screenshot 102 - Création d'un fichier "test" avec `micro text-who.txt`

```
root@debian-wazu2:/home/debian-wazu# micro text-who.txt
```

8.4 Analyse des événements Whodata

Nous visualisons maintenant beaucoup plus de détails dans le tableau de bord Wazuh grâce aux règles personnalisées et au système d'audit :

Screenshot 103 - Liste des événements Whodata détectés dans le tableau de bord



Informations détaillées disponibles :

- Nom d'utilisateur Linux (UID et nom)
- Nom du processus effectuant la modification
- ID du processus (PID)
- ID du processus parent (PPID)
- Chemin complet de la commande exécutée
- Arguments de la ligne de commande
- Groupe de l'utilisateur (GID)
- Contexte SELinux (si applicable)
- Résultat de l'appel système

Ces informations permettent une traçabilité complète et sont essentielles pour :

- Identifier précisément l'origine d'une modification
- Détecter des comportements suspects
- Répondre aux exigences d'audit et de conformité (PCI-DSS, HIPAA, etc.)

Screenshot 104 - Détails complets d'un événement Whodata sous Linux incluant UID, PID, PPID et GID

Document Details

View surrounding documents

View single document

Table JSON

t _index	wazuh-alerts-4.x-2025.10.08
t agent.id	001
t agent.ip	192.168.111.63
t agent.name	Wazuh-Secours
t decoder.name	syscheck_new_entry
t full_log	File '/home/debian-wazu/test-who.txt' added Mode: whodata
t id	1759954241.5730819
t input.type	log
t location	syscheck
t manager.name	debian-wazu
t rule.description	File added to the system.
# rule.firedtimes	5
t rule.gdpr	II_5.1.f
t rule.gpg13	4.11
t rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
t rule.hipaa	164.312.c.1, 164.312.c.2
t rule.id	554
# rule.level	5
rule.mail	false

Screenshot 105 - Informations étendues montrant la commande exécutée, les arguments et le contexte d'exécution

Document Details		View surrounding documents	View single document	×
† rule.pci_dss	11.5			
† rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3			
† syscheck.audit.effective_use r.id	0			
† syscheck.audit.effective_use r.name	root			
† syscheck.audit.group.id	0			
† syscheck.audit.group.name	root			
† syscheck.audit.login_user.id	1000			
† syscheck.audit.login_user.na me	debian-wazu			
† syscheck.audit.process.cwd	/home/debian-wazu			
† syscheck.audit.process.id	61457			
† syscheck.audit.process.name	/usr/bin/micro			
† syscheck.audit.process.paren t_cwd	/home/debian-wazu			
† syscheck.audit.process.paren t_name	/usr/bin/bash			
† syscheck.audit.process.ppid	51572			
† syscheck.audit.user.id	0			
† syscheck.audit.user.name	root			
† syscheck.event	added			
† syscheck.gid_after	0			
† syscheck.gname_after	root			

Ces informations permettent une traçabilité complète et sont essentielles pour :

- Identifier précisément l'origine d'une modification
- Détecter des comportements suspects
- Répondre aux exigences d'audit et de conformité (PCI-DSS, HIPAA, etc.)

Partie 3 :

Détection d'attaque par force brute et Active Response

Introduction et objectifs

Wazuh est capable d'identifier des attaques par force brute en analysant les tentatives d'authentification multiples échouées. Dans cette partie, nous allons simuler une attaque SSH par force brute à l'aide de l'outil Hydra, puis configurer une réponse active (*Active Response*) pour bloquer automatiquement l'adresse IP de l'attaquant.

À l'issue de ce tutoriel, nous serons capables de :

Note importante → Nous simulerons une attaque par force brute **échouée** (sans mot de passe valide) pour observer les alertes de détection de *Wazuh* et le mécanisme d'*Active Response*, sans compromettre réellement le compte utilisateur.

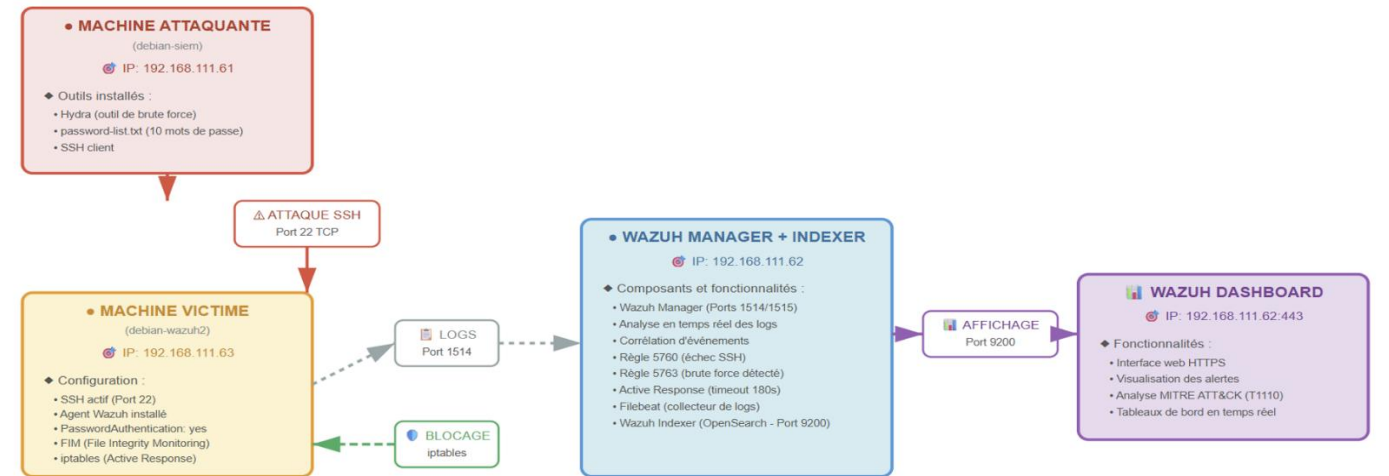
- Configurer SSH pour accepter les authentifications par mot de passe
- Installer et utiliser Hydra pour simuler une attaque par force brute
- Configurer l'*Active Response* dans *Wazuh* pour bloquer automatiquement les IP malveillantes
- Analyser les alertes générées par *Wazuh* (Rules 5760, 5763, 651, 652)
- Vérifier le blocage et le déblocage automatique des IP
- Comprendre le mécanisme de détection d'intrusion

IMPORTANT → Cette démonstration doit être réalisée uniquement dans un environnement de test contrôlé. L'utilisation d'outils comme Hydra sur des systèmes sans autorisation est **illégal**e et constitue une infraction pénale.

Architecture et prérequis

Notre laboratoire est composé de trois machines principales qui interagissent pour simuler et détecter une attaque par force brute :

ARCHITECTURE DU LABORATOIRE WAZUH - DÉTECTION BRUTE FORCE SSH



CHRONOLOGIE DES ÉVÉNEMENTS - SCÉNARIO D'ATTAQUE PAR FORCE BRUTE	
T+0s - ATTAQUE :	Hydra (192.168.111.61) lance 10 tentatives de connexion SSH vers debian-wazuh@192.168.111.63 (port 22)
T+10s - COLLECTE :	L'agent Wazuh collecte les échecs SSH dans /var/log/auth.log et les envoie au Manager (port 1514) → Règle 5760 déclenchée 8 fois
T+40s - RÉPONSE :	Le Manager détecte le pattern d'attaque (Règle 5763) et ordonne le blocage de 192.168.111.61 via iptables (durée: 180 secondes)
T+40s - VISUALISATION :	Les alertes sont indexées dans OpenSearch (port 9200) et affichées dans le Dashboard avec classification MITRE T1110 (Brute Force)

CHRONOLOGIE DES ÉVÉNEMENTS - SCÉNARIO D'ATTAQUE PAR FORCE BRUTE

- T+0s - ATTAQUE :** Hydra (192.168.111.61) lance 10 tentatives de connexion SSH vers debian-wazuh@192.168.111.63 (port 22)
- T+10s - COLLECTE :** L'agent Wazuh collecte les échecs SSH dans /var/log/auth.log et les envoie au Manager (port 1514) → Règle 5760 déclenchée 8 fois
- T+40s - RÉPONSE :** Le Manager détecte le pattern d'attaque (Règle 5763) et ordonne le blocage de 192.168.111.61 via iptables (durée: 180 secondes)
- T+40s - VISUALISATION :** Les alertes sont indexées dans OpenSearch (port 9200) et affichées dans le Dashboard avec classification MITRE T1110 (Brute Force)

Description des composants et de leurs relations

Élément	Rôle	Interactions principales
Machine Attaquante <i>(debian-siem)</i>	Simule une attaque brute-force SSH à l'aide de Hydra	Envoie des tentatives de connexion SSH vers la machine victime
Machine Victime <i>(debian-wazu2)</i>	Cible de l'attaque SSH, équipée de l'agent Wazuh pour la surveillance	Transmet les logs d'authentification et d'alerte au serveur Wazuh Manager
Serveur Wazuh Manager	Analyse les logs reçus, applique les règles de détection et déclenche des réponses automatiques	Génère des alertes et les envoie au Dashboard Wazuh
Dashboard Wazuh	Interface de visualisation et d'analyse des alertes et événements de sécurité	Affiche les résultats des détections et permet l'analyse des incidents

Flux d'information :

1. **Attaque SSH** → La machine attaquante (*debian-siem*) lance une attaque brute-force sur le port 22 de la machine victime (*debian-wazu2*)
2. **Collecte des logs** → L'agent *Wazuh* sur la machine victime (*debian-wazu2*) enregistre les tentatives de connexion et les envoie au *Wazuh Manager* (port 1514)
3. **Analyse et détection** → Le *Wazuh Manager* applique ses règles de corrélation pour identifier les comportements suspects
4. **Réponse active** → Si configurée, une action automatique (blocage IP via iptables) est déclenchée sur la machine victime (*debian-wazu2*)
5. **Visualisation** → Le Dashboard *Wazuh* affiche les alertes et permet l'analyse des événements de sécurité

Tableau des prérequis techniques

Machine	SSH Serveur	SSH Client	Agent Wazuh	Hydra	Rôle
VICTIME (<i>debian-wazu2</i>)	OUI obligatoire	Oui (par défaut)	OUI obligatoire	Non	Reçoit l'attaque SSH
ATTAQUANTE (<i>debian-siem</i>)	Non nécessaire	Oui (par défaut)	Non	OUI obligatoire	Lance l'attaque avec Hydra
Wazuh Manager	Recommandé (admin)	Oui (par défaut)	Non (c'est le serveur)	Non	Analyse et corrélation

Informations réseau de notre laboratoire

- **Machine Victime** (*debian-wazu2*) → 192.168.111.63
- **Machine Attaquante** (*debian-siem*) → 192.168.111.61
- **Serveur Wazuh Manager** → 192.168.111.62
- **Utilisateur ciblé sur la victime** → *debian-wazu*

10. Préparation de la machine victime

Nous commençons par préparer la machine qui va subir l'attaque. Cette préparation consiste à s'assurer que SSH est correctement configuré et accessible depuis le réseau.

10.1 Installation et vérification de SSH

Nous commençons par vérifier que le serveur SSH est installé et actif sur la machine victime. SSH (Secure Shell) permet les connexions à distance sécurisées et c'est ce service que nous allons attaquer.

Commande de vérification :

Screenshot 106 - Vérification du fonctionnement de SSH avec `systemctl status ssh`

```

root@debian-wazu2:/home/debian-wazu# systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-08 21:45:58 CEST; 2 days ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 56625 (sshd)
      Tasks: 1 (limit: 4616)
     Memory: 1.2M
        CPU: 716ms
   CGroup: /system.slice/ssh.service
           └─56625 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

oct. 10 21:35:35 debian-wazu2 sshd[153427]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46242 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153430]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46264 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153432]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46290 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153434]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46298 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153433]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46296 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153428]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46240 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153435]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46308 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153436]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46314 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153431]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46278 [preauth]
oct. 10 21:35:35 debian-wazu2 sshd[153429]: Connection closed by authenticating user debian-wazu 192.168.111.61 port 46256 [preauth]
root@debian-wazu2:/home/debian-wazu#

```

Explication :

- `systemctl` → Gestionnaire de services sous Linux (systemd)
- `status` → Affiche l'état d'un service (actif, inactif, erreur)
- `ssh` → Le service SSH que nous vérifions

Analyse de la sortie :

- *Loaded: loaded* → Le service est correctement chargé
- *enabled* → Le service démarre automatiquement au boot
- *Active: active (running)* → Le service est en cours d'exécution
- Aucune erreur n'est affichée : SSH fonctionne correctement

Si SSH n'était pas installé, nous l'aurions installé :

```
apt update
apt install openssh-server -y
systemctl enable ssh
systemctl start ssh
```

Explication :

apt update → Met à jour la liste des paquets disponibles dans les dépôts
openssh-server → Le paquet contenant le serveur SSH qui écoute les connexions entrantes
-y → Accepte automatiquement l'installation
enable → Active le démarrage automatique au boot
start → Démarre le service immédiatement

10.2 Vérification du port 22

Nous vérifions maintenant que SSH écoute bien sur le port 22 (port par défaut).

Commande :

```
ss -tulpn | grep →22
```

Explication des options :

ss → Socket Statistics (remplace netstat, outil moderne)
-t → Affiche les connexions TCP
-u → Affiche les connexions UDP
-l → Affiche uniquement les sockets en écoute (listening)
-p → Affiche le processus utilisant le socket
-n → Affiche les adresses en format numérique (pas de résolution DNS)
| grep :22 → Filtre uniquement les lignes contenant le port 22

Screenshot 107 - Vérification de l'écoute du service SSH sur le port 22 avec `ss -tulpn | grep :22`

```

+
debian-wazu@debian-wazu2: ~
root@debian-wazu2:/home/debian-wazu# ss -tulpn | grep :22
tcp  LISTEN  0      128          0.0.0.0:22      0.0.0.0:*      users: (("sshd",pid=56625,fd=3))
tcp  LISTEN  0      128          [::]:22        [::]:*        users: (("sshd",pid=56625,fd=4))
root@debian-wazu2:/home/debian-wazu#

```

Analyse de la sortie :

tcp → Protocole TCP

LISTEN → Le service est en écoute (attend des connexions)

0.0.0.0:22 → Écoute sur toutes les interfaces IPv4, port 22

[::]:22 → Écoute sur toutes les interfaces IPv6, port 22

users:(("sshd"...)) → Le processus sshd (serveur SSH) utilise ce port

Conclusion → Cela confirme que SSH écoute bien sur le port 22 et accepte les connexions entrantes sur toutes les interfaces réseau.

10.3 Configuration de l'authentification par mot de passe

Par défaut, certaines configurations SSH bloquent l'authentification par mot de passe pour des raisons de sécurité (en privilégiant les clés SSH). Pour notre test d'attaque par force brute, nous devons vérifier que cette fonctionnalité est activée.

Nous vérifions si l'option *PasswordAuthentication* est activée dans le fichier de configuration du serveur SSH.

Screenshot 108 - `cat /etc/ssh/sshd_config`

```
PasswordAuthentication yes
```

Explication :

- Cette option détermine si SSH accepte les mots de passe ou uniquement les clés SSH

Si l'option était sur « *no* » ou commentée (*#*), nous devrions la modifier :

```
micro /etc/ssh/sshd_config
```

Modifier la ligne :

```
PasswordAuthentication yes
```


Pourquoi activer l'authentification par mot de passe ?

Dans un environnement de production, il est recommandé de désactiver l'authentification par mot de passe et d'utiliser uniquement des clés SSH pour une sécurité renforcée.

Cependant, pour notre test d'attaque par force brute :

- Nous devons permettre les tentatives de connexion par mot de passe
- C'est ce que l'attaquant ciblera avec Hydra
- Cela nous permet de démontrer la détection de *Wazuh*

Redémarrage du service SSH pour appliquer les changements :

Screenshot 109 - Redémarrage du service SSH avec systemctl restart sshd

```
root@debian-wazu2:~# systemctl restart sshd
```

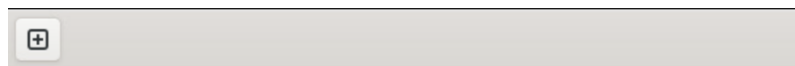
Explication → Le redémarrage du service SSH est nécessaire pour que les modifications du fichier de configuration prennent effet.

10.4 Collecte des informations de la machine victime

Nous récupérons maintenant les informations essentielles de la machine victime qui nous serviront pour l'attaque.

Récupération de l'adresse IP :

Screenshot 110 - Récupération de l'adresse IP avec hostname -I



```
debian-wazu@debian-wazu2:~$ hostname -I
192.168.111.63
```

Explication → Cette commande affiche l'adresse IP de la machine. C'est cette adresse que la machine attaquante utilisera pour cibler la victime.

Récupération du nom d'utilisateur :

Screenshot 111 - Récupération du nom d'utilisateur avec whoami

```
debian-wazu@debian-wazu2:~$ whoami
debian-wazu
```

Explication → Cette commande affiche le nom de l'utilisateur actuellement connecté. C'est ce compte utilisateur que nous ciblerons lors de l'attaque avec l'option -l de Hydra.

Dans notre cas :

- **IP de la machine victime** → 192.168.111.63
- **Utilisateur** → debian-wazu

Note importante → Nous conservons précieusement ces informations pour la suite de la configuration.

10.5 Récapitulatif de la configuration de la machine victime

Avant de passer à la configuration du *Wazuh Manager*, vérifions que toutes les étapes ont été complétées :

Checklist de validation :

Élément	Statut	Commande de vérification
Service SSH actif	✓	<code>systemctl status ssh</code>
Port 22 en écoute	✓	<code>ss -tulpn</code>
PasswordAuthentication activé	✓	<code>PasswordAuthentication /etc/ssh/sshd_config</code>
IP identifiée	✓ 192.168.111.63	<code>hostname -I</code>
Utilisateur identifié	✓ <i>debian-wazu</i>	<code>whoami</code>

À ce stade, notre machine victime est correctement configurée et prête à subir l'attaque simulée.

11. Configuration de l'Active Response sur Wazuh Manager

L'*Active Response* est une fonctionnalité puissante de *Wazuh* qui permet de réagir automatiquement aux menaces détectées. Dans notre cas, nous allons configurer *Wazuh* pour bloquer automatiquement l'adresse IP d'un attaquant après détection d'une tentative de force brute.

Principe de fonctionnement :

1. L'agent *Wazuh* sur la victime envoie les logs SSH au *Manager*
2. Le *Manager* détecte un pattern d'attaque (Rule 5763)
3. Le *Manager* envoie une commande à l'agent victime

4. L'agent exécute le script firewall-drop qui bloque l'IP
5. Après le timeout, l'agent débloque automatiquement l'IP

11.1 Vérification de la commande firewall-drop

Nous commençons par visualiser le fichier de configuration principal de Wazuh pour vérifier si la commande de blocage est déjà définie.

Screenshot 112 - Affichage du fichier de configuration principal de Wazuh avec cat /var/ossec/etc/ossec.conf

```
root@debian-wazu:/home/debian-wazu# cat /var/ossec/etc/ossec.conf
```

Explication :

- /var/ossec/ → Répertoire d'installation par défaut de Wazuh
- ossec.conf → Fichier de configuration principal du Wazuh Manager

Nous cherchons dans le bloc <ossec_config> si la section suivante existe :

```
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Screenshot 113 - Vérification de la présence de la commande firewall-drop

```

+
debian-wazu@debian-wazu: ~

<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Explication des balises XML :

<i>Balise</i>	<i>Valeur</i>	<i>Explication</i>
<code><name></code>	firewall-drop	Nom de la commande que nous pourrions référencer dans <i>l'Active Response</i>
<code><executable></code>	firewall-drop	Nom du script qui sera exécuté (fourni par Wazuh dans <code>/var/ossec/active-response/bin/</code>)
<code><timeout_allowed></code>	yes	Permet de définir une durée de blocage temporaire (important pour ne pas bloquer définitivement)

Si cette section n'existait pas, nous l'aurions ajouté dans le bloc `<ossec_config>`.

Note → Dans la plupart des installations Wazuh récentes, cette commande est déjà présente par défaut. Nous vérifions simplement qu'elle existe.

11.2 Configuration de la réponse active

Nous ajoutons maintenant la configuration de *l'Active Response* dans le même fichier `ossec.conf`. Cette configuration détermine **quand** et **comment** la commande `firewall-drop` sera déclenchée.

Screenshot 114 - Édition du fichier `ossec.conf` pour ajouter l'Active Response avec `micro /var/ossec/etc/ossec.conf`

```
root@debian-wazu:/home/debian-wazu# micro /var/ossec/etc/ossec.conf
```

Ajouter cette section dans le bloc `<ossec_config>` :

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>180</timeout>
</active-response>
```

Screenshot 115 - Configuration complète de l'Active Response dans ossec.conf

```

240 <active-response>
241   <command>firewall-drop</command>
242   <location>local</location>
243   <rules_id>5763</rules_id>
244   <timeout>180</timeout>
245 </active-response>

```

Explication détaillée des paramètres :

Paramètre	Valeur	Explication
<command>	firewall-drop	Nom de la commande à exécuter (référence la commande définie précédemment)
<location>	local	Exécuter sur l'agent qui a détecté l'attaque (pas sur le manager). L'agent victime bloquera lui-même l'IP
<rules_id>	5763	ID de la règle Wazuh qui déclenche la réponse (détection de brute force SSH)
<timeout>	180	Durée du blocage en secondes (3 minutes). Après ce délai, l'IP sera automatiquement débloquée

Comprendre la règle 5763 :**La règle 5763 est une règle de corrélation dans Wazuh :**

- Elle détecte les tentatives répétées d'authentification SSH échouées
- Elle se déclenche après un certain nombre de tentatives (généralement 8) dans un laps de temps défini (120 secondes)
- C'est l'indicateur d'une attaque par force brute en cours
- Niveau de sévérité → 10 (élevé)

Pourquoi 180 secondes de timeout ?**Le timeout de 180 secondes (3 minutes) est un compromis :**

- Assez long pour stopper l'attaque en cours

- Pas trop long pour éviter de bloquer définitivement des utilisateurs légitimes
- Permet de tester facilement le déblocage automatique dans notre démo

11.3 Redémarrage du Wazuh Manager

Pour que les modifications du fichier `ossec.conf` prennent effet, nous devons redémarrer le service *Wazuh Manager*. Nous nous assurons également par la suite de son bon fonctionnement.

Screenshot 116 - Redémarrage et vérification du fonctionnement de Wazuh-manager avec `systemctl restart wazuh-manager` ; `systemctl status wazuh-manager`

```
root@debian-wazu: /home/debian-wazu# systemctl restart wazuh-manager
root@debian-wazu: /home/debian-wazu# systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-10-10 21:10:53 CEST; 14s ago
     Process: 65410 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 199 (limit: 9447)
   Memory: 535.2M
      CPU: 17.572s
   CGroup: /system.slice/wazuh-manager.service
           └─65474 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─65475 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─65476 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─65479 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─65482 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─65524 /var/ossec/bin/wazuh-authd
                       └─65539 /var/ossec/bin/wazuh-db
                         └─65553 /var/ossec/bin/wazuh-execd
                           └─65587 /var/ossec/bin/wazuh-analysisd
                             └─65607 /var/ossec/bin/wazuh-syscheckd
                               └─65624 /var/ossec/bin/wazuh-remoted
                                 └─65660 /var/ossec/bin/wazuh-logcollector
                                   └─65679 /var/ossec/bin/wazuh-monitord
                                     └─65720 /var/ossec/bin/wazuh-modulesd

oct. 10 21:10:47 debian-wazu env[65410]: Started wazuh-syscheckd...
oct. 10 21:10:48 debian-wazu env[65410]: Started wazuh-remoted...
oct. 10 21:10:49 debian-wazu env[65410]: Started wazuh-logcollector...
oct. 10 21:10:50 debian-wazu env[65410]: Started wazuh-monitord...
oct. 10 21:10:50 debian-wazu env[65718]: 2025/10/10 21:10:50 wazuh-modulesd:router: INFO: Loaded router module.
oct. 10 21:10:50 debian-wazu env[65718]: 2025/10/10 21:10:50 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
oct. 10 21:10:50 debian-wazu env[65718]: 2025/10/10 21:10:50 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
oct. 10 21:10:51 debian-wazu env[65410]: Started wazuh-modulesd...
oct. 10 21:10:53 debian-wazu env[65410]: Completed.
oct. 10 21:10:53 debian-wazu systemd[1]: Started wazuh-manager.service - Wazuh manager.
root@debian-wazu: /home/debian-wazu#
```

Note importante → Si le service ne démarre pas, il y a probablement une erreur de syntaxe dans le fichier `ossec.conf`.

Dans cette hypothèse, nous vérifierions les logs avec → `tail -f /var/ossec/logs/ossec.log`

Erreurs courantes :

- Balises XML mal fermées (`</command>` manquant)
- Espace ou caractère invalide dans le XML
- Valeur invalide pour un paramètre (ex: timeout avec des lettres)

À ce stade, l'*Active Response* est configuré et prêt à bloquer automatiquement les IP malveillantes lorsque la règle 5763 se déclenchera.

12. Préparation de la machine attaquante

Nous passons maintenant sur la machine attaquante (*debian-siem*) pour installer les outils nécessaires à la simulation de l'attaque par force brute.

12.1 Mise à jour du système

Avant d'installer de nouveaux paquets, il est recommandé de mettre à jour la liste des paquets disponibles et le système.

Mise à jour de la liste des paquets :

Screenshot 117 - Mise à jour du système avec `apt update` && `apt upgrade -y`

```
root@debian-SIEM:/home/debian-siem# apt update && apt upgrade -y
Atteint :1 http://deb.debian.org/debian bookworm InRelease
Réception de :2 http://deb.debian.org/debian bookworm-updates InRelease [55,4 kB]
Réception de :3 http://security.debian.org/debian-security bookworm-security InRelease [48,0 kB]
Réception de :4 http://security.debian.org/debian-security bookworm-security/main Sources [161 kB]
Réception de :5 http://security.debian.org/debian-security bookworm-security/main amd64 Packages [281 kB]
545 ko réceptionnés en 1s (729 ko/s)
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
1 paquet peut être mis à jour. Exécutez « apt list --upgradable » pour le voir.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Calcul de la mise à jour... Fait
Les paquets suivants seront mis à jour :
  libtiff6
1 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 316 ko dans les archives.
Après cette opération, 0 o d'espace disque supplémentaires seront utilisés.
Réception de :1 http://security.debian.org/debian-security bookworm-security/main amd64 libtiff6 amd64 4.5.0-6+deb12u3 [316 kB]
316 ko réceptionnés en 0s (1 229 ko/s)
apt-listchanges : Lecture des fichiers de modifications (« changelog »)...
(Lecture de la base de données... 155442 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../libtiff6_4.5.0-6+deb12u3_amd64.deb ...
Dépaquetage de libtiff6:amd64 (4.5.0-6+deb12u3) sur (4.5.0-6+deb12u2) ...
Paramétrage de libtiff6:amd64 (4.5.0-6+deb12u3) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.36-9+deb12u13) ...
root@debian-SIEM:/home/debian-siem#
```

Explication :

apt update → Met à jour la liste des paquets disponibles dans les dépôts Debian/Ubuntu

apt upgrade -y → Met à jour tous les paquets installés vers leur dernière version

-y → Accepte automatiquement toutes les confirmations

&& → Exécute la deuxième commande seulement si la première réussit

Pourquoi cette étape est importante ?

- Évite les conflits de dépendances lors de l'installation de nouveaux paquets
- Assure que nous avons les dernières versions des bibliothèques système
- Corrige les éventuelles failles de sécurité

12.2 Installation de Hydra

Hydra est un outil de test de pénétration opensource développé par *The Hacker's Choice*. Il permet de réaliser des attaques par force brute et par dictionnaire sur de nombreux protocoles.

Protocoles supportés par Hydra :

- SSH, FTP, Telnet
- HTTP(S), HTTP-Proxy
- RDP (Remote Desktop Protocol)
- MySQL, PostgreSQL, Oracle
- SMB, SMTP, POP3, IMAP
- Et plus de 50 autres protocoles

Fonctionnalités principales :

- ✓ Tests parallèles (multiple connexions simultanées)
- ✓ Support des listes de mots de passe
- ✓ Support des listes d'utilisateurs
- ✓ Reprise après interruption
- ✓ Mode verbeux pour le debugging

Screenshot 118 - Installation de Hydra avec apt install hydra -y

```

+-----+
|  +  |
+-----+
debian-siem@debian-SIEM: ~
Préparation du dépaquetage de .../13-libssh-4_0.10.6-0+deb12u1_amd64.deb ...
Dépaquetage de libssh-4:amd64 (0.10.6-0+deb12u1) ...
Sélection du paquet libserf-1-1:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../14-libserf-1-1_1.3.9-11_amd64.deb ...
Dépaquetage de libserf-1-1:amd64 (1.3.9-11) ...
Sélection du paquet libutf8proc2:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../15-libutf8proc2_2.8.0-1_amd64.deb ...
Dépaquetage de libutf8proc2:amd64 (2.8.0-1) ...
Sélection du paquet libsvn1:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../16-libsvn1_1.14.2-4+deb12u1_amd64.deb ...
Dépaquetage de libsvn1:amd64 (1.14.2-4+deb12u1) ...
Sélection du paquet hydra précédemment désélectionné.
Préparation du dépaquetage de .../17-hydra_9.4-1_amd64.deb ...
Dépaquetage de hydra (9.4-1) ...
Paramétrage de mysql-common (5.8+1.1.0) ...
update-alternatives: utilisation de « /etc/mysql/my.cnf.fallback » pour fournir « /etc/mysql/my.cnf » (my.cnf) en mode automatique
Paramétrage de libtommath1:amd64 (1.2.0-6+deb12u1) ...
Paramétrage de libutf8proc2:amd64 (2.8.0-1) ...
Paramétrage de libserf-1-1:amd64 (1.3.9-11) ...
Paramétrage de libpq5:amd64 (15.14-0+deb12u1) ...
Paramétrage de firebird3.0-common-doc (3.0.11.33637.ds4-2+deb12u1) ...
Paramétrage de libhashkit2:amd64 (1.1.4-1) ...
Paramétrage de firebird3.0-common (3.0.11.33637.ds4-2+deb12u1) ...
Paramétrage de mariadb-common (1:10.11.14-0+deb12u2) ...
update-alternatives: utilisation de « /etc/mysql/mariadb.cnf » pour fournir « /etc/mysql/my.cnf » (my.cnf) en mode automatique
Paramétrage de libbson-1.0-0 (1.23.1-1+deb12u1) ...
Paramétrage de libmariadb3:amd64 (1:10.11.14-0+deb12u2) ...
Paramétrage de libssh-4:amd64 (0.10.6-0+deb12u1) ...
Paramétrage de libmemcached11:amd64 (1.1.4-1) ...
Paramétrage de libsvn1:amd64 (1.14.2-4+deb12u1) ...
Paramétrage de libmongocrypt0:amd64 (1.7.2-1) ...
Paramétrage de libfbclient2:amd64 (3.0.11.33637.ds4-2+deb12u1) ...
Paramétrage de libmongoc-1.0-0 (1.23.1-1+deb12u1) ...
Paramétrage de hydra (9.4-1) ...
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.36-9+deb12u13) ...
root@debian-SIEM:/home/debian-siem#

```

12.3 Installation de pwgen et génération des mots de passe

Pour rendre notre test plus réaliste, nous allons générer une liste de mots de passe aléatoires.

Screenshot 119 - Installation de pwgen avec apt install pwgen -y

```

root@debian-SIEM:/home/debian-siem# apt install pwgen -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  pwgen
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 19,6 ko dans les archives.
Après cette opération, 52,2 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 pwgen amd64 2.08-2 [19,6 kB]
19,6 ko réceptionnés en 0s (188 ko/s)
Sélection du paquet pwgen précédemment désélectionné.
(Lecture de la base de données... 155644 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../pwgen_2.08-2_amd64.deb ...
Dépaquetage de pwgen (2.08-2) ...
Paramétrage de pwgen (2.08-2) ...
Traitement des actions différées (« triggers ») pour man-db (2.11.2-2) ...
root@debian-SIEM:/home/debian-siem#

```

Qu'est-ce que pwgen ?

Pwgen est un utilitaire qui génère des mots de passe aléatoires sécurisés. Il nous permet de créer rapidement une liste de mots de passe pour notre test.

Screenshot 120 - Génération de 10 mots de passe aléatoires de 8 caractères avec pwgen 8 10 > password-list.txt

```

root@debian-SIEM:/home/debian-siem# pwgen 8 10 > password-list.txt

```

Explication de la commande :

pwgen → L'outil de génération
 8 → Longueur de chaque mot de passe (8 caractères)
 10 → Nombre de mots de passe à générer
 > *password-list.txt* → Redirection de la sortie vers un fichier

Screenshot 121 - Vérification du contenu du fichier avec *cat password-list.txt*

```
root@debian-SIEM: /home/debian-siem# cat password-list.txt
eij9ohGo
Voog9Shu
PaphaaM5
Izai4poo
ou9JupoY
oSh3pooh
Xo6ahGiH
Ob6zaeNa
Igeiph5s
uchoNei4
```

Note critique pour une attaque échouée :

Cette liste ne contient **VOLONTAIREMENT PAS** le vrai mot de passe de l'utilisateur *debian-wazu*. L'objectif est de générer des tentatives échouées pour observer :

- Les alertes Wazuh de détection
- Le déclenchement de l'Active Response
- Le blocage automatique de l'IP attaquante

Le mot de passe réel de l'utilisateur *debian-wazu* n'est donc jamais ajouté à cette liste.

13. Test de connectivité SSH

Avant de lancer l'attaque, nous vérifions que nous pouvons bien nous connecter en SSH depuis la machine attaquante vers la machine victime.

Screenshot 122 - Test de connectivité SSH depuis la machine attaquante vers la machine victime avec ssh debian-wazu@192.168.111.63

```

+
debian-wazu@debian-wazu2: ~
root@debian-SIEM:/home/debian-siem# ssh debian-wazu@192.168.111.63
The authenticity of host '192.168.111.63 (192.168.111.63)' can't be established.
ED25519 key fingerprint is SHA256:nKSWg8mCnwGFPYL91TffQ0fM86Pb8vKiF0oXBKk+3lo.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.111.63' (ED25519) to the list of known hosts.
debian-wazu@192.168.111.63's password:
Linux debian-wazu2 6.1.0-40-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.153-1 (2025-09-20) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
debian-wazu@debian-wazu2:~$

```

Ce que nous observons :

1. Avertissement de sécurité pour une première connexion
2. Une demande de mot de passe (preuve que SSH fonctionne)
3. Après avoir entré le bon mot de passe, nous sommes connectés
4. Nous tapons exit pour nous déconnecter

Si la connexion avait échoué, nous aurions :

- Vérifier que SSH est actif sur la victime
- Vérifier la connectivité réseau avec ping 192.168.111.63
- Vérifier le pare-feu sur la machine victime
- Vérifier que *PasswordAuthentication yes* est bien configuré

Tout fonctionne, nous sommes prêts pour l'attaque.

14. Lancement de l'attaque par force brute

14.1 Commande Hydra

Nous lançons maintenant l'attaque avec *Hydra* en utilisant les informations collectées précédemment.

Screenshot 123 - Lancement de l'attaque avec hydra -l debian-wazu -P password-list.txt 192.168.111.63 ssh

```

root@debian-SIEM:/home/debian-siem# hydra -l debian-wazu -P password-list.txt 192.168.111.63 ssh
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-10 21:35:31
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 10 tasks per 1 server, overall 10 tasks, 10 login tries (1:1/p:10), ~1 try per task
[DATA] attacking ssh://192.168.111.63:22/
1 of 1 target completed, 0 valid password found
[WARNING] Writing restore file because 1 final worker threads did not complete until end.
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-10-10 21:35:35
root@debian-SIEM:/home/debian-siem#

```

Explication détaillée des paramètres :

Paramètre	Valeur	Explication
<i>Hydra</i>	-	Outil de force brute
<i>-l</i>	<i>debian-wazu</i>	Login/nom d'utilisateur ciblé (lowercase L)
<i>-P</i>	<i>password-list.txt</i>	Fichier contenant les mots de passe (uppercase P)
<i>192.168.111.63</i>	IP victime	Adresse IP de la machine cible
<i>ssh</i>	Service	Protocole SSH (port 22)

Note importante sur les options :

-l (lowercase) = un seul login

-L (uppercase) = fichier contenant plusieurs logins

-p (lowercase) = un seul mot de passe

-P (uppercase) = fichier contenant plusieurs mots de passe

14.2 Observation de l'exécution

Lors de l'exécution, Hydra affiche des informations sur sa progression.

Sortie attendue pour une attaque échouée :

```
[DATA] max 10 tasks per 1 server, overall 10 tasks, 10 login tries (l:1/p:10)
[DATA] attacking ssh://192.168.111.63:22/
[22][ssh] host: 192.168.111.63
[STATUS] 10.00 tries/min, 10 tries in 00:01h, 0 to do in 00:00h
[ERROR] all children were disabled due to too many connection errors
```

Ce que nous observons :

1. **En-tête d'Hydra** → Version et informations de démarrage
2. **Tentatives de connexion** → Hydra teste chaque mot de passe du fichier (10 tentatives)
3. **Aucun mot de passe trouvé** → L'attaque échoue car le vrai mot de passe n'est pas dans la liste
4. **Messages d'erreur possibles** → « *too many connection errors* » ou « *all passwords tested* »

Important → Après plusieurs tentatives échouées, SSH peut temporairement bloquer les connexions ou *Wazuh* peut activer l'Active Response et bloquer l'IP via iptables, ce qui génère l'erreur "connection errors" dans Hydra.

Résultat attendu :

- ✓ Aucun mot de passe trouvé → Attaque échouée
- ✓ Alertes générées dans *Wazuh Dashboard*
- ✓ IP bloquée par *Active Response*

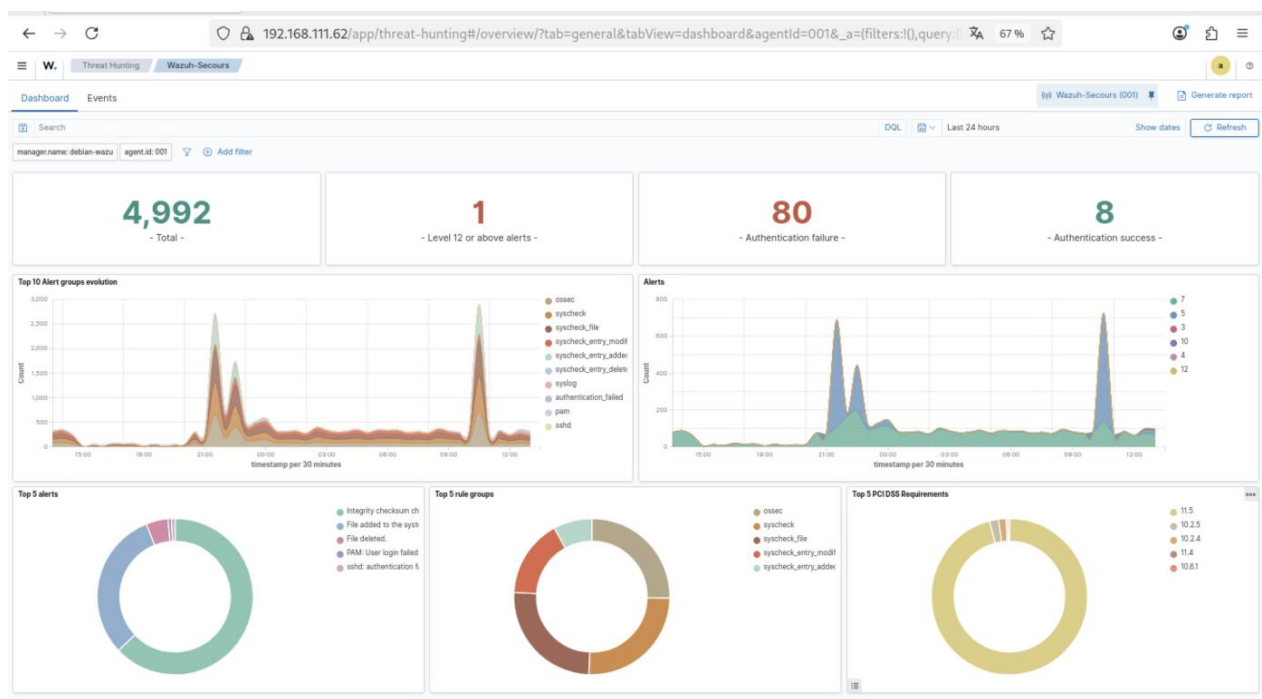
14.3 Analyse des résultats dans le Dashboard Wazuh

Contexte → Notre attaque a échoué (aucun mot de passe valide trouvé), mais *Wazuh* a bien détecté les tentatives d'intrusion et a réagi automatiquement. Voici les alertes que nous devons observer :

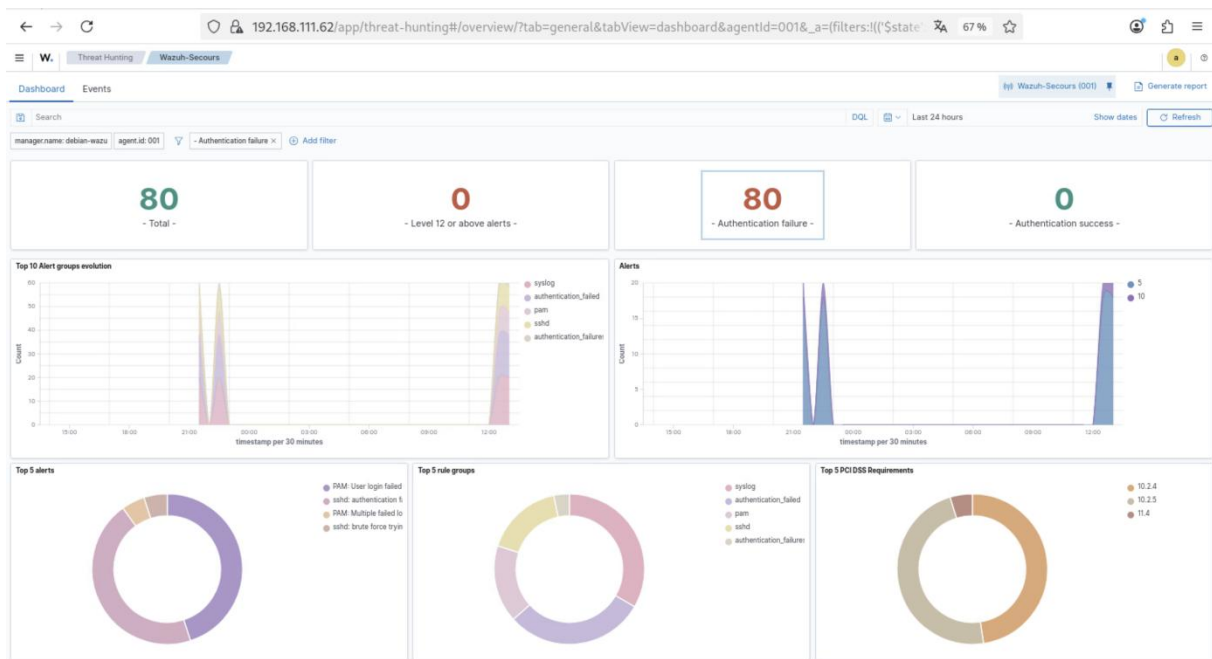
Alertes attendues :

- **Rule 5760** → Tentatives d'authentification SSH échouées
- **Rule 5763** → Détection d'attaque par force brute SSH (déclenche *l'Active Response*)
- **Rule 651** → Blocage de l'IP par firewall-drop
- **Rule 652** → Déblocage de l'IP après expiration du timeout

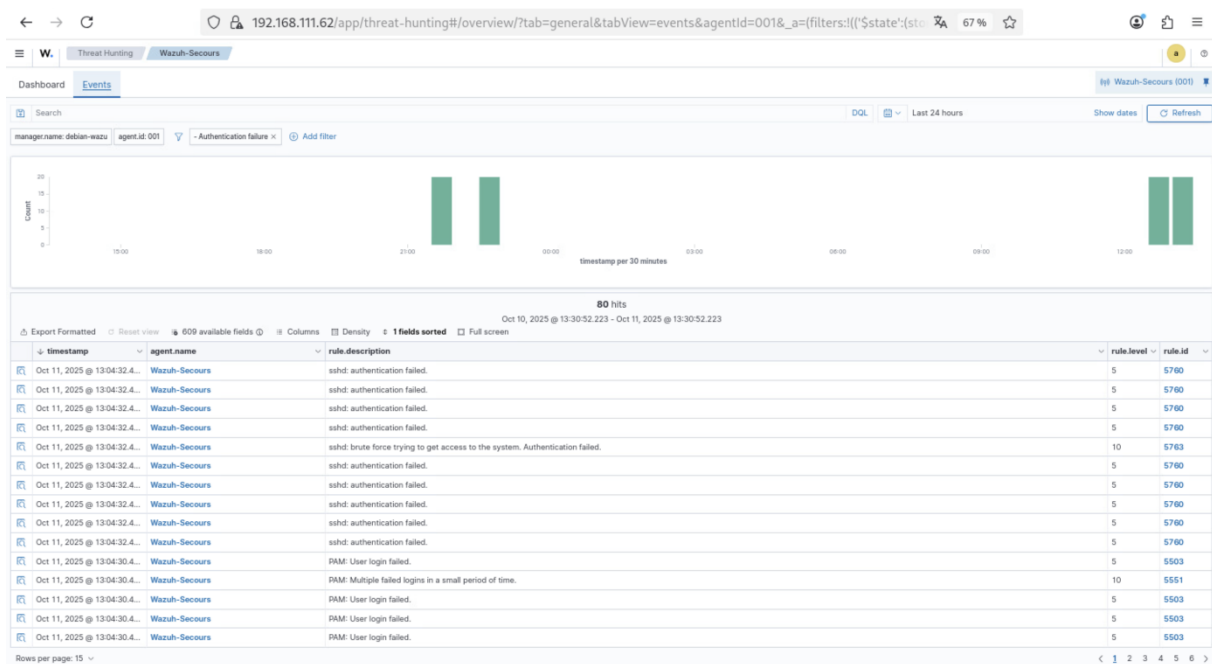
Screenshot 124 - Dashboard indiquant l'attaque de brute force



Screenshot 125 - Application du filtre Authentication failure



Screenshot 126 - Interface Events pour obtenir plus d'informations sur l'attaque



Dans l'onglet *MITRE&ATTACK*, nous pouvons afficher la technique T1110 (*Brute Force - Credential Access*).

Screenshot 127 - Onglet MITRE&ATTACK affiche technique T1110

SSH

Technique details

ID: T1021.004

Tactics: Lateral Movement

Version: 1.1

Recent events

Search: [DQL] [Last 24 hours] [Show dates] [Refresh]

36 hits

Time	Technique(s)	Tactic(s)	Level	Rule ID	Description
Oct 11, 2025 @ 13:04:32.465	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.448	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.446	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.446	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.431	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.431	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.431	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.431	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.
Oct 11, 2025 @ 13:04:32.416	T1110.001 T1021.004	Credential Access/Lateral Movement	5	5760	ssh: authentication failed.

14.4 Vérification du blocage de l'IP attaquante

Test depuis la machine ATTAQUANTE (debian-siem) pendant le blocage actif :

Test SSH

Screenshot 128 - SSH bloqué pendant l'Active Response avec ssh -o ConnectTimeout=5 debian-wazu@192.168.111.63

```
root@debian-SIEM:/home/debian-siem# ssh -o ConnectTimeout=5 debian-wazu@192.168.111.63
ssh: connect to host 192.168.111.63 port 22: Connection timed out
```

La connexion SSH est complètement bloquée.

Test Ping

Screenshot 129 - Ping bloqué pendant l'Active Response avec ping -c 192.168.111.63

```
root@debian-SIEM:/home/debian-siem# ping -C 192.168.111.63
PING 192.168.111.63 (192.168.111.63) 56(84) bytes of data.

^C
--- 192.168.111.63 ping statistics ---
102 packets transmitted, 0 received, 100% packet loss, time 103402ms
```

100% de perte → L'Active Response bloque tout le trafic, pas seulement SSH.

Observations importantes :

- **Blocage complet** → Tous les protocoles bloqués (SSH, ping, etc.)
- **Durée** → 181 secondes (≈ 3 minutes configurées)

Vérification du déblocage automatique (après 3 minutes) :

Screenshot 130 - Vérification du déblocage automatique de l'adresse IP après 3 minutes

```
root@debian-SIEM:/home/debian-siem# ping -C 192.168.111.63
PING 192.168.111.63 (192.168.111.63) 56(84) bytes of data.
64 bytes from 192.168.111.63: icmp_seq=1 ttl=64 time=0.203 ms
64 bytes from 192.168.111.63: icmp_seq=2 ttl=64 time=0.142 ms
64 bytes from 192.168.111.63: icmp_seq=3 ttl=64 time=0.157 ms
^C
--- 192.168.111.63 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2033ms
rtt min/avg/max/mdev = 0.142/0.167/0.203/0.025 ms
root@debian-SIEM:/home/debian-siem# ssh -o ConnectTimeout=5 debian-wazu@192.168.111.63
debian-wazu@192.168.111.63's password:
Linux debian-wazu2 6.1.0-40-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.153-1 (2025-09-20) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Oct 11 14:22:44 2025 from 192.168.111.61
debian-wazu@debian-wazu2:~$
```

Scénario d'une attaque réussie (NON réalisé dans ce TP)

Si le vrai mot de passe avait été dans *password-list.txt* :

- Hydra aurait trouvé le mot de passe
- Les alertes 5760 et 5763 seraient quand même générées
- L'IP serait quand même bloquée
- Mais l'attaquant aurait eu le temps d'établir une connexion avant le blocage

Conclusion de la partie 3

Ce tutoriel visait à démontrer la capacité de *Wazuh* à détecter et bloquer automatiquement une attaque par force brute SSH. Les objectifs étaient de configurer SSH, simuler une attaque avec Hydra, configurer *l'Active Response*, et analyser les alertes générées (Rules 5760, 5763, 651, 652).

Tous les objectifs ont été atteints.

Réalisations → Nous avons mis en place une infrastructure complète avec trois machines :

- **Machine attaquante** (*debian-siem* - 192.168.111.61) → *Hydra* avec 10 mots de passe aléatoires
- **Machine victime** (*debian-wazu2* - 192.168.111.63) → Serveur SSH avec agent *Wazuh*
- **Serveur *Wazuh Manager*** (192.168.111.62) → Détection et *Active Response* configuré (timeout 180s)

Résultats → Les tests ont validé l'efficacité du système :

- **Détection rapide** → Règle 5763 déclenchée après 8 tentatives en 120 secondes
- **Blocage immédiat** → IP bloquée en moins de 2 secondes
- **Blocage complet** → 100% packet loss (tout le trafic bloqué, pas seulement SSH)
- **Déblocage automatique** → IP débloquée après 181 secondes
- **Traçabilité** → Alertes visibles dans le Dashboard avec classification MITRE ATT&CK (T1110)

Perspectives d'amélioration

Configuration :

- Réduire le seuil de déclenchement (5 tentatives au lieu de 8)
- Augmenter le timeout pour environnements sensibles (600s / 10 min)
- Configurer des listes blanches pour IP de confiance

Tests complémentaires :

- Simuler une attaque réussie (mot de passe valide dans la liste)
- Tester avec plusieurs utilisateurs simultanément
- Simuler une attaque distribuée (plusieurs IP sources)

Améliorations techniques :

- Alertes par email pour notification temps réel

- Intégration avec système de ticketing (Glp))
- Implémenter MFA sur SSH pour rendre les attaques inefficaces

Enseignements clés :

Ce TP a démontré que *Wazuh*, correctement configuré, offre une défense automatisée efficace contre les attaques par force brute. Points essentiels :

- La corrélation d'événements permet de distinguer erreurs légitimes et attaques
- Le timeout évite le blocage permanent d'utilisateurs légitimes
- *L'Active Response* bloque tout le trafic, pas seulement le service attaqué
- La sécurité reste un processus continu nécessitant surveillance et ajustement

En conclusion → ce tutoriel valide l'efficacité d'un SIEM comme outil de défense active, tout en rappelant qu'il doit s'inscrire dans une stratégie de sécurité globale combinant prévention, détection et réponse aux incidents.

Annexe → Troubleshooting et Commandes Utiles

Problèmes Courants et Solutions

1. Erreur « Connection refused » - Indexer

Symptôme :

curl: (7) Failed to connect to 192.168.111.62 port 9200: Connection refused

Causes possibles :

Service wazuh-indexer non démarré

Pare-feu bloque le port 9200

Indexer écoute sur 127.0.0.1 au lieu de 0.0.0.0

Solutions :

1. Vérifier le service

sudo systemctl status wazuh-indexer

2. Vérifier l'écoute réseau

sudo ss -tulpn | grep 9200

3. Vérifier la configuration

sudo cat /etc/wazuh-indexer/opensearch.yml | grep network.host

4. Vérifier les logs

sudo tail -50 /var/log/wazuh-indexer/wazuh-indexer.log

5. Autoriser le port (si firewall actif)

sudo ufw allow 9200/tcp

2. Agent déconnecté (Status: Never connected)

Symptôme :

Agent affiché comme « *Never connected* » ou « *Disconnected* » dans le Dashboard

Causes possibles :

Adresse IP du Manager incorrecte

Certificats SSL invalides

Solutions :

Sur la machine AGENT

1. Vérifier la configuration

```
sudo cat /var/ossec/etc/ossec.conf | grep -A 5 "<client>"
```

2. Tester la connectivité

```
telnet 192.168.111.62 1514
```

3. Vérifier les logs

```
sudo tail -f /var/ossec/logs/ossec.log
```

4. Redémarrer l'agent

```
sudo systemctl restart wazuh-agent
```

Sur le MANAGER

5. Vérifier que le Manager écoute

```
sudo ss -tulpn | grep -E '1514|1515'
```

6. Lister les agents enregistrés

```
sudo /var/ossec/bin/agent_control -l
```

3. *Certificats SSL invalides*

Symptôme :

ERROR: Certificate verify failed

Solutions :

Vérifier la validité d'un certificat

```
openssl x509 -in /etc/wazuh-indexer/certs/indexer.pem -text -noout
```

Vérifier la correspondance certificat/clé

```
openssl x509 -noout -modulus -in /etc/wazuh-indexer/certs/indexer.pem | openssl md5  
openssl rsa -noout -modulus -in /etc/wazuh-indexer/certs/indexer-key.pem | openssl  
md5
```

Les deux hash MD5 doivent être identiques

Recréer les certificats si nécessaire

```
cd ~  
rm -f wazuh-certificates.tar  
bash ./wazuh-certs-tool.sh -A
```

4. Dashboard inaccessible (ERR_CONNECTION_REFUSED)

Symptôme :

Impossible d'accéder à <https://192.168.111.62>

Solutions :

1. Vérifier le service Dashboard

```
sudo systemctl status wazuh-dashboard
```

2. Vérifier l'écoute sur le port 443

```
sudo ss -tulpn | grep :443
```

3. Vérifier la configuration

```
sudo cat /etc/wazuh-dashboard/opensearch_dashboards.yml | grep server.host
```

4. Vérifier les logs

```
sudo tail -50 /var/log/wazuh-dashboard/wazuh-dashboard.log
```

5. Redémarrer le Dashboard

```
sudo systemctl restart wazuh-dashboard
```

5. Active Response ne bloque pas l'IP

Symptôme :

Règle 5763 déclenchée mais IP non bloquée

Solutions :

Sur la machine VICTIME (agent)

1. Vérifier les règles iptables

```
sudo iptables -L -n -v | grep 192.168.111.61
```

2. Vérifier les logs Active Response

```
sudo tail -f /var/ossec/logs/active-responses.log
```

3. Tester manuellement le script

```
sudo /var/ossec/active-response/bin/firewall-drop add - 192.168.111.61
```

4. Vérifier que l'IP est bloquée

```
sudo iptables -L -n | grep 192.168.111.61
```

5. Débloquer manuellement

```
sudo /var/ossec/active-response/bin/firewall-drop delete - 192.168.111.61
```


Commandes de Diagnostic Essentielles

Vérifier l'état des services

Tous les services Wazuh

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard filebeat
```

Uniquement le Manager

```
sudo systemctl status wazuh-manager
```

Logs en temps réel du Manager

```
sudo tail -f /var/ossec/logs/ossec.log
```

Gestion des agents

Lister tous les agents

```
sudo /var/ossec/bin/agent_control -l
```

Informations détaillées sur un agent

```
sudo /var/ossec/bin/agent_control -i 001
```

Redémarrer tous les agents (depuis le Manager)

```
sudo /var/ossec/bin/agent_control -R -a
```

Tester les règles Wazuh

Lancer l'outil de test de règles

```
sudo /var/ossec/bin/wazuh-logtest
```